DNSR: A Security-Hardened Iterative DNS Resolver Built from Scratch in Go

BCSE308P – COURSE PROJECT

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Computer Science and Engineering (Cyber Security)

in

Department of Cybersecurity and Intelligent Systems

by

Student Name(s)

Register Number(s)



School of Computer Science Engineering and Information Systems [SCORE]

VIT, Vellore

November, 2026

November, 2026

DECLARATION

I hereby declare that the BCSE308P – Course Project entitled "DNSR: A Security-Hardened Iterative DNS Resolver Built from Scratch in Go" submitted by me, for the award of the degree of *Bachelor of Computer Science and Engineering (Cyber Security)*, School of Computer Science Engineering and Information Systems to VIT is a record of bonafide work carried out by me under the supervision of Faculty Guide Name, Designation, SCORE, VIT, Vellore.

I further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Vellore

Date:

Signature of the Candidate

CERTIFICATE

This is to certify that the **BCSE308P – Course Project** entitled "DNSR: A Security-Hardened Iterative DNS Resolver Built from Scratch in Go" submitted by Student Name & Reg. No., SCORE, VIT, for the award of the degree of *Bachelor of Computer Science and Engineering (Cyber Security)*, School of Computer Science Engineering and Information Systems, is a record of bonafide work carried out under my supervision during the period dd.mm.yyyy to dd.mm.yyyy, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The project fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place: Vellore

Date:

Signature of the VIT-SCORE – Guide

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, Faculty Guide Name, Designation, School of Computer Science Engineering and Information Systems (SCORE), VIT, Vellore, for the guidance, encouragement and valuable suggestions given throughout this project.

I thank the Dean of SCORE, the Head of the Department of Cybersecurity and Intelligent Systems and all the faculty members of the department for providing the knowledge, facilities and environment needed to carry out this work. I am grateful to the management of Vellore Institute of Technology for the opportunity to undertake this course project.

I also acknowledge the authors of the Internet standards and of the open-source tools on which this project builds, and the operators of the public DNS infrastructure whose servers answered the many queries sent during testing.

Finally, I thank my family and friends for their constant support and encouragement.

Student Name

EXECUTIVE SUMMARY

The Domain Name System (DNS) turns names into addresses for almost every Internet connection, which makes it a prime target for spoofing and cache-poisoning attacks. Yet the recursive resolver, the component that walks the DNS tree and decides which answers to trust, is usually hidden inside large production systems or libraries.

This project presents DNSR, an iterative DNS resolver written from scratch in Go using only the standard library. DNSR encodes and decodes the RFC 1035 message format by hand, resolves names starting from the root servers, follows referrals and glue, resolves glueless delegations, chases CNAMEs, separates NXDOMAIN from NODATA, retries truncated replies over TCP, and caches answers and delegations. It defends against forgery with random query IDs and source ports, strict reply matching and bailiwick checks on referrals and glue, and it runs as a traced command-line tool or a recursive server.

Nine offline tests against a fake DNS hierarchy, including hostile servers, all pass. Live, DNSR resolved 15 popular domains with the same response codes as Google Public DNS (median 571 ms cold, under 1 ms cached), and 34.7 million fuzzing inputs caused no parser failure.

CONTENTS

		Page No.
	Acknowledgement	iv
	Executive Summary	v
	Table of Contents	vi
	List of Figures	viii
	List of Tables	x
	Abbreviations	xi
	Symbols and Notations	xiii
1	**INTRODUCTION**	1
	1.1 Objective	2
	1.2 Motivation	3
	1.3 Background	4
2	**PROJECT DESCRIPTION AND GOALS**	8
	2.1 Literature Review	8
	2.2 Gap Identification	11
	2.3 Problem Statement	12
	2.4 Project Goals	12
	2.5 Scope of the Project	13
3	**TECHNICAL SPECIFICATION**	14
	3.1 Functional Requirements	14
	3.2 Security Requirements	14
	3.3 Non-Functional Requirements	15
	3.4 Hardware and Software Specification	15
	3.5 DNS Message Format	16
	3.6 Operating Limits and Constants	18
	3.7 Command-Line Interface Specification	19
4	**DESIGN APPROACH AND DETAILS**	20
	4.1 Design Approach / Materials and Methods	20
	4.2 Codes and Standards	33
	4.3 Constraints, Alternatives and Trade-offs	34

5 SCHEDULE, TASKS AND MILESTONES 36
5.1 Task Breakdown 36
5.2 Gantt Chart 36
5.3 Milestones 37
5.4 Risk Management 37

6 PROJECT DEMONSTRATION 39
6.1 Sample Codes 39
6.2 Sample Screen Shots 50

7 COST ANALYSIS / RESULT AND DISCUSSION 59
7.1 Experimental Setup 59
7.2 Functional Verification 59
7.3 Live Resolution: Correctness and Latency 60
7.4 Effect of Caching 62
7.5 Record Types and Special Cases 63
7.6 Server-Mode Evaluation 64
7.7 Security and Robustness Evaluation 65
7.8 Codec Performance 67
7.9 Security Review Findings 68
7.10 Achievement of Goals 69
7.11 Cost Analysis 69
7.12 Discussion 70

8 SUMMARY 71
8.1 Findings and Results 71
8.2 Recommendations 71
8.3 Future Work 72

9 REFERENCES 73

Appendix A – Complete Source Code 76

Appendix B – User Manual (README) 95

LIST OF FIGURES

Figure No.	Title	Page No.
1.1	Part of the DNS Namespace Used in This Project	5
1.2	Recursive Query from a Stub and Iterative Queries from the Resolver	6
2.1	Kaminsky-Style Cache Poisoning by an Off-Path Attacker	9
3.1	Layout of a DNS Message and Its 12-Byte Header	16
3.2	Name Compression: a Pointer to the Question Name	18
4.1	System Architecture of DNSR	21
4.2	Iterative Resolution Flowchart	24
4.3	Sequence of Messages for a Cold-Cache Lookup of www.github.com	26
4.4	Bailiwick Decisions for Replies from a .com Server	29
4.5	Structure of the Recursive Server Mode	31
4.6	Offline Test Hierarchy on Loopback Addresses	32
5.1	Gantt Chart of the Project Schedule	37
6.1	Building DNSR and Running the Offline Test Suite	50
6.2	Usage Message Printed When No Name Is Given	51
6.3	Resolving www.github.com from the Root Servers	51
6.4	MX Lookup for gmail.com	52
6.5	AAAA Lookup for www.wikipedia.org Through a CNAME	52
6.6	NXDOMAIN and NODATA Responses	53
6.7	Glueless Delegation for github.io	54
6.8	Glueless Delegation and NS Lookup for vit.ac.in	54
6.9	Truncated UDP Reply Retried over TCP for apple.com TXT	55
6.10	Delegation Cache and Answer Cache in One Run	56
6.11	Quiet Mode Printing Only the Answers	56
6.12	DNSR Running as a Recursive Server	57
6.13	A DNS Client Querying the DNSR Server	57
6.14	Random-Packet Test, Benchmarks and Coverage-Guided Fuzzing	58
7.1	Cold-Cache Resolution Time per Domain	61
7.2	Network Queries per Cold-Cache Resolution	62
7.3	Effect of the Delegation Cache and the Answer Cache	63

7.4	Server Mode: Latency of First and Repeated Queries	65
7.5	Probability of a Successful Blind Spoof as a Function of Forged Packets	66
7.6	Outcome of One Million Random and Mutated Packets	67

LIST OF TABLES

Table No.	Title	Page No.
1.1	Record Types Supported by DNSR	6
2.1	Comparison of Existing DNS Resolution Approaches	11
2.2	Project Goals and How They Are Measured	12
3.1	Functional Requirements	14
3.2	Security Requirements	14
3.3	Development and Test Environment	15
3.4	Header Fields and Their Use in DNSR	17
3.5	Resolver Constants	18
3.6	Command-Line Usage and Flags	19
4.1	Module Description	22
4.2	Main Data Structures	22
4.3	Threat Model	27
4.4	Test Cases of the Offline Suite	32
4.5	Standards Followed	33
4.6	Design Alternatives and Decisions	34
5.1	Task Breakdown and Schedule	36
5.2	Project Milestones	37
5.3	Risks and Mitigations	37
7.1	Offline Test Results	59
7.2	Live Cold-Cache Resolution of 15 Domains (Three Runs Each)	60
7.3	Latency with Empty, Delegation and Answer Caches (ms, hops in brackets)	62
7.4	Live Results for Each Supported Record Type	63
7.5	Server Mode: First and Repeated Query (ms)	64
7.6	Fuzzing Results for ParseMessage	67
7.7	Benchmark Results (Intel Core i7-13650HX)	67
7.8	Findings of the Security Review	68
7.9	Goals and Results	69
7.10	Cost of Resources	69

LIST OF ABBREVIATIONS

AA	Authoritative Answer (header flag)
AAAA	IPv6 address record
API	Application Programming Interface
CDN	Content Delivery Network
CLI	Command-Line Interface
CNAME	Canonical Name record
DNS	Domain Name System
DNSSEC	DNS Security Extensions
DoH	DNS over HTTPS
DoT	DNS over TLS
EDNS(0)	Extension Mechanisms for DNS, version 0
FQDN	Fully Qualified Domain Name
IANA	Internet Assigned Numbers Authority
ID	Transaction Identifier
IETF	Internet Engineering Task Force
IP	Internet Protocol
ISP	Internet Service Provider
mDNS	Multicast DNS
MTU	Maximum Transmission Unit
MX	Mail Exchanger record
NODATA	Name exists but has no records of the requested type
NS	Name Server record
NXDOMAIN	Non-Existent Domain (response code 3)
OPT	EDNS(0) option pseudo-record
OS	Operating System
PTR	Pointer record (reverse lookup)
QR	Query/Response (header flag)
RA	Recursion Available (header flag)
RCODE	Response Code
RD	Recursion Desired (header flag)
RDATA	Resource Record Data
RFC	Request for Comments
RR	Resource Record
RRset	Resource Record Set

RTT	Round-Trip Time
SERVFAIL	Server Failure (response code 2)
SLD	Second-Level Domain
SOA	Start of Authority record
TC	Truncation (header flag)
TCP	Transmission Control Protocol
TLD	Top-Level Domain
TTL	Time To Live
TXT	Text record
UDP	User Datagram Protocol

SYMBOLS AND NOTATIONS

P	Probability that a blind spoofing attempt succeeds
N	Number of equally likely (ID, port) combinations
k	Number of forged replies that arrive before the genuine reply
Rport	Size of the ephemeral source-port range
$t$exp	Absolute expiry time of a cached RRset
$t$store	Time at which an RRset was stored in the cache
$t$now	Current time
TTLi	Time to live of the i-th record of an RRset (seconds)
TTLrem	Remaining time to live returned on a cache hit
Li	Latency of the i-th resolution (ms)
$\bar{L}$	Mean latency over n resolutions (ms)
Lcold, Lcached	Latency with an empty cache and with a warm cache
ρ	Relative latency reduction due to caching (%)
n	Number of measured resolutions
$\lfloor x \rfloor$	Floor of x (largest integer not greater than x)

CHAPTER 1

INTRODUCTION

The Domain Name System (DNS) is the naming service of the Internet. Every time a browser opens a web page, a mail server delivers a message or a phone application contacts its back-end, a DNS lookup first turns a human-readable name such as `www.github.com` into the numeric IP address that the network actually routes on. Because almost every connection starts with such a lookup, DNS is one of the most heavily used and most security-critical protocols in existence. If an attacker can change the answer to a DNS question, the attacker can silently redirect users to a server of their choice, without breaking any encryption and without touching the victim's computer.

Despite its importance, the inner working of DNS is invisible to most programmers and students. Applications call a library function such as `getaddrinfo()` or Go's `net.LookupHost()`, the operating system forwards the question to a resolver run by an Internet Service Provider (ISP) or a public service such as 8.8.8.8, and an answer comes back a few milliseconds later. The part that does the real work, the *recursive resolver* that walks the DNS tree from the root servers down to the authoritative server of the domain, is hidden inside large production systems such as BIND or Unbound. The rules by which a resolver decides which data to trust, which are exactly the rules that attackers try to break, are therefore rarely seen, tested or understood.

This project, **DNSR**, opens that black box. DNSR is an iterative (recursive) DNS resolver written from scratch in the Go programming language, using only Go's standard library. No DNS library is used: every byte of the binary message format defined in RFC 1035 is encoded and decoded by hand. Starting from the thirteen root server addresses published by the Internet Assigned Numbers Authority (IANA), DNSR sends non-recursive queries, follows referrals level by level, chases aliases, handles glueless delegations and falls back to TCP when a reply is too large for UDP. Along the way it applies the defensive checks that protect a resolver against spoofing and cache poisoning, and it prints a readable trace of every step so that the resolution process can be observed and studied.

DNSR can be used in two ways: as a command-line tool that resolves one or more names and shows the full trace, or as a recursive DNS server that standard DNS clients can query. Its behaviour is verified by an offline test suite that builds a complete fake DNS hierarchy on loopback addresses, by live measurements against the real Internet, and by fuzz testing of the packet parser. This report describes the motivation, design, implementation, testing and results of the project.

1.1 Objective

The primary objective of this project is to design, implement and evaluate a small, transparent and security-hardened recursive DNS resolver that resolves names directly from the root servers, without relying on any upstream resolver or DNS library. The specific objectives are listed below.

- ✓ To implement an RFC 1035 compliant encoder and decoder for DNS messages, including the header, question and resource-record sections, domain-name compression and the EDNS(0) OPT pseudo-record, and to support the A, AAAA, NS, CNAME, SOA, PTR, MX and TXT record types.
- ✓ To implement iterative resolution starting from the IANA root hints, including referral following, use of glue records, resolution of glueless delegations, CNAME chasing, and correct handling of NXDOMAIN and NODATA responses.
- ✓ To harden the resolver against spoofing and cache poisoning by using random transaction IDs, strict matching of each reply against its query, bailiwick checks on both referrals and glue records, and loop protection in the packet parser.
- ✓ To improve performance with a TTL-aware answer cache and a delegation (zone to name-server) cache, together with automatic fallback from UDP to TCP for truncated responses.
- ✓ To provide two modes of operation: a command-line interface that prints a human-readable resolution trace, and a recursive DNS server mode that can be queried by standard DNS clients.
- ✓ To verify correctness and robustness through an automated offline test suite, live measurements against Internet domains, and fuzz testing of the message parser, and to analyse the remaining security gaps.

1.2 Motivation

DNS cache poisoning is one of the oldest and most damaging attacks on Internet infrastructure. In 2008 Dan Kaminsky showed that an off-path attacker could poison the cache of almost any recursive resolver within seconds by asking it for random, non-existent names and racing forged replies against the genuine ones [9]. The forged replies did not carry a fake address for the queried name; instead they carried a forged *referral* and forged *glue* records that redirected the entire domain to the attacker. The attack forced an emergency, industry-wide patch of resolver software. The lessons learned, randomise everything the attacker must guess and accept only records that the replying server is entitled to give, are now codified in RFC 5452 [8] and in the data-ranking rules of RFC 2181 [4].

These defences are simple to state but easy to get wrong, and research continues to find new ways around them. Son and Shmatikov [10] showed that weak bailiwick rules in deployed resolvers still allowed poisoning, and the SAD DNS attack of 2020 [11] used a side channel in the operating system to infer randomised source ports, bringing Kaminsky-style poisoning back to modern resolvers. The NXNSAttack [24] showed that the way a resolver follows referrals can be abused to amplify traffic towards victim servers. For a student of cyber security, the best way to understand these attacks is to build a resolver that must defend against them and to see exactly where each check sits in the code.

A second motivation is education and transparency. Tools such as `dig +trace` show the referral chain but do not cache or validate anything, while production resolvers are hundreds of thousands of lines of C code. A compact resolver of about one thousand lines, which prints each referral, each glue decision and each cache hit, makes the protocol concrete. It can be used in class to demonstrate how DNS works, in the laboratory to reproduce attacks against a fake DNS hierarchy, and as a base for further research such as DNSSEC validation or privacy features.

Finally, the project is motivated by the value of writing protocol code from first principles. Parsing untrusted binary data received from the network is a classic source of vulnerabilities: buffer over-reads, infinite loops caused by compression pointers and crashes caused by inconsistent length fields. Implementing the parser by hand in a memory-safe language, and then attacking it with millions of random and mutated

packets, provides practical experience in secure coding that a library-based approach would hide.

1.3 Background

This section introduces the DNS concepts that are needed to follow the rest of the report: the namespace, the different kinds of DNS servers, the difference between recursive and iterative queries, the message format and the most important record types.

1.3.1 The DNS namespace and zones

The DNS namespace is a tree. The root of the tree is written as a single dot. Below the root are the top-level domains (TLDs) such as `com.`, `org.`, `in.` and `io.`; below them are second-level domains such as `github.com.` and `ac.in.`, and so on. Each node is identified by its *fully qualified domain name* (FQDN), which lists the labels from the node up to the root, for example `www.github.com.`. Each label may be at most 63 bytes long and a complete name at most 255 bytes [2].

Administration of the tree is divided into *zones*. A zone is a part of the tree that is managed by one organisation and served by a set of authoritative name servers. The root zone delegates each TLD to the TLD operator, the `com.` zone delegates `github.com.` to GitHub's name servers, and so on. A delegation is expressed by NS (name server) records in the parent zone. Fig. 1.1 shows part of the tree with the zones that appear in the experiments of this project.

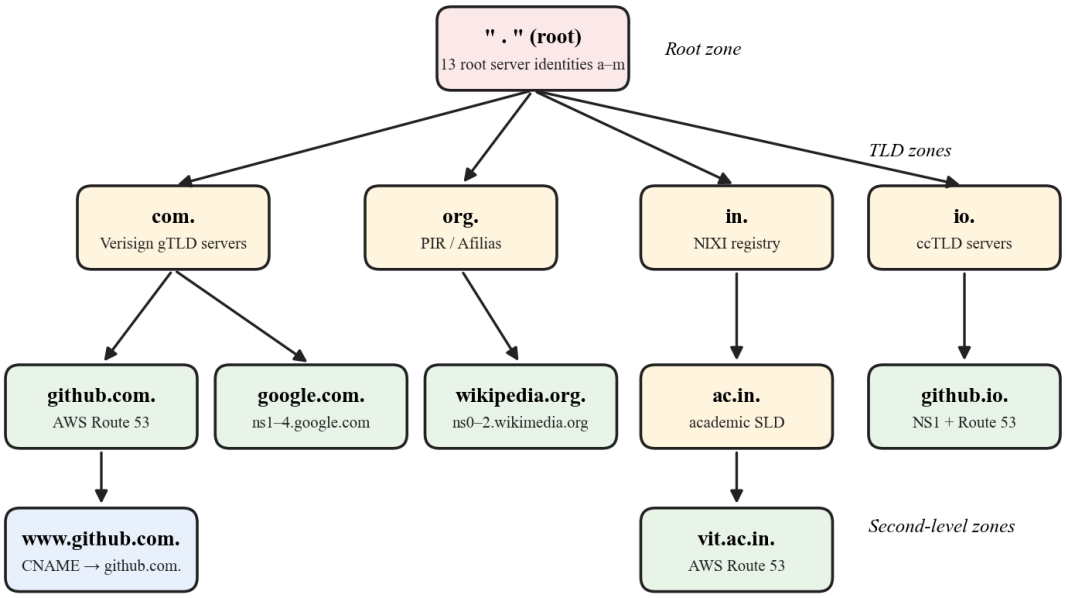


Fig. 1.1 Part of the DNS Namespace Used in This Project

1.3.2 Stub, recursive and authoritative servers

Three kinds of DNS software cooperate to answer a question. A *stub resolver* is the small client inside the operating system; it sends the question to a recursive resolver with the Recursion Desired (RD) flag set and waits for the final answer. A *recursive resolver* accepts such questions and does all the work: it asks the root servers, the TLD servers and finally the authoritative servers, following referrals until it obtains an answer, which it caches and returns to the stub. An *authoritative server* holds the actual data of a zone and answers only for that zone; for names outside its zone it either refers the resolver to a server lower in the tree or refuses.

The queries that a recursive resolver sends to authoritative servers are *iterative* (non-recursive): the RD flag is cleared, and each server answers with what it knows, either the final data or a referral to the next zone down. Fig. 1.2 shows the eight messages exchanged when a stub asks for `www.github.com` and the recursive resolver starts with an empty cache. DNSR plays the role of the recursive resolver in this figure.

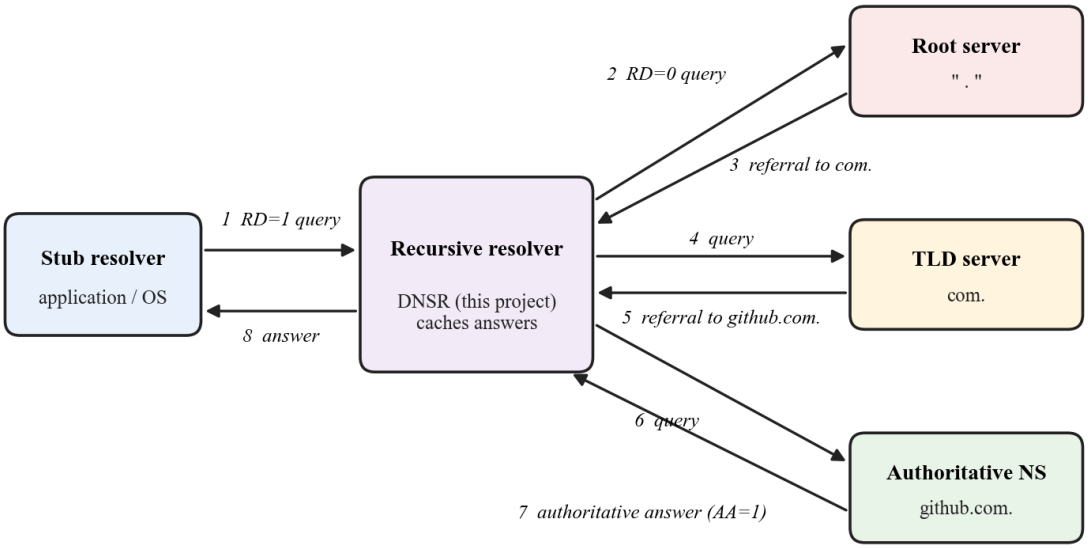


Fig. 1.2 Recursive Query from a Stub and Iterative Queries from the Resolver

1.3.3 Resource records and caching

All DNS data is stored as *resource records* (RRs). Every RR has an owner name, a type, a class (almost always IN for Internet), a time to live (TTL) in seconds and type-specific data called RDATA. Records with the same owner, type and class form an RRset. The TTL tells a resolver how long it may cache the RRset; a well-behaved resolver counts the TTL down while the record sits in the cache and returns the remaining TTL to its clients. Table 1.1 lists the record types handled by DNSR.

Table 1.1 Record Types Supported by DNSR

Type	Code	Meaning	RDATA handled by DNSR
A	1	IPv4 address of a host	4-byte address printed in dotted form
NS	2	Authoritative name server of a zone	Domain name (may be compressed)
CNAME	5	Canonical name: the owner is an alias	Domain name of the target
SOA	6	Start of authority: zone parameters	MNAME, RNAME and five 32-bit timers
PTR	12	Pointer, used for reverse lookups	Domain name
MX	15	Mail exchanger for a domain	16-bit preference and a domain name
TXT	16	Free-form text strings	One or more length-prefixed strings
AAAA	28	IPv6 address of a host	16-byte address printed in IPv6 form
OPT	41	EDNS(0) pseudo-record (not data)	Encoded in queries; skipped in replies

1.3.4 Why DNS is a target

Classic DNS runs over UDP and carries no authentication. A reply is accepted if it arrives at the right port, carries the right 16-bit transaction ID and answers the right question. An attacker who is not on the network path between the resolver and the server (an *off-path* attacker) cannot see the query, but can send many forged replies that guess these values. If one forged reply arrives before the genuine one and is accepted, its records enter the cache and are served to every client of the resolver until they expire. The defences against this are discussed in detail in Chapter 2 and their implementation in Chapter 4. Cryptographic protection is available through DNSSEC [13], but DNSSEC validation is a large addition and is outside the scope of this project; it is listed as future work.

CHAPTER 2

PROJECT DESCRIPTION AND GOALS

This chapter reviews the literature on which the project builds, identifies the gap that it addresses, states the problem formally and sets out the goals and scope of the work.

2.1 Literature Review

2.1.1 Foundations of the Domain Name System

The DNS was designed by Mockapetris and standardised in RFC 1034 and RFC 1035 [1], [2]. RFC 1034 describes the concepts: the hierarchical namespace, zones and delegation, the roles of resolvers and name servers, and the algorithm a resolver follows. RFC 1035 specifies the implementation, in particular the binary message format with its fixed 12-byte header and four variable sections, the encoding of names as sequences of length-prefixed labels and the compression scheme that replaces repeated name suffixes with 14-bit pointers. Mockapetris and Dunlap [3] later described the design goals behind the system, namely scalability, distributed administration and aggressive caching, and the lessons learned from its first years of operation.

RFC 2181 [4] clarified several ambiguous parts of the original specification. Most important for this project is its section on the *ranking of data*: data from the answer section of an authoritative reply is trusted more than data from the authority or additional section, and a resolver must not let lower-ranked data overwrite higher-ranked data. This ranking is the formal basis of the bailiwick rule used by modern resolvers. The current terminology of DNS, including the terms *in-bailiwick*, *glue* and *referral*, is collected in RFC 8499 [26].

2.1.2 Protocol extensions and transport

RFC 1035 limited UDP messages to 512 bytes. As DNS answers grew, especially with IPv6 addresses, long TXT records and DNSSEC signatures, this limit became too small. The Extension Mechanisms for DNS, EDNS(0), defined in RFC 6891 [6], allow a client to place an OPT pseudo-record in the additional section to advertise a larger UDP buffer. Large UDP packets, however, are fragmented by IP and fragments are often dropped by firewalls. The DNS Flag Day 2020 initiative [20] therefore recommended a buffer size of 1232 bytes, which fits in the minimum IPv6

path MTU without fragmentation. If a reply is still too large, the server sets the TC (truncation) bit and the client must repeat the query over TCP, where each message is preceded by a two-byte length field. RFC 7766 [7] made TCP support mandatory for all DNS implementations.

Negative answers come in two forms. NXDOMAIN (name error) means that the queried name does not exist at all; NODATA means that the name exists but has no records of the requested type. Both are recognised by the SOA record that the authoritative server places in the authority section, and RFC 2308 [5] specifies how they may be cached. RFC 8020 [27] adds that an NXDOMAIN for a name implies that no names exist below it.

2.1.3 Cache poisoning and its defences

Because classic DNS is unauthenticated, a resolver can be tricked into caching forged records. Early attacks simply guessed the transaction ID, which many implementations generated sequentially. Kaminsky [9] showed in 2008 that the attack becomes far more powerful when the attacker forces the resolver to ask for many different non-existent names (so that no cached answer stops the race) and places a forged NS record and forged glue in each fake reply, so that one success hijacks a whole domain. Fig. 2.1 illustrates the attack.

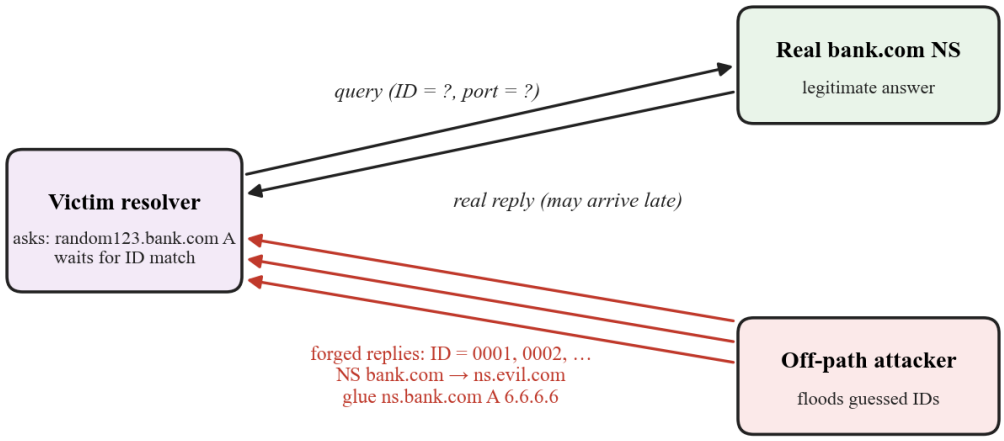


Fig. 2.1 Kaminsky-Style Cache Poisoning by an Off-Path Attacker

RFC 5452 [8] summarises the defences that followed. A resolver must use an unpredictable transaction ID and an unpredictable UDP source port, must accept a reply

only if its source address, ID and question exactly match an outstanding query, and should treat out-of-zone records with suspicion. Dagon et al. [28] proposed adding further entropy by randomising the upper and lower case of letters in the query name (the 0x20 technique), and DNS Cookies (RFC 7873) [29] add a lightweight shared secret between client and server. Son and Shmatikov [10] systematically examined how real resolvers accept records and showed that weak or missing bailiwick checks allow poisoning even when IDs and ports are random. Man et al. [11] presented SAD DNS, which uses an ICMP rate-limit side channel to discover the randomised source port, demonstrating that cache poisoning remains an active research area. The broader threat analysis of DNS is given in RFC 3833 [25].

Resolvers can also be abused as attack amplifiers. The NXNSAttack [24] uses referrals that list many name servers without glue; a resolver that eagerly resolves the address of every listed name server sends a large number of queries for each query the attacker makes. This shows that the referral-following logic must limit how much work a single upstream reply can trigger.

2.1.4 Performance, caching and privacy

Jung et al. [12] studied real DNS traces and found that caching of NS records, which lets a resolver skip the root and TLD servers on later queries, is what most reduces load and latency, while the benefit of caching A records levels off quickly because of short TTLs. This result directly supports the separate delegation cache used in DNSR. On the privacy side, QNAME minimisation [14] reduces how much of the query name each upstream server sees, and DNS over TLS [15] and DNS over HTTPS [16] encrypt the path between the stub and the recursive resolver. These measures protect privacy but do not replace the integrity checks a recursive resolver must perform on the path to the authoritative servers. DNSSEC [13] adds data-origin authentication through digital signatures, but deployment is still incomplete and a validating resolver must still perform all of the checks above for unsigned zones.

2.1.5 Existing implementations

Production resolvers such as BIND 9 [18] and Unbound [17] implement the full protocol, including DNSSEC validation, but they are large C code bases with hundreds of thousands of lines, which makes them hard to study for learning or research. In the Go ecosystem, the `miekg/dns` library [19] provides message parsing and is widely

used, but it hides the wire format behind abstractions, and Go's standard `net.Resolver` is only a stub that delegates recursion to an upstream server. Online tools and `dig +trace` show the referral chain but do not cache or validate anything. Table 2.1 compares these approaches with the proposed work.

Table 2.1 Comparison of Existing DNS Resolution Approaches

Approach	Iterative from root	Hand-written wire format	Explicit security checks	Suitability for learning / research
OS stub / Go net.Resolver	No (delegates)	Hidden	Relies on upstream	Low: a black box
miekg/dns library	Only if the user builds it	No (library)	Left to the user	Medium
BIND 9 / Unbound	Yes	Yes	Yes, including DNSSEC	Low: very large code base
dig +trace / online tools	Yes (trace only)	Hidden	No caching or validation	Medium: observation only
DNSR (this project)	Yes	Yes, about 1,000 lines	ID and question match, bailiwick, loop limit	High: small, traced and tested

2.2 Gap Identification

The literature and the existing tools leave the following gaps, which this project sets out to close.

- ✓ **Opacity of existing tools.** Stub resolvers and libraries such as Go's `net.Resolver` hand recursion to an upstream server, so the referral chain, glue handling and caching decisions cannot be seen.
- ✓ **Complexity of production resolvers.** BIND and Unbound are complete but very large, which makes it impractical to trace one specific behaviour, for example glue acceptance, for learning or experiments.
- ✓ **Security checks are rarely made explicit.** Bailiwick rules for referrals and glue, together with reply matching, are the main defence against Kaminsky-style poisoning, but most teaching material and simple implementations leave them out.

- ✓ **Lack of offline, repeatable testing.** Resolution behaviour depends on live Internet servers that change over time, which makes edge cases such as glueless delegation, poisoned glue and truncation hard to reproduce.
- ✓ **Library dependence hides the wire format.** Relying on a DNS library prevents a hands-on understanding of name compression, EDNS(0) and the malformed-packet risks in parsing.

2.3 Problem Statement

DNS is a critical and frequently attacked layer of Internet infrastructure, yet the way a recursive resolver walks the hierarchy, judges which records to trust and caches results is usually hidden inside large production systems or high-level libraries. There is a need for a compact, transparent and verifiable recursive resolver that implements the protocol from first principles, shows every step of resolution, and builds in the security checks that prevent cache-poisoning attacks, so that DNS behaviour and its weaknesses can be studied, demonstrated and tested in a controlled environment.

2.4 Project Goals

The goals of the project follow from the objectives in Section 1.1 and are grouped below into functional, security, quality and usability goals. Each goal is later checked against the results in Chapter 7.

Table 2.2 Project Goals and How They Are Measured

ID	Goal	Measure of success
G1	Correct wire-format codec	All supported record types parsed from live replies; compressed names decoded; malformed packets rejected without crashing
G2	Complete iterative resolution	Names resolved from the root for 15 popular domains with the same response code as a public resolver
G3	Correct special cases	CNAME, NXDOMAIN, NODATA, glueless delegation and TC to TCP fallback demonstrated both offline and live
G4	Spoofing and poisoning defences	Forged or mismatched replies ignored; out-of-bailiwick referrals and glue rejected in tests
G5	Caching for performance	Measurable latency reduction for repeated and sibling names
G6	Two modes of operation	CLI trace output, and a server that answers standard DNS clients
G7	Robust parser	No panic or hang under large-scale random and mutation-based fuzzing

ID	Goal	Measure of success
G8	Small and dependency-free	Standard library only; single binary; about 1,000 lines of code

2.5 Scope of the Project

The project covers the resolver side of DNS over IPv4 transport, for the record types listed in Table 1.1, with UDP and TCP transport to authoritative servers. It covers positive caching of answers and caching of delegations, and a recursive server that listens on UDP. The following items are explicitly outside the scope of this version and are discussed as future work in Chapter 8: DNSSEC validation, IPv6 transport to name servers (AAAA glue is parsed but not used), negative caching, QNAME minimisation, encrypted transports (DoT and DoH) and TCP listening in server mode.

CHAPTER 3

TECHNICAL SPECIFICATION

This chapter specifies the requirements that DNSR must satisfy, the hardware and software environment used to build and test it, and the protocol details, message layout, flags and limits, that the implementation follows.

3.1 Functional Requirements

Table 3.1 Functional Requirements

ID	Requirement
FR1	Accept a domain name and a record type (A, AAAA, NS, MX, TXT, CNAME, SOA or PTR) from the command line; the default type is A.
FR2	Resolve the name iteratively, starting from the 13 IANA root server addresses, without using any upstream recursive resolver.
FR3	Follow referrals using NS records from the authority section and glue address records from the additional section.
FR4	When a referral has no usable glue, resolve the address of one of the listed name servers with a nested lookup (glueless delegation).
FR5	Follow CNAME aliases by restarting resolution for the target name and return the CNAME together with the final records.
FR6	Distinguish NXDOMAIN (name does not exist) from NODATA (name exists, no records of the type) and return the SOA from the authority section.
FR7	Retry a query over TCP when a UDP reply has the TC bit set.
FR8	Cache positive answers with a TTL that counts down, and cache delegations so that later queries can start below the root.
FR9	Print a human-readable trace of every step, with an option to suppress it and print only the answer, and an option to disable colour.
FR10	Run as a recursive DNS server on a configurable UDP address and answer standard DNS queries from clients.

3.2 Security Requirements

Table 3.2 Security Requirements

ID	Requirement	Threat addressed
SR1	Use a random 16-bit transaction ID for every query	Blind ID guessing
SR2	Accept a reply only if ID, QR bit, question name and question type match the query; otherwise keep waiting until the deadline	Forged and stray replies
SR3	Accept a referral only if the child zone is strictly below the current zone and contains the queried name	Redirection to an unrelated zone

ID	Requirement	Threat addressed
SR4	Accept a glue record only if it belongs to one of the referred name servers and lies inside the zone of the server that sent it	Kaminsky-style glue poisoning
SR5	Accept only answer records whose owner is the queried name	Injection of unrelated records into the cache
SR6	Bound every read in the parser and limit compression-pointer jumps	Crashes and hangs from malformed packets
SR7	Limit hops, nesting depth, servers tried per step and time per query	Resolution loops and resource exhaustion
SR8	Listen on the loopback address by default in server mode	Exposure as an open resolver

3.3 Non-Functional Requirements

- ✓ **Portability:** the resolver must build into a single static binary for Windows, Linux and macOS with the standard Go tool chain.
- ✓ **Dependencies:** only the Go standard library may be used (`net`, `encoding/binary`, `sync`, `flag`, `math/rand` and similar).
- ✓ **Performance:** a cold-cache resolution should complete within a few seconds; cached answers should be returned without network traffic.
- ✓ **Concurrency safety:** in server mode, many queries may be resolved at the same time, so the shared caches must be protected against data races.
- ✓ **Readability:** the code should stay small enough (about 1,000 lines) to be read in one sitting, with one file per layer.
- ✓ **Testability:** all resolution logic must be testable offline without access to the Internet.

3.4 Hardware and Software Specification

Table 3.3 Development and Test Environment

Item	Specification
Processor	13th Gen Intel Core i7-13650HX (14 cores, 20 threads)
Operating system	Windows 11 Home Single Language (64-bit)
Language and tool chain	Go (module declares go 1.22; built and tested with go1.27.0 windows/amd64)
Libraries used by DNSR	Go standard library only
Testing	Go `testing` package: unit and integration tests, benchmarks and native fuzzing

Item	Specification
Live validation client	dnspython 2.7.0 [31] on Python 3.13 (independent DNS implementation)
Reference resolver for comparison	Google Public DNS (8.8.8.8)
Network	Outbound UDP and TCP port 53 to the Internet
Minimum to run	Any 64-bit machine able to run Go binaries; less than 10 MB of memory in normal use

3.5 DNS Message Format

Every DNS message, query or reply, has the same layout, shown in Fig. 3.1. The 12-byte header is followed by four sections. The question section normally holds exactly one question. The answer section holds records that answer the question, the authority section holds NS records (for a referral) or an SOA record (for a negative answer), and the additional section holds helpful extra records such as glue addresses and the EDNS(0) OPT pseudo-record.

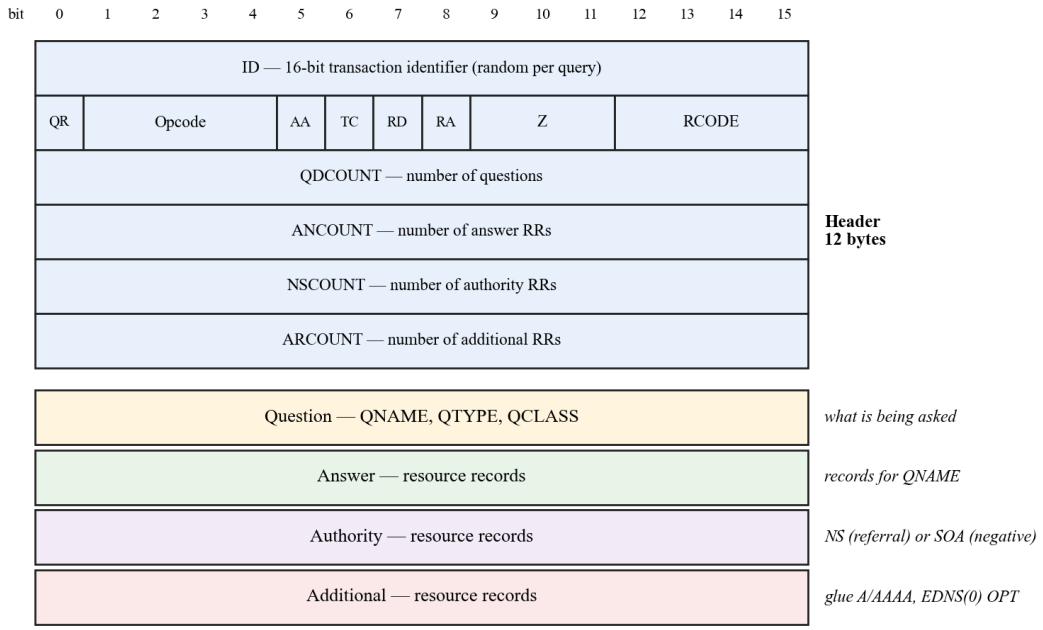


Fig. 3.1 Layout of a DNS Message and Its 12-Byte Header

The meaning of each header flag and how DNSR uses it is summarised in Table 3.4. All multi-byte integers in a DNS message are in network byte order (big-endian), which DNSR reads and writes with Go's `encoding/binary` package.

Table 3.4 Header Fields and Their Use in DNSR

Field	Bits	Meaning	Use in DNSR
ID	16	Transaction identifier copied into the reply	Random per query; must match in the reply
QR	1	0 = query, 1 = response	Replies must have QR = 1; server drops packets with QR = 1
Opcode	4	Kind of query (0 = standard)	Always 0
AA	1	Authoritative answer	AA = 1 with no answer records means NODATA
TC	1	Message was truncated	Triggers retry over TCP
RD	1	Recursion desired	0 in outgoing queries; copied back in server replies
RA	1	Recursion available	Set to 1 in server-mode replies
Z	3	Reserved	0
RCODE	4	Response code	NOERROR, NXDOMAIN, SERVFAIL and others handled
QDCOUNT ... ARCOUNT	4 x 16	Number of entries in each section	Drive the parsing loops; QDCOUNT must be 1

3.5.1 Domain-name encoding and compression

A name is encoded as a sequence of labels, each preceded by a one-byte length, and terminated by a zero byte that stands for the root. Because the same suffix appears many times in a reply (for example `github.com` in the question, the answer and the authority records), RFC 1035 allows the tail of a name to be replaced by a two-byte pointer whose two top bits are set to 1 and whose remaining 14 bits give the offset of an earlier occurrence. Fig. 3.2 shows a typical case: the owner name of the answer record is just the pointer `C0 0C`, which points back to offset 12 where the question name starts.

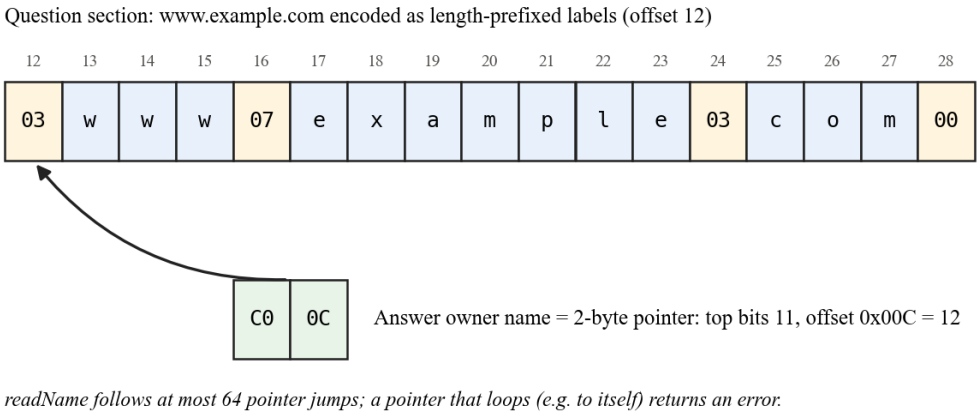


Fig. 3.2 Name Compression: a Pointer to the Question Name

Pointers are a well-known attack surface. A pointer that points to itself, or two pointers that point to each other, would make a naive parser loop forever. DNSR therefore counts the jumps and returns an error after 64 of them, and it checks every offset against the length of the message before reading it.

3.5.2 EDNS(0) and transport

Every query that DNSR sends carries an OPT pseudo-record in the additional section. Its owner name is the root, its type is 41, the CLASS field is reused to advertise the UDP payload size (1232 bytes) and the TTL field carries extended flags (all zero, so the DO bit for DNSSEC is not set). If a reply still does not fit, the server sets TC and DNSR repeats the identical query over TCP. Over TCP each message is preceded by its length as a 16-bit integer, as defined in RFC 1035 Section 4.2.2.

3.6 Operating Limits and Constants

Table 3.5 lists the constants that bound the work DNSR does for one query. They protect against loops in misconfigured or malicious delegations and keep the worst-case time of a query predictable.

Table 3.5 Resolver Constants

Constant	Value	Purpose
Timeout	2 s per exchange	Deadline for each UDP or TCP exchange with one server
MaxHops	40	Maximum number of referral steps in one resolution

Constant	Value	Purpose
Nesting depth	8	Maximum depth of CNAME and glueless sub-resolutions
Servers per step	4	At most four randomly chosen servers are tried per step
Compression jumps	64	Maximum pointer jumps when decoding one name
EDNS(0) buffer	1232 bytes	Advertised UDP payload size (DNS Flag Day 2020)
UDP receive buffer	4096 bytes	Buffer used to read replies
Label length	1 to 63 bytes	Enforced when encoding names
Server listen address	127.0.0.1:5353 (default)	Loopback only unless changed with -addr
Server receive buffer	1500 bytes	Maximum query size accepted in server mode

3.7 Command-Line Interface Specification

DNSR is a single executable with two forms of invocation. Table 3.6 lists the flags. Several names may be given in one invocation; they are resolved in order and share the same caches, which makes the effect of caching easy to observe.

Table 3.6 Command-Line Usage and Flags

Command or flag	Description
dnsr [flags] <name> [name...]	Resolve each name from the root down and print the trace and the answer
-t TYPE	Record type: A, AAAA, NS, MX, TXT, CNAME, SOA or PTR (default A)
-q	Quiet mode: print only the final answer, not the trace
-nocolor	Disable ANSI colours (colour is also disabled automatically when output is not a terminal)
dnsr serve [-addr host:port]	Run as a recursive DNS server on UDP (default 127.0.0.1:5353)
serve -trace	Print the full resolution trace for every query received by the server
Exit status	0 if every name resolved without error, 1 if any resolution failed, 2 on a usage error

CHAPTER 4

DESIGN APPROACH AND DETAILS

This chapter describes how DNSR was designed: the method followed, the architecture and its modules, the data structures, the resolution algorithm, the security mechanisms, caching and transport, the server mode and the test environment. It then lists the codes and standards that the design follows and discusses the constraints, alternatives and trade-offs that shaped it.

4.1 Design Approach / Materials and Methods

4.1.1 Development methodology

The project followed an incremental, test-driven methodology. Each protocol feature was first specified from the relevant RFC, then implemented in isolation, then verified by a test before the next layer was built on top of it. Building from the bottom up meant that the wire-format codec was reliable before the resolver used it, and that each resolver feature (referrals, glue, CNAMEs, negative answers, truncation) had a dedicated test in the fake DNS hierarchy before it was tried against the live Internet. The steps were as follows.

- ✓ **Requirement analysis and protocol study:** RFC 1034/1035, RFC 2181 (data ranking), RFC 6891 (EDNS(0)), RFC 7766 (TCP) and RFC 5452 (forgery resilience) were studied to fix the functional and security requirements of Chapter 3.
- ✓ **Wire-format codec:** message encoding and decoding were written by hand, including compression-pointer handling with loop protection and bounds checks on every read, so that malformed packets return errors instead of crashing the parser.
- ✓ **Iterative resolution engine:** the referral-following loop was built from the root hints, followed by CNAME chasing, NXDOMAIN and NODATA detection, glueless delegation and the hop and depth limits.
- ✓ **Security hardening:** random query IDs, strict reply matching and the two bailiwick checks were added and each was covered by a test with a deliberately malicious server.

- ✓ **Caching and transport:** the answer cache, the delegation cache, randomised server selection with fail-over and the UDP to TCP fallback were added.
- ✓ **Server mode:** the resolver was exposed as a concurrent UDP DNS server.
- ✓ **Validation:** the resolver was tested offline, measured live against 15 popular domains, compared with a public resolver, fuzzed and benchmarked (Chapter 7).

4.1.2 System architecture

DNSR follows a layered architecture, shown in Fig. 4.1. The interface layer offers two entry points: the command-line client in `main.go` and the UDP recursive server in `server.go`. Both pass requests to the resolution engine in `resolver.go`, which holds the iterative algorithm, the security checks and the two caches. The engine talks to the Internet through a small transport layer that sends UDP queries with an EDNS(0) buffer of 1232 bytes and switches to TCP when the TC bit is set. All messages are built and parsed by the wire codec in `wire.go`, which implements the RFC 1035 binary format by hand. Each layer depends only on the layer below it, which keeps the code easy to test and to read.

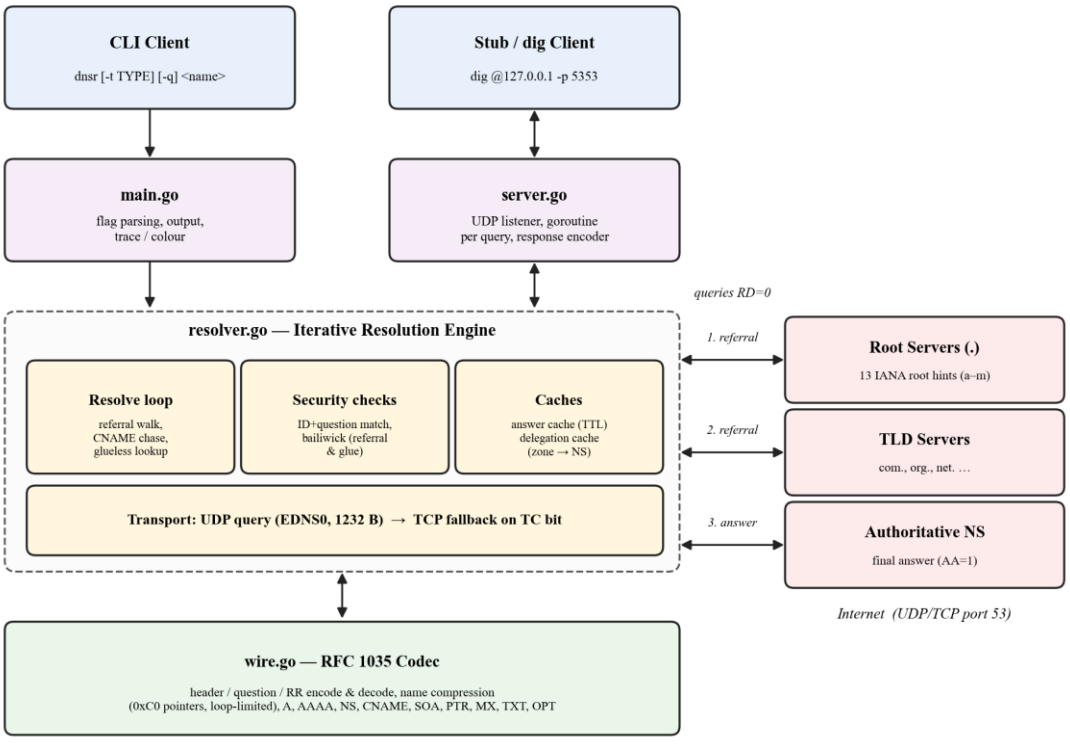


Fig. 4.1 System Architecture of DNSR

Table 4.1 describes the responsibility of each source file. The whole program, excluding tests, is 969 lines of Go; the test file adds 292 lines.

Table 4.1 Module Description

Module (lines)	Responsibility
wire.go (320)	Defines the Header, Question, RR and Message types; encodes names and queries (RD = 0 with an EDNS(0) OPT record); decodes compressed names with a 64-jump loop limit; parses the RDATA of A, AAAA, NS, CNAME, PTR, MX, SOA and TXT records; skips OPT records in replies.
resolver.go (427)	Root hints; iterative resolution loop; answer cache with TTL count-down; delegation cache (closestKnownZone); referral and glue processing with bailiwick checks; glueless NS lookup; CNAME restart; random server selection with fail-over; UDP exchange with anti-spoofing checks; TCP fallback; trace output.
server.go (133)	Listens on UDP; handles each query in its own goroutine; validates the query; builds responses (QR = 1, RA = 1) and re-encodes names inside RDATA so that no stale compression pointers are sent.
main.go (89)	Parses command-line flags (-t, -q, -nocolor), selects CLI or server mode, enables colour only on a terminal, prints the trace and the answer and sets the exit status.
resolver_test.go (292)	Builds a fake DNS hierarchy on 127.0.0.10 to 127.0.0.15 and runs nine deterministic tests, including adversarial servers.

4.1.3 Data structures

The design keeps the data model close to the RFC so that each structure can be checked against the specification. A parsed message is a `Message` holding a `Header`, a slice of `Question` values and three slices of `RR` values. Each `RR` keeps both a human-readable `Data` string, used for display and for the server's re-encoding, and the `Raw` RDATA bytes, used for types that DNSR does not interpret. Table 4.2 lists the main structures.

Table 4.2 Main Data Structures

Structure	Fields	Purpose
Header	ID, QR, Opcode, AA, TC, RD, RA, Rcode, QDCount, ANCount, NSCount, ARCount	Decoded 12-byte message header
Question	Name, Type, Class	One entry of the question section
RR	Name, Type, Class, TTL, Data, Raw	One resource record
Message	Header, Questions, Answers, Authority, Additional	Whole parsed message
nameserver	Name, IP	A server that can be queried

Structure	Fields	Purpose
cacheEntry	rrs, expires	Cached RRset and its absolute expiry time
Resolver	Timeout, MaxHops, Trace, Port, Roots, cache, zoneNS, mutex	Resolver configuration and shared state
Result	Rcode, Answers, Authority, Hops, Elapsed	Outcome of one resolution, returned to the caller

The two caches are Go maps protected by one mutex. The answer cache maps the key `name|TYPE` (for example `github.com.|A`) to a `cacheEntry`. The delegation cache `zoneNS` maps a zone name such as `github.com.` to the list of name servers (with addresses) that serve it. Root hints are not stored in the map; they are the fallback when no deeper zone is known.

4.1.4 Resolution algorithm

The heart of DNSR is the function `resolve`. Its control flow is shown in Fig. 4.2 and its logic in the pseudo-code that follows. Resolution starts by checking the answer cache. On a miss, the resolver picks the deepest zone it already knows from the delegation cache, or the root if it knows none, and enters a loop. In each iteration it sends a non-recursive query to up to four randomly chosen servers of the current zone and classifies the first valid reply.

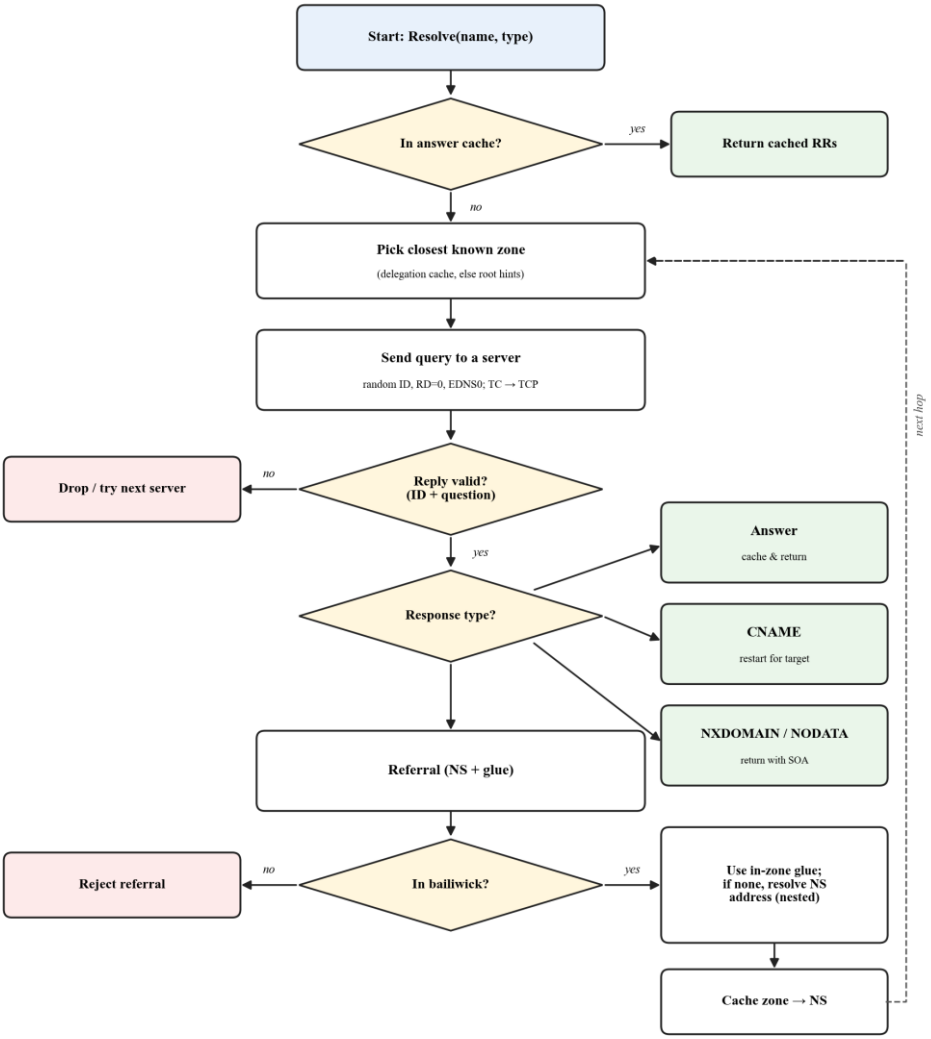


Fig. 4.2 Iterative Resolution Flowchart

```
resolve(name, type, depth, hops):
    if depth > 8:                       return error "resolution loop / too deep"
    if cache has (name, type):          return cached records (TTL = time remaining)
    zone, servers := closest known zone of name (delegation cache, else root hints)
    while hops < MaxHops (40):
        reply, server := query up to 4 random servers of zone        (UDP, TCP if
TC=1)
        if reply.rcode == NXDOMAIN:     return NXDOMAIN + authority (SOA)
        if reply.rcode != NOERROR:      drop server; if none left return rcode;
continue
        answers := records in reply.answer whose owner == name
        if answers of the requested type exist:
            cache them; return them
        if a CNAME for name exists:
            sub := resolve(cname.target, type, depth+1, hops+1)
            return [cname] + sub.answers
        if reply.AA:                    return NODATA + authority (SOA)
        ns := NS records in reply.authority;  child := owner of ns
        if ns is empty:                 return error "lame response"
        if child not strictly below zone or name not under child:
                                        return error "out-of-bailiwick referral"
        next := glue A records for ns that lie inside zone
        if next is empty:               # glueless delegation
            for each ns target in random order:
```

```
                addr := resolve(target, A, depth+1, 0)
                if addr found: next := addr; break
        delegation cache[child] := next
        zone, servers := child, next;  hops := hops + 1
    return error "resolution loop / too deep"
```

Four properties of the algorithm deserve attention. First, every exit from the loop is explicit, and two independent counters (hops and depth) guarantee termination even when servers misbehave. Second, a CNAME is never trusted together with records for its target that arrive in the same packet: resolution of the target restarts from the cache and the delegation tree, so a server can only vouch for names in its own zone. Third, a referral is only followed after the bailiwick check, so the delegation cache can only ever move *down* the tree towards the queried name. Fourth, the nested resolutions for CNAME targets and glueless name servers reuse the same function, so they inherit the same checks and limits.

4.1.5 Example: resolving www.github.com

Fig. 4.3 shows the messages exchanged in a real resolution of `www.github.com` captured during testing. The root server refers the resolver to the 13 `.com` servers; one of them refers it to GitHub's eight name servers hosted by Amazon Route 53; the GitHub server answers with a CNAME to `github.com`. The resolver then restarts for `github.com`, but because the delegation for `github.com.` is now cached it goes straight to GitHub's name server, which returns the address. The complete resolution took four network round trips and 557 ms.

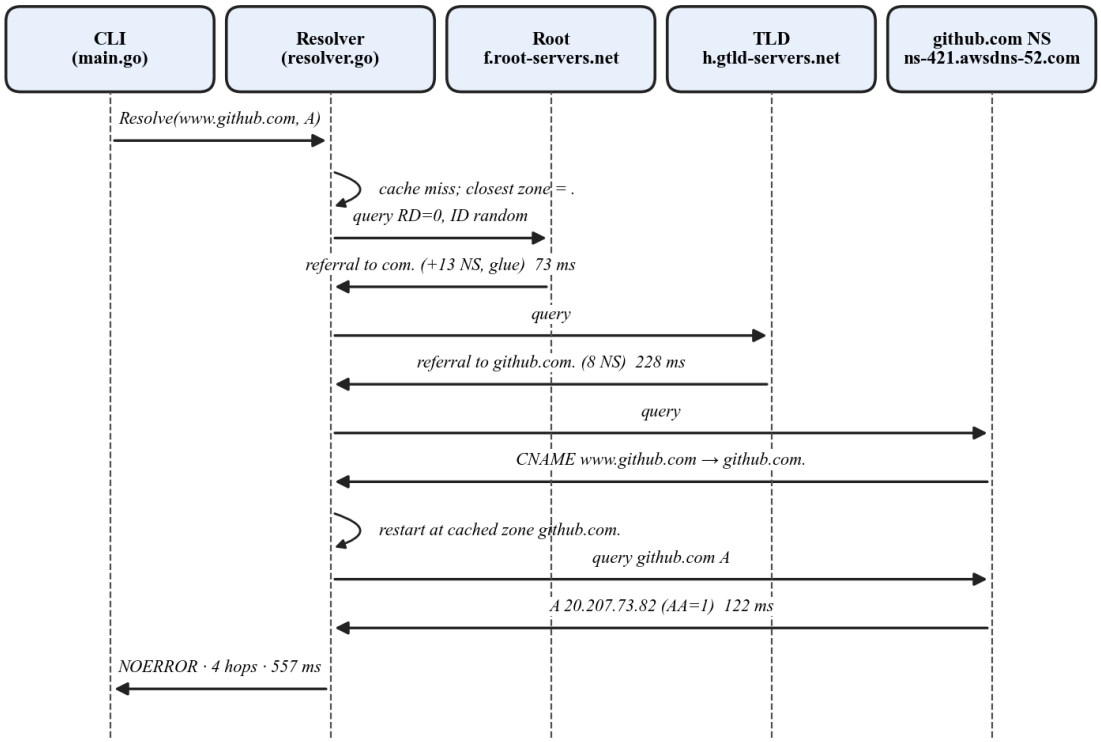


Fig. 4.3 Sequence of Messages for a Cold-Cache Lookup of www.github.com

4.1.6 Wire-format codec design

The codec is written as a set of small functions, each responsible for one element of the format. Encoding is simple because DNSR only encodes queries and, in server mode, responses without compression. `encodeName` splits a name at the dots, rejects empty labels and labels longer than 63 bytes, and writes each label with its length byte followed by the terminating zero. `BuildQuery` writes the header (random ID, all flags zero, QDCOUNT = 1, ARCOUNT = 1), the question and the OPT record.

Decoding is where the risks lie, because the input comes from the network and may be hostile. The decoder follows three rules throughout. (1) Before any read, the offset plus the length of the read is compared with the length of the message; if it would run past the end the function returns `errShort`. (2) Lengths inside the packet (RDLENGTH, label lengths, TXT string lengths) are never trusted beyond the message boundary. (3) Loops are bounded: the number of records read is bounded by the header counts and the packet length, and the number of compression jumps is bounded by 64. `readName` additionally rejects the reserved label types 0x40 and 0x80. Names inside

RDATA (NS, CNAME, PTR, MX and SOA) are decoded with the same `readName` function because they too may be compressed.

4.1.7 Security design

The security design starts from an explicit threat model, given in Table 4.3. DNSR is designed to defend fully against the off-path attacker and against a malicious or compromised authoritative server trying to affect names outside its own zone. It cannot defend against an on-path attacker who can read the queries, because classic DNS has no cryptographic protection; that requires DNSSEC.

Table 4.3 Threat Model

Adversary	Capabilities	Goal	DNSR defence
Off-path spoofer	Sends forged UDP packets with any source address; cannot see queries	Insert forged records into the cache	Random ID and source port; full reply matching; answer filtering
Malicious authoritative server	Controls the replies for its own zone	Poison names in other zones	Bailiwick checks on referrals and glue; owner-name filtering; CNAME restart
Malformed-packet sender	Sends arbitrary bytes	Crash or hang the resolver	Bounds checks; pointer-jump limit; Go memory safety
Misconfigured delegation	Loops in NS or CNAME chains	Exhaust the resolver	Hop, depth, server and time limits
On-path attacker	Sees and modifies traffic	Forge any answer	Out of scope (needs DNSSEC)

Defence 1: unpredictable queries and strict reply matching

For each query DNSR draws a fresh 16-bit ID from Go's random number generator and sends it from a new UDP socket, for which the operating system chooses a random ephemeral source port. The socket is *connected* to the server's address, so the kernel discards datagrams from any other address or port. When a datagram arrives, DNSR accepts it only if (a) the ID equals the one sent, (b) the QR bit is set, (c) there is exactly one question and (d) its name (compared case-insensitively) and type equal those of the query. A datagram that fails any check is ignored and DNSR keeps reading until the 2-second deadline. Ignoring, rather than failing, matters: if a forged packet

caused the exchange to fail, an attacker could block resolution simply by sending garbage.

The benefit of this randomness can be quantified. If an attacker manages to deliver *k* forged replies before the genuine reply arrives, and each forged reply guesses uniformly among *N* equally likely combinations of ID and port, the probability that at least one forgery is accepted is given by Eqn. 4.1.

$$P = 1 - \left(1 - \frac{1}{N}\right)^{k} \approx \frac{k}{N} \tag{4.1}$$

With only a random ID, N = 2^{16} = 65,536, so about 45,000 forged packets give an even chance of success, which is achievable within one query timeout on a fast network. With a random source port in addition, *N* is multiplied by the size of the ephemeral port range (Eqn. 4.2). On Windows the dynamic range is 49152 to 65535 [32], that is R = 16,384 ports, which raises the effort by four orders of magnitude, to about 7.4×10^{8} packets for a 50% chance. Section 7.7 plots both curves.

$$N = 2^{16} \times R_{port} = 65536 \times 16384 \approx 1.07 \times 10^{9} \tag{4.2}$$

Defence 2: bailiwick checks on referrals and glue

Even a genuine reply from a real server must not be allowed to affect names that the server is not responsible for. DNSR applies two bailiwick rules, illustrated in Fig. 4.4. For a referral, the child zone named by the NS records must lie strictly below the zone of the server that sent it, and the queried name must lie at or below the child zone. A `.com` server can therefore delegate `example.com.` when asked for `www.example.com`, but it cannot delegate `net.` or re-delegate `com.` itself. For glue, an address record in the additional section is used only if its owner is one of the name servers listed in the referral *and* lies inside the zone of the server that sent it. A `.com` server may supply the address of `ns1.example.com`, but an address it supplies for `ns.dnshost.org` is ignored; DNSR resolves that name separately from the `.org` hierarchy. This is precisely the check that defeats Kaminsky's forged-glue technique.

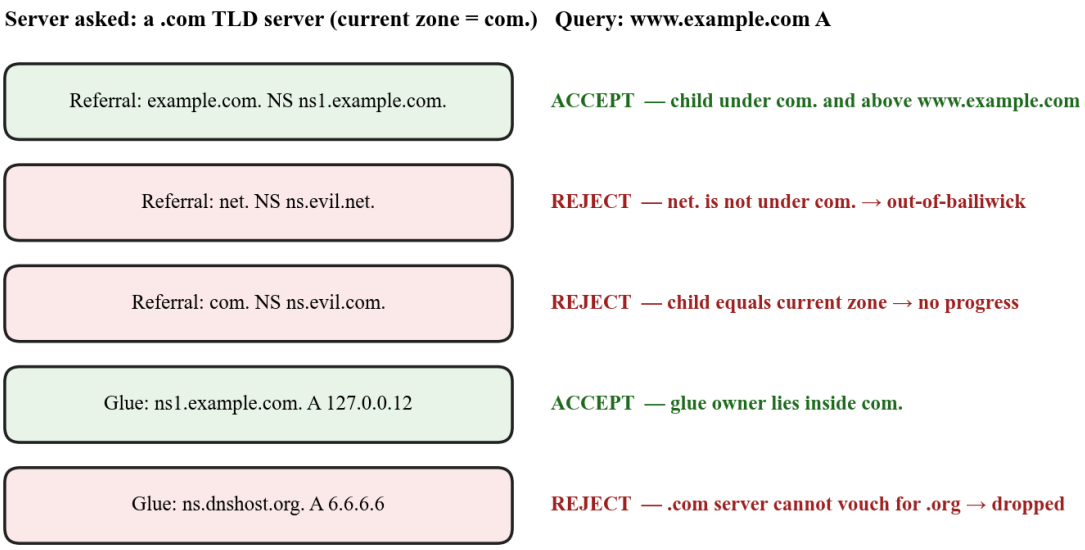


Fig. 4.4 Bailiwick Decisions for Replies from a .com Server

Defence 3: answer filtering and CNAME restart

Only records in the answer section whose owner name equals the queried name are considered; any other record is ignored and never cached. When the answer is a CNAME, DNSR follows it by starting a new resolution for the target rather than accepting records for the target from the same packet. Additional and authority records from a reply are never inserted into the answer cache.

Defence 4: safe parsing and resource limits

Go is memory-safe, so an out-of-range read causes a panic instead of silent memory corruption, but a panic would still crash the resolver. DNSR's parser therefore checks every length before reading, as described in Section 4.1.6, so that malformed input produces an error value. The hop limit (40), depth limit (8), four servers per step and the two-second deadline bound the amount of work and time that any reply can cause. Chapter 7 reports the results of fuzzing the parser with more than 34 million inputs.

4.1.8 Caching design

DNSR keeps two caches. The *answer cache* stores RRsets that answered a question. Because the records of an RRset may carry different TTLs, the entry expires when the shortest TTL expires (Eqn. 4.3), where t_store is the time the RRset was stored.

$$t_{exp} = t_{store} + min_i\, TTL_i \tag{4.3}$$

When an entry is returned, every record's TTL is rewritten to the time that remains (Eqn. 4.4), as a real resolver does, so that a client never caches a record for longer than its owner intended.

$$TTL_{rem} = \lfloor t_{exp} - t_{now} \rfloor \tag{4.4}$$

The *delegation cache* stores, for every zone the resolver has been referred to, the name servers and addresses it learned. Before a resolution, `closestKnownZone` strips labels from the left of the queried name (`api.github.com.`, `github.com.`, `com.`, `.`) until it finds a zone in the cache. A second lookup under the same domain therefore skips the root and TLD servers completely, which Section 7.4 shows is the most effective optimisation for real workloads, in agreement with Jung et al. [12].

4.1.9 Transport design

Each exchange uses a new UDP socket with a deadline equal to the timeout. The reply is read into a 4096-byte buffer and parsed. If the TC bit is set, the same query bytes are sent over a new TCP connection to the same server, framed with a two-byte length prefix; the reply length is read first and then exactly that many bytes are read with `io.ReadFull`. The maximum message size over TCP is therefore 65,535 bytes, which also bounds the memory a TCP reply can consume. Servers in a zone are tried in random order, which spreads load and makes the next server harder to predict, and a server that returns an error code is removed from the candidate list for the rest of that resolution.

4.1.10 Server-mode design

In server mode DNSR acts as a recursive resolver for other clients. Its structure is shown in Fig. 4.5. The main loop reads one datagram at a time, copies it into a fresh buffer and starts a goroutine to handle it, so a slow resolution never blocks other clients. The handler parses the packet and drops it silently if it cannot be parsed, has the QR bit set or does not have exactly one question. Otherwise it calls the shared `Resolver`, whose caches are protected by a mutex, and builds a response with QR = 1, RA = 1, the client's RD bit, the original ID and question, and the answer and authority records. If resolution fails the response carries SERVFAIL.

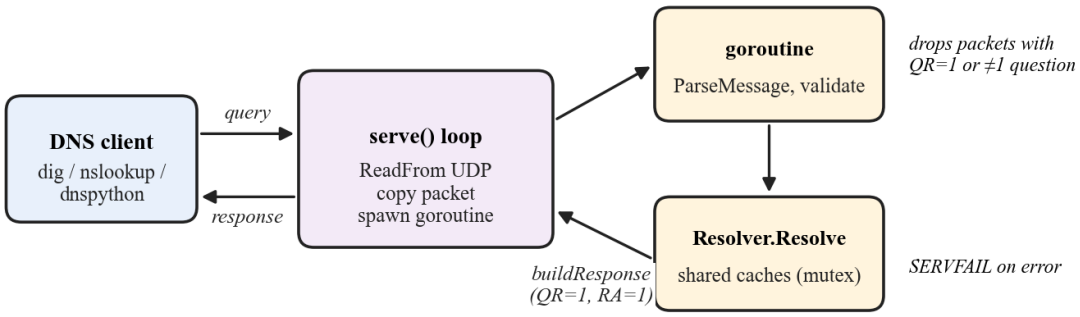


Fig. 4.5 Structure of the Recursive Server Mode

A subtle design point is the encoding of the response. Records parsed from upstream replies may contain compressed names whose pointers refer to offsets in the *upstream* packet. Copying the raw RDATA into a new packet would leave those pointers pointing at random bytes. `encodeRData` therefore re-encodes the names inside NS, CNAME, PTR, MX and SOA records from their decoded form; other types are copied as raw bytes, which is safe because they contain no names.

4.1.11 Test environment design

Testing a resolver against the live Internet is not repeatable: servers, addresses and TTLs change, and edge cases such as poisoned glue cannot be produced on demand. DNSR's test suite therefore builds a complete fake DNS hierarchy inside the test process, shown in Fig. 4.6. Six small authoritative servers listen on loopback addresses 127.0.0.10 to 127.0.0.15, port 15353, over both UDP and TCP. The resolver under test is configured with the fake root as its only root hint and port 15353 as its destination port; no other change is needed, which means the tests exercise exactly the same code that runs in production.

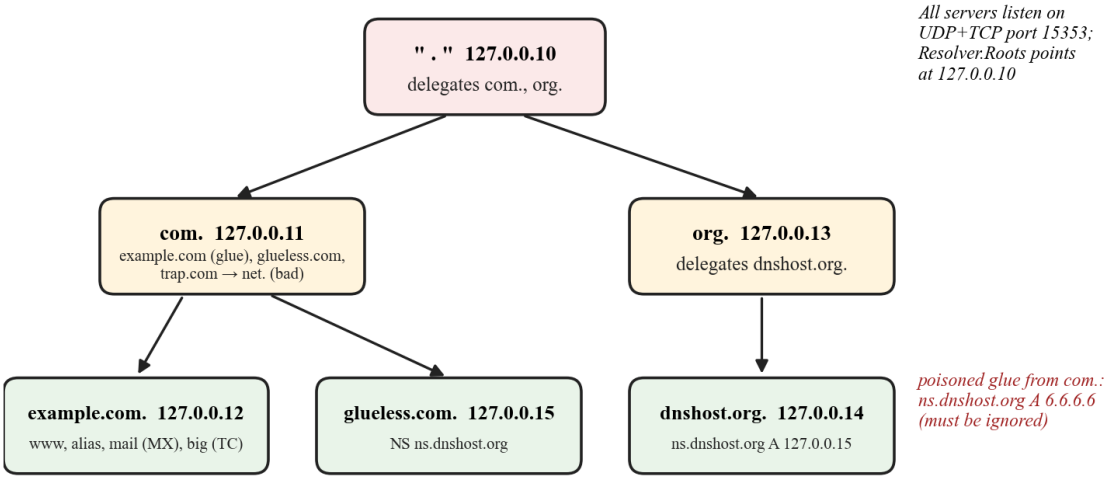


Fig. 4.6 Offline Test Hierarchy on Loopback Addresses

The fake hierarchy includes deliberately hostile behaviour. The `com.` server delegates `trap.com.` by returning NS records for `net.`, an out-of-bailiwick referral, and it attaches poisoned glue (`ns.dnshost.org A 6.6.6.6`) to the delegation of `glueless.com.`, whose real name server lives under `.org`. The `example.com.` server sets the TC bit on UDP replies for one name so that the TCP fallback is exercised. Table 4.4 lists the nine tests.

Table 4.4 Test Cases of the Offline Suite

Test	What it verifies
TestReferralChain	Full root to TLD to authoritative walk returns the correct A record
TestDelegationCache	A second lookup in the same zone skips the root and TLD servers
TestCNAME	The alias is followed and the CNAME is placed before the final answer
TestNXDomainAndNoData	Non-existent names and missing record types are told apart
TestMX	MX RDATA (preference and exchange) is parsed correctly
TestGluelessDelegationIgnoresPoisonedGlue	Out-of-bailiwick glue is discarded and the name server's address is resolved independently
TestOutOfBailiwickReferralRejected	A server cannot redirect resolution to an unrelated zone
TestTruncationFallsBackToTCP	A TC = 1 UDP reply triggers a successful retry over TCP
TestNameCompression	Compressed names decode correctly and a self-referencing pointer is rejected

4.2 Codes and Standards

DNSR implements a protocol defined entirely by Internet standards, so conformance to the relevant RFCs is the main standard the design follows. Table 4.5 lists each standard and the parts that DNSR implements. In addition, the code follows the conventions of the Go language.

Table 4.5 Standards Followed

Standard	Title	Used for
RFC 1034 [1]	Domain Names: Concepts and Facilities	Namespace, zones, resolver algorithm
RFC 1035 [2]	Domain Names: Implementation and Specification	Message format, name encoding and compression, RR types, TCP framing
RFC 2181 [4]	Clarifications to the DNS Specification	Data ranking and trust (basis of the bailiwick rules); RRset TTLs
RFC 2308 [5]	Negative Caching of DNS Queries	Recognising NXDOMAIN and NODATA by the SOA in authority
RFC 3596 [30]	DNS Extensions to Support IPv6	AAAA record format
RFC 5452 [8]	Measures for Making DNS More Resilient against Forged Answers	Random ID and port, reply matching
RFC 6891 [6]	Extension Mechanisms for DNS (EDNS(0))	OPT record, UDP payload size
RFC 7766 [7]	DNS Transport over TCP	TCP fallback on truncation
RFC 8499 [26]	DNS Terminology	Terms such as referral, glue, bailiwick
DNS Flag Day 2020 [20]	Recommended EDNS buffer size	1232-byte buffer
IANA named.root [21]	Root hints file	Addresses of the 13 root servers

The coding standards applied are those of the Go project: all code is formatted with `gofmt`, checked with `go vet`, and organised as one package with one file per layer. Errors are returned as values rather than through panics, exported identifiers start with a capital letter, and comments are limited to file headers and function descriptions. The references in this report follow the IEEE citation style.

4.3 Constraints, Alternatives and Trade-offs

4.3.1 Constraints

- ✓ **No DNS library:** the project brief required the protocol to be implemented from first principles, so every encoding and decoding step had to be written and tested by hand.
- ✓ **Network access:** live resolution needs outbound UDP and TCP port 53. Some campus and corporate networks block this and force all DNS through their own resolver; the offline test hierarchy removes this dependency for testing.
- ✓ **Windows port conflicts:** on Windows the default server port 5353 is already used by the system's multicast DNS (mDNS) service, so the server must be started on another port such as 8053 (see Chapter 7).
- ✓ **IPv4 only:** the test machine and many networks lack reliable IPv6 connectivity, so name servers are contacted only over IPv4.
- ✓ **Time:** the project had to be completed within one semester, which ruled out DNSSEC validation (a substantial cryptographic sub-project).

4.3.2 Alternatives considered

Several design decisions had reasonable alternatives. Table 4.6 records the options considered and the reason for each choice.

Table 4.6 Design Alternatives and Decisions

Decision	Alternatives	Choice and reason
Language	C, Rust, Python, Go	Go [22]: memory-safe, fast, built-in concurrency (goroutines), excellent networking standard library, single static binary
Message handling	miekg/dns library; hand-written codec	Hand-written: required by the brief and needed to study and harden the parser
Resolution model	Forward to an upstream resolver; iterate from the root	Iterate from the root: the object of study and independent of any third party
Server selection	Fixed order; random; RTT-based (as in Unbound)	Random: simple and spreads load; RTT-based selection left as future work
Parallelism	Query all servers in parallel; sequential fail-over	Sequential with a 2 s timeout: easier to trace and polite to servers, at the cost of slower recovery from a dead server
Cache structure	Single map; separate answer and delegation caches	Separate caches: delegations are the most valuable to keep (Jung et al. [12])

Decision	Alternatives	Choice and reason
Glueless handling	Fail; resolve all NS names; resolve until one works	Resolve in random order until one address is found
Testing	Live Internet only; mocks of the network layer; real servers on loopback	Real servers on loopback: tests the real code path, deterministic and offline

4.3.3 Trade-offs

The main trade-off is between simplicity and completeness. Keeping DNSR to about one thousand lines makes every security decision visible, but it means that features of production resolvers, such as negative caching, prefetching, serve-stale, RTT tracking and DNSSEC, are absent. A second trade-off is between latency and politeness: querying one server at a time with a two-second timeout means that an unreachable server adds up to two seconds to a resolution, but it avoids sending duplicate traffic to the DNS infrastructure. A third trade-off concerns strictness. Rejecting a referral that does not move strictly down the tree prevents redirection attacks, but a resolver that is too strict can fail on unusual but legitimate configurations; the checks in DNSR were chosen to follow RFC 2181 closely so that all 15 domains in the live evaluation resolved correctly. Finally, the trace output favours readability (colours, arrows and indentation) over machine parsing; quiet mode (-q) is provided for scripts.

CHAPTER 5

SCHEDULE, TASKS AND MILESTONES

The project was carried out over a fourteen-week semester. The work was divided into twelve tasks grouped into four phases: study and design, core implementation, hardening and server mode, and validation and documentation. Testing was not a separate phase at the end; the offline test hierarchy was built early and extended with every feature, as required by the test-driven method of Section 4.1.1.

5.1 Task Breakdown

Table 5.1 Task Breakdown and Schedule

No.	Task	Weeks	Output
1	Literature survey and study of RFC 1034/1035, 2181, 5452, 6891, 7766	1 to 2	Notes, list of requirements
2	Requirement analysis and design of the architecture	2	Requirements (Chapter 3), module plan
3	Wire-format codec: header, question, RR parsing, name compression, EDNS(0)	3 to 4	wire.go
4	Iterative resolver core: root hints, referral loop, glue	4 to 6	First end-to-end resolution
5	CNAME chasing, NXDOMAIN / NODATA, glueless delegations	6 to 7	Complete resolution logic
6	Security hardening: random IDs, reply matching, bailiwick checks, limits	7 to 8	Hardened resolver
7	Answer cache, delegation cache, TCP fallback	8 to 9	Caching and transport
8	Recursive server mode	9	server.go
9	Offline test hierarchy and test cases (incremental)	5 to 10	resolver_test.go, 9 tests
10	Live validation and measurements	11 to 12	Results of Chapter 7
11	Fuzzing and benchmarking of the parser	12	Robustness results
12	Report, diagrams and documentation	11 to 14	This report, README

5.2 Gantt Chart

Fig. 5.1 shows the same schedule as a Gantt chart. Blue bars are design and implementation tasks; green bars are testing, validation and documentation tasks. The overlap of the test task with the implementation tasks reflects the test-driven approach.

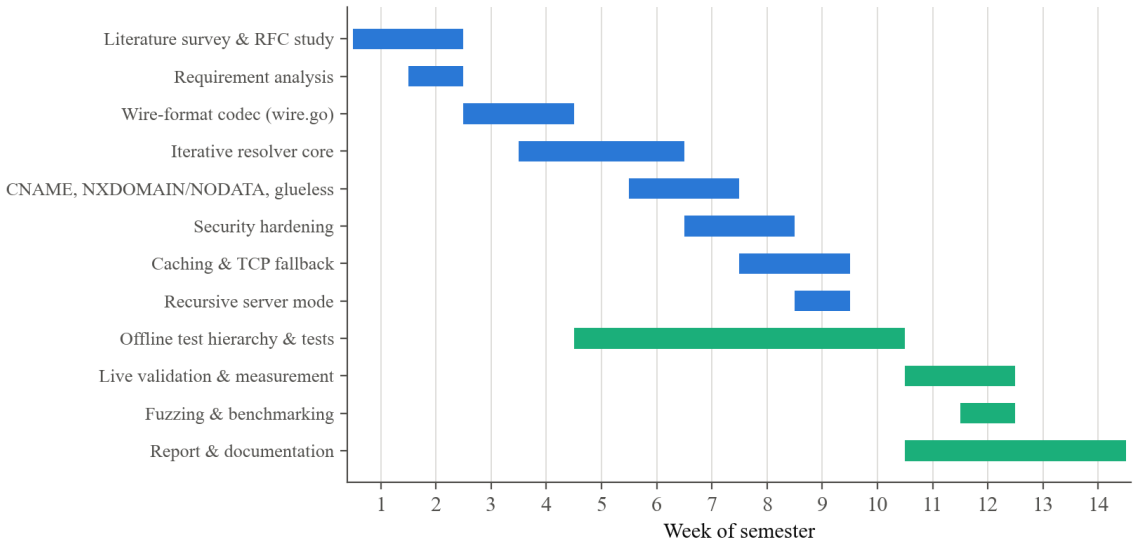


Fig. 5.1 Gantt Chart of the Project Schedule

5.3 Milestones

Table 5.2 Project Milestones

Milestone	Week	Completion criterion
M1: Design frozen	2	Requirements and architecture reviewed
M2: Codec complete	4	All record types parsed; compression test passes
M3: First live resolution	6	www.github.com resolved from the root servers
M4: Resolver feature-complete	8	CNAME, negative answers, glueless and bailiwick tests pass
M5: Caching, TCP and server	9	Delegation-cache, TCP and server tests pass
M6: Validation complete	12	Live comparison, cache measurements and fuzzing done
M7: Final report	14	Report and documentation submitted

5.4 Risk Management

Table 5.3 lists the main risks identified at the start of the project and how they were handled.

Table 5.3 Risks and Mitigations

Risk	Impact	Mitigation
Network blocks outbound port 53	Live tests impossible	Offline loopback hierarchy for all functional tests; live tests run from an unrestricted network
Live DNS data changes between runs	Results not repeatable	Functional tests offline; live results reported with run dates and repeated three times

Risk	Impact	Mitigation
Parser crashes on malformed input	Denial of service	Bounds checks everywhere; fuzzing with Go's native fuzzer
Scope creep (DNSSEC, IPv6)	Schedule slip	Explicit out-of-scope list (Section 2.5); items moved to future work
Port conflicts on Windows	Server mode cannot start	Configurable -addr flag; documented alternative port

CHAPTER 6

PROJECT DEMONSTRATION

This chapter demonstrates the working system. Section 6.1 presents the most important parts of the source code with an explanation of each, and Section 6.2 shows the program in operation through screenshots of real runs against the live Internet and the test suite. The complete source code is given in Appendix A.

6.1 Sample Codes

The listings below are taken unchanged from the project source. They are grouped by layer, from the wire codec up to the server.

6.1.1 Building a query (wire.go)

`encodeName` turns a dotted name into length-prefixed labels and rejects invalid labels. `BuildQuery` writes a 12-byte header with a caller-supplied random ID and all flags cleared (so RD = 0), then the question, then the EDNS(0) OPT record that advertises a 1232-byte UDP buffer.

```
func encodeName(buf []byte, name string) ([]byte, error) {
    name = strings.TrimSuffix(name, ".")
    if name != "" {
        for _, label := range strings.Split(name, ".") {
            if len(label) == 0 || len(label) > 63 {
                return nil, fmt.Errorf("bad label %q in %q", label, name)
            }
            buf = append(buf, byte(len(label)))
            buf = append(buf, label...)
        }
    }
    return append(buf, 0), nil
}

// BuildQuery makes a non-recursive query (RD=0) with an EDNS0 OPT record
// advertising a 1232-byte UDP buffer (the DNS Flag Day 2020 value).
func BuildQuery(id uint16, name string, qtype RRType) ([]byte, error) {
    b := make([]byte, 12, 512)
    binary.BigEndian.PutUint16(b[0:], id)
    binary.BigEndian.PutUint16(b[2:], 0)
    binary.BigEndian.PutUint16(b[4:], 1)
    binary.BigEndian.PutUint16(b[10:], 1)

    var err error
    if b, err = encodeName(b, name); err != nil {
        return nil, err
    }
    b = binary.BigEndian.AppendUint16(b, uint16(qtype))
    b = binary.BigEndian.AppendUint16(b, 1)

    b = append(b, 0)
    b = binary.BigEndian.AppendUint16(b, uint16(TypeOPT))
```

```
    b = binary.BigEndian.AppendUint16(b, 1232)
    b = binary.BigEndian.AppendUint32(b, 0)
    b = binary.BigEndian.AppendUint16(b, 0)
    return b, nil
}
```

6.1.2 Decoding compressed names safely (wire.go)

`readName` is the most security-sensitive function in the program, because it follows offsets taken from untrusted input. Every read is bounds-checked, the end offset of the name in the original stream is remembered at the first pointer, the number of pointer jumps is limited to 64, and the reserved label types are rejected.

```
// readName decodes a possibly-compressed name starting at off.
// Returns the name and the offset just past it in the *original* stream.
func readName(msg []byte, off int) (string, int, error) {
    var labels []string
    end := -1
    for jumps := 0; ; {
        if off >= len(msg) {
            return "", 0, errShort
        }
        l := int(msg[off])
        switch {
        case l == 0:
            if end < 0 {
                end = off + 1
            }
            return strings.Join(labels, ".") + ".", end, nil
        case l&0xC0 == 0xC0:
            if off+1 >= len(msg) {
                return "", 0, errShort
            }
            if end < 0 {
                end = off + 2
            }
            jumps++
            if jumps > 64 {
                return "", 0, errors.New("compression pointer loop")
            }
            off = int(binary.BigEndian.Uint16(msg[off:]) & 0x3FFF)
        case l&0xC0 != 0:
            return "", 0, fmt.Errorf("unsupported label type 0x%x", l)
        default:
            off++
            if off+l > len(msg) {
                return "", 0, errShort
            }
            labels = append(labels, string(msg[off:off+l]))
            off += l
        }
    }
}
```

6.1.3 Parsing resource records (wire.go)

`readRR` reads the fixed part of a record, checks that RDLENGTH stays within the message, and then decodes the RDATA according to the type. Names inside RDATA are decoded with `readName` on the whole message because they may be compressed. Unknown types are shown in the generic RFC 3597 `\#` hexadecimal form.

```
func readRR(msg []byte, off int) (RR, int, error) {
    var rr RR
    var err error
    if rr.Name, off, err = readName(msg, off); err != nil {
        return rr, 0, err
    }
    if off+10 > len(msg) {
        return rr, 0, errShort
    }
    rr.Type = RRType(binary.BigEndian.Uint16(msg[off:]))
    rr.Class = binary.BigEndian.Uint16(msg[off+2:])
    rr.TTL = binary.BigEndian.Uint32(msg[off+4:])
    rdlen := int(binary.BigEndian.Uint16(msg[off+8:]))
    off += 10
    if off+rdlen > len(msg) {
        return rr, 0, errShort
    }
    rd := msg[off : off+rdlen]
    rr.Raw = rd

    switch rr.Type {
    case TypeA:
        if rdlen == 4 {
            rr.Data = net.IP(rd).String()
        }
    case TypeAAAA:
        if rdlen == 16 {
            rr.Data = net.IP(rd).String()
        }
    case TypeNS, TypeCNAME, TypePTR:
        rr.Data, _, err = readName(msg, off)
    case TypeMX:
        if rdlen >= 3 {
            var host string
            host, _, err = readName(msg, off+2)
            rr.Data = fmt.Sprintf("%d %s", binary.BigEndian.Uint16(rd), host)
        }
    case TypeSOA:
        var mname, rname string
        var p int
        if mname, p, err = readName(msg, off); err == nil {
            if rname, p, err = readName(msg, p); err == nil && p+20 <= off+rdlen {
                rr.Data = fmt.Sprintf("%s %s %d %d %d %d %d", mname, rname,
                    binary.BigEndian.Uint32(msg[p:]),
binary.BigEndian.Uint32(msg[p+4:]),
                    binary.BigEndian.Uint32(msg[p+8:]),
binary.BigEndian.Uint32(msg[p+12:]),
                    binary.BigEndian.Uint32(msg[p+16:]))
            }
        }
    case TypeTXT:
        var parts []string
        for i := 0; i < len(rd); {
            n := int(rd[i])
            i++
            if i+n > len(rd) {
                break
```

```
            }
            parts = append(parts, fmt.Sprintf("%q", rd[i:i+n]))
            i += n
        }
        rr.Data = strings.Join(parts, " ")
    default:
        rr.Data = fmt.Sprintf("\\# %d %x", rdlen, rd)
    }
    if err != nil {
        return rr, 0, err
    }
    return rr, off + rdlen, nil
}
```

6.1.4 The iterative resolution loop (resolver.go)

The `resolve` function implements the algorithm of Section 4.1.4. It checks the cache, starts at the closest known zone and then loops: query a server, handle NXDOMAIN and error codes, look for direct answers and CNAMEs whose owner is the queried name, detect NODATA, and otherwise validate and follow the referral, resolving a name server's address itself when there is no usable glue.

```
func (r *Resolver) resolve(name string, qtype RRType, depth, hops int) (*Result, error) {
    if depth > 8 {
        return nil, ErrLoop
    }
    if rrs, ok := r.cacheGet(name, qtype); ok {
        r.tracef(depth, "%s %s %s", r.c(cGreen, "✓ cache hit"), name, qtype)
        return &Result{Rcode: RcodeNoError, Answers: rrs, Hops: hops}, nil
    }

    zone, servers := r.closestKnownZone(name)
    if zone != "." {
        r.tracef(depth, "%s starting at cached zone %s", r.c(cDim, "↷"), r.c(cBold, zone))
    }

    for ; hops < r.MaxHops; hops++ {
        msg, ns, rtt, err := r.queryAny(servers, name, qtype, depth)
        if err != nil {
            return nil, fmt.Errorf("querying %s servers for %s: %w", zone, name, err)
        }
        if msg.Header.Rcode == RcodeNXDomain {
            r.tracef(depth, "%s %s says %s (%s)", r.c(cRed, "✗"), ns.Name, r.c(cRed,
"NXDOMAIN"), rtt)
            return &Result{Rcode: RcodeNXDomain, Authority: msg.Authority, Hops: hops + 1},
nil
        }
        if msg.Header.Rcode != RcodeNoError {
            r.tracef(depth, "%s %s returned %s, trying next server", r.c(cYellow, "!"),
ns.Name, rcodeString(msg.Header.Rcode))
            servers = without(servers, ns)
            if len(servers) == 0 {
                return &Result{Rcode: msg.Header.Rcode, Hops: hops + 1}, nil
            }
            continue
        }

        if len(msg.Answers) > 0 {
            var direct, cnames []RR
            for _, rr := range msg.Answers {
                if !strings.EqualFold(rr.Name, name) {
```

```
                    continue
                }
                if rr.Type == qtype {
                    direct = append(direct, rr)
                } else if rr.Type == TypeCNAME {
                    cnames = append(cnames, rr)
                }
            }
            if len(direct) > 0 {
                r.tracef(depth, "%s %s answered %s (%s)", r.c(cGreen, "✓"), ns.Name,
                    r.c(cGreen, fmt.Sprintf("%d %s record(s)", len(direct), qtype)), rtt)
                r.cachePut(name, qtype, direct)
                return &Result{Rcode: RcodeNoError, Answers: direct, Hops: hops + 1}, nil
            }
            if len(cnames) > 0 {
                target := fqdn(cnames[0].Data)
                r.tracef(depth, "%s %s is an alias → %s, restarting from the top",
r.c(cCyan, "↪ CNAME"), name, r.c(cBold, target))
                sub, err := r.resolve(target, qtype, depth+1, hops+1)
                if err != nil {
                    return nil, err
                }
                sub.Answers = append([]RR{cnames[0]}, sub.Answers...)
                return sub, nil
            }
        }
        if msg.Header.AA {
            r.tracef(depth, "%s %s: name exists but has no %s records (NODATA)",
r.c(cYellow, "Ø"), ns.Name, qtype)
            return &Result{Rcode: RcodeNoError, Authority: msg.Authority, Hops: hops + 1},
nil
        }

        var nsRRs []RR
        var child string
        for _, rr := range msg.Authority {
            if rr.Type == TypeNS {
                nsRRs = append(nsRRs, rr)
                child = strings.ToLower(rr.Name)
            }
        }
        if len(nsRRs) == 0 {
            return nil, fmt.Errorf("lame response from %s: no answer, no referral",
ns.Name)
        }

        if !isSubdomain(name, child) || !isSubdomain(child, zone) || child == zone {
            return nil, fmt.Errorf("out-of-bailiwick referral from %s: %s is not under %s",
ns.Name, child, zone)
        }

        next := r.glueFor(nsRRs, msg.Additional, zone)
        names := make([]string, 0, len(nsRRs))
        for _, rr := range nsRRs {
            names = append(names, strings.TrimSuffix(rr.Data, "."))
        }
        r.tracef(depth, "%s %s → referral to %s %s (%s)", r.c(cCyan, "↓"), ns.Name,
r.c(cBold, child),
            r.c(cDim, fmt.Sprintf("[%s]", summarize(names, 3))), rtt)

        if len(next) == 0 {
            r.tracef(depth, "%s no glue for %s — resolving a nameserver's address first",
r.c(cYellow, "…"), child)
            for _, nsrr := range shuffled(nsRRs) {
                sub, err := r.resolve(fqdn(nsrr.Data), TypeA, depth+1, 0)
                if err != nil || sub.Rcode != RcodeNoError {
                    continue
                }
                for _, a := range sub.Answers {
                    if a.Type == TypeA {
                        next = append(next, nameserver{fqdn(nsrr.Data), a.Data})
                    }
```

```
                }
                if len(next) > 0 {
                    break
                }
            }
            if len(next) == 0 {
                return nil, fmt.Errorf("could not find an address for any %s nameserver",
child)
            }
        }

        r.mu.Lock()
        r.zoneNS[child] = next
        r.mu.Unlock()
        zone, servers = child, next
    }
    return nil, ErrLoop
}
```

6.1.5 Bailiwick helpers (resolver.go)

`isSubdomain` compares names on label boundaries (so `evilgithub.com.` is not under `github.com.`). `glueFor` keeps only address records that belong to one of the referred name servers and lie inside the zone of the server that sent them.

```
// isSubdomain reports whether child is at or below parent ("a.b.c." under "b.c.").
func isSubdomain(child, parent string) bool {
    if parent == "." {
        return true
    }
    return child == parent || strings.HasSuffix(child, "."+parent)
}

// glueFor pairs NS names with A records from the Additional section.
// Glue is only trusted if it lies inside the zone of the server that sent it
// (askedZone) — a .com server may vouch for ns1.example.com, not for ns.evil.org.
func (r *Resolver) glueFor(nsRRs, additional []RR, askedZone string) []nameserver {
    want := map[string]bool{}
    for _, rr := range nsRRs {
        want[fqdn(rr.Data)] = true
    }
    var out []nameserver
    for _, rr := range additional {
        n := fqdn(rr.Name)
        if rr.Type == TypeA && want[n] && isSubdomain(n, askedZone) {
            out = append(out, nameserver{n, rr.Data})
        }
    }
    return out
}
```

6.1.6 Anti-spoofing exchange and TCP fallback (resolver.go)

`exchange` sends one query from a new connected UDP socket with a deadline and keeps reading until a datagram passes all reply checks; forged or stray packets are

skipped. A truncated reply causes the same query to be sent over TCP with two-byte length framing.

```
// exchange sends one query over UDP, retrying over TCP if the answer is truncated.
func (r *Resolver) exchange(ip, name string, qtype RRType) (*Message, error) {
    id := uint16(rand.Uint32())
    q, err := BuildQuery(id, name, qtype)
    if err != nil {
        return nil, err
    }
    addr := net.JoinHostPort(ip, r.Port)

    conn, err := net.DialTimeout("udp", addr, r.Timeout)
    if err != nil {
        return nil, err
    }
    defer conn.Close()
    conn.SetDeadline(time.Now().Add(r.Timeout))
    if _, err := conn.Write(q); err != nil {
        return nil, err
    }
    buf := make([]byte, 4096)
    for {
        n, err := conn.Read(buf)
        if err != nil {
            return nil, err
        }
        m, err := ParseMessage(buf[:n])
        if err != nil {
            return nil, err
        }
        if m.Header.ID != id || !m.Header.QR || len(m.Questions) != 1 ||
            !strings.EqualFold(m.Questions[0].Name, name) || m.Questions[0].Type !=
qtype {
            continue
        }
        if m.Header.TC {
            r.tracef(0, "%s response truncated, retrying over TCP", r.c(cYellow,
"⇄"))
            return exchangeTCP(addr, q, id, r.Timeout)
        }
        return m, nil
    }
}

// DNS over TCP: same message, prefixed with a 2-byte length (RFC 1035 §4.2.2).
func exchangeTCP(addr string, q []byte, id uint16, timeout time.Duration) (*Message,
error) {
    conn, err := net.DialTimeout("tcp", addr, timeout)
    if err != nil {
        return nil, err
    }
    defer conn.Close()
    conn.SetDeadline(time.Now().Add(timeout))
    frame := binary.BigEndian.AppendUint16(nil, uint16(len(q)))
    if _, err := conn.Write(append(frame, q...)); err != nil {
        return nil, err
    }
    var lb [2]byte
    if _, err := io.ReadFull(conn, lb[:]); err != nil {
        return nil, err
    }
    buf := make([]byte, binary.BigEndian.Uint16(lb[:]))
    if _, err := io.ReadFull(conn, buf); err != nil {
        return nil, err
    }
    m, err := ParseMessage(buf)
```

```
    if err != nil {
        return nil, err
    }
    if m.Header.ID != id {
        return nil, errors.New("TCP response ID mismatch")
    }
    return m, nil
}
```

6.1.7 Caches (resolver.go)

The answer cache stores an RRset until its shortest TTL expires and rewrites TTLs to the time remaining on every hit. `closestKnownZone` walks up the queried name, label by label, to find the deepest cached delegation.

```
func (r *Resolver) cacheGet(name string, t RRType) ([]RR, bool) {
    r.mu.Lock()
    defer r.mu.Unlock()
    e, ok := r.cache[name+"|"+t.String()]
    if !ok || time.Now().After(e.expires) {
        return nil, false
    }
    left := uint32(time.Until(e.expires).Seconds())
    out := make([]RR, len(e.rrs))
    for i, rr := range e.rrs {
        rr.TTL = left
        out[i] = rr
    }
    return out, true
}

func (r *Resolver) cachePut(name string, t RRType, rrs []RR) {
    if len(rrs) == 0 {
        return
    }
    min := rrs[0].TTL
    for _, rr := range rrs {
        if rr.TTL < min {
            min = rr.TTL
        }
    }
    r.mu.Lock()
    r.cache[name+"|"+t.String()] = cacheEntry{rrs, time.Now().Add(time.Duration(min) *
time.Second)}
    r.mu.Unlock()
}

func (r *Resolver) closestKnownZone(name string) (string, []nameserver) {
    r.mu.Lock()
    defer r.mu.Unlock()
    for z := name; ; {
        if ns, ok := r.zoneNS[z]; ok && len(ns) > 0 {
            return z, ns
        }
        if z == "." {
            return ".", r.Roots
        }
        if i := strings.IndexByte(z, '.'); i >= 0 && i+1 < len(z) {
            z = z[i+1:]
        } else {
            z = "."
        }
```

```
    }
}
```

6.1.8 Recursive server (server.go)

The server reads datagrams in a loop and hands each one to a goroutine. Invalid packets are dropped, failures are reported as SERVFAIL, and `encodeRData` re-encodes names inside RDATA so that no compression pointer from an upstream packet leaks into the response.

```
func encodeRData(rr RR) ([]byte, error) {
    switch rr.Type {
    case TypeNS, TypeCNAME, TypePTR:
        return encodeName(nil, rr.Data)
    case TypeMX:
        var pref uint16
        var host string
        if _, err := fmt.Sscanf(rr.Data, "%d %s", &pref, &host); err != nil {
            return nil, err
        }
        return encodeName(binary.BigEndian.AppendUint16(nil, pref), host)
    case TypeSOA:
        f := strings.Fields(rr.Data)
        if len(f) != 7 {
            return nil, fmt.Errorf("bad SOA %q", rr.Data)
        }
        b, err := encodeName(nil, f[0])
        if err != nil {
            return nil, err
        }
        if b, err = encodeName(b, f[1]); err != nil {
            return nil, err
        }
        for _, s := range f[2:] {
            var v uint32
            fmt.Sscan(s, &v)
            b = binary.BigEndian.AppendUint32(b, v)
        }
        return b, nil
    default:
        return rr.Raw, nil
    }
}

func serve(addr string, r *Resolver) error {
    pc, err := net.ListenPacket("udp", addr)
    if err != nil {
        return err
    }
    log.Printf("dnsr listening on udp://%s — try: dig @%s -p %s example.com",
        pc.LocalAddr(), strings.Split(pc.LocalAddr().String(), ":")[0],
strings.Split(pc.LocalAddr().String(), ":")[1])

    buf := make([]byte, 1500)
    for {
        n, client, err := pc.ReadFrom(buf)
        if err != nil {
            return err
        }
        pkt := append([]byte(nil), buf[:n]...)
        go func() {
```

```
            m, err := ParseMessage(pkt)
            if err != nil || len(m.Questions) != 1 || m.Header.QR {
                return
            }
            q := m.Questions[0]
            res, err := r.Resolve(q.Name, q.Type)
            var out []byte
            if err != nil {
                log.Printf("%-30s %-5s SERVFAIL (%v)", q.Name, q.Type, err)
                out = buildResponse(m.Header.ID, q, m.Header.RD, RcodeServFail, nil,
nil, nil)
            } else {
                log.Printf("%-30s %-5s %-8s %d answer(s), %d hops, %s", q.Name,
q.Type,
                    rcodeString(res.Rcode), len(res.Answers), res.Hops,
res.Elapsed.Round(1e6))
                out = buildResponse(m.Header.ID, q, m.Header.RD, res.Rcode,
res.Answers, res.Authority, nil)
            }
            pc.WriteTo(out, client)
        }()
    }
}
```

6.1.9 Adversarial tests (resolver_test.go)

Two of the nine tests check the security properties directly against the hostile servers of the fake hierarchy: poisoned glue for `ns.dnshost.org` must be ignored, and a referral from the `com.` server to `net.` must be rejected.

```
func TestGluelessDelegationIgnoresPoisonedGlue(t *testing.T) {
    r, trace := newTestResolver(t)
    res, err := r.Resolve("glueless.com", TypeA)
    if err != nil || len(res.Answers) != 1 || res.Answers[0].Data != "10.0.0.1" {
        t.Fatalf("%+v %v\n%s", res, err, trace)
    }
    if strings.Contains(trace.String(), "6.6.6.6") {
        t.Errorf("used out-of-bailiwick glue:\n%s", trace)
    }
    if !strings.Contains(trace.String(), "no glue") {
        t.Errorf("expected glueless path:\n%s", trace)
    }
}

func TestOutOfBailiwickReferralRejected(t *testing.T) {
    r, _ := newTestResolver(t)
    _, err := r.Resolve("www.trap.com", TypeA)
    if err == nil || !strings.Contains(err.Error(), "bailiwick") {
        t.Fatalf("expected bailiwick error, got %v", err)
    }
}
```

6.1.10 Fuzzing and benchmark harness

For the robustness evaluation in Chapter 7 an additional test file was used. `FuzzParseMessage` feeds inputs generated by Go's coverage-guided fuzzer to

`ParseMessage`; `TestRandomAndMutatedPackets` parses one million random and mutated packets with a fixed seed so that the run can be repeated; the two benchmarks measure the cost of parsing a reply and building a query.

```
import (
    "math/rand"
    "testing"
)

func seedPackets() [][]byte {
    q, _ := BuildQuery(0x1234, "www.example.com.", TypeA)
    resp := []byte{
        0x12, 0x34, 0x81, 0x80, 0, 1, 0, 1, 0, 0, 0, 0,
        3, 'w', 'w', 'w', 7, 'e', 'x', 'a', 'm', 'p', 'l', 'e', 3, 'c', 'o', 'm', 0,
        0, 1, 0, 1,
        0xC0, 0x0C, 0, 1, 0, 1, 0, 0, 0x0E, 0x10, 0, 4, 1, 2, 3, 4,
    }
    return [][]byte{q, resp}
}

func FuzzParseMessage(f *testing.F) {
    for _, s := range seedPackets() {
        f.Add(s)
    }
    f.Fuzz(func(t *testing.T, b []byte) {
        ParseMessage(b)
    })
}

func TestRandomAndMutatedPackets(t *testing.T) {
    rng := rand.New(rand.NewSource(42))
    seeds := seedPackets()
    errs, oks := 0, 0
    for i := 0; i < 1_000_000; i++ {
        var b []byte
        if i%2 == 0 {
            b = make([]byte, rng.Intn(600))
            rng.Read(b)
        } else {
            s := seeds[rng.Intn(len(seeds))]
            b = append([]byte(nil), s...)
            for k := 0; k < 1+rng.Intn(6); k++ {
                b[rng.Intn(len(b))] = byte(rng.Intn(256))
            }
            if rng.Intn(4) == 0 {
                b = b[:rng.Intn(len(b))]
            }
        }
        if _, err := ParseMessage(b); err != nil {
            errs++
        } else {
            oks++
        }
    }
    t.Logf("1,000,000 packets: %d rejected with error, %d parsed, 0 panics", errs,
oks)
}

func BenchmarkParseMessage(b *testing.B) {
    p := seedPackets()[1]
    for i := 0; i < b.N; i++ {
        ParseMessage(p)
    }
}
```

```
func BenchmarkBuildQuery(b *testing.B) {
    for i := 0; i < b.N; i++ {
        BuildQuery(uint16(i), "www.example.com.", TypeA)
    }
}
```

6.2 Sample Screen Shots

The screenshots below were captured on the Windows 11 test machine in PowerShell. Live runs query the real root, TLD and authoritative servers, so the exact servers chosen and the timings differ from run to run. In the trace, → marks a query sent, ↓ a referral, ↪ a CNAME, ⋯ a glueless delegation, ✓ an answer, ✗ NXDOMAIN, ∅ NODATA and ⇄ a TCP retry; nested work is indented with │.

6.2.1 Build and offline test suite

Fig. 6.1 shows the program being built with the standard Go tool chain and the nine offline tests passing against the fake DNS hierarchy on loopback.

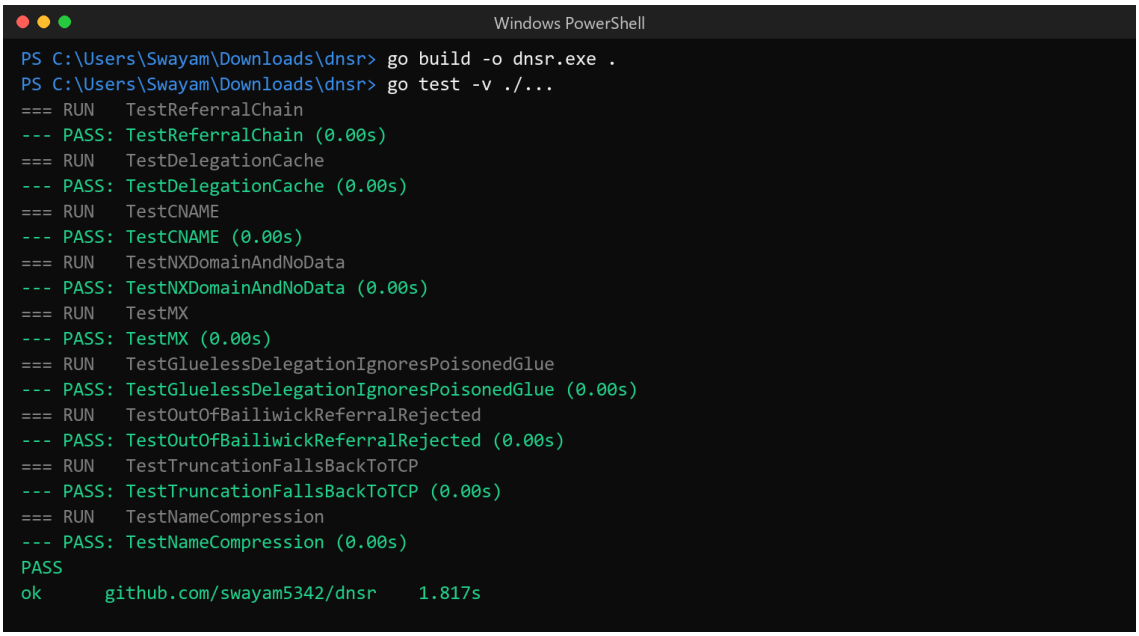


Fig. 6.1 Building DNSR and Running the Offline Test Suite

6.2.2 Usage

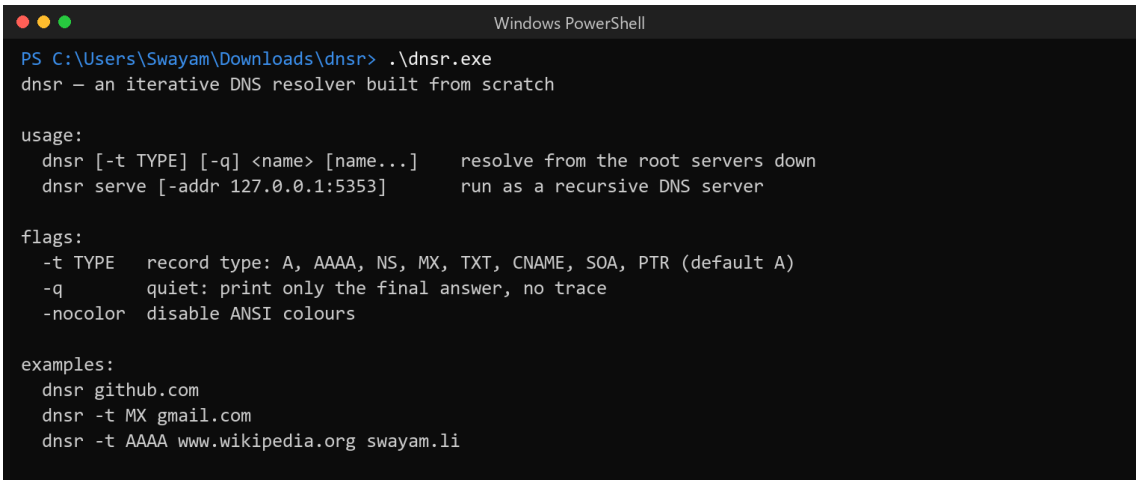


Fig. 6.2 Usage Message Printed When No Name Is Given

6.2.3 Iterative resolution with a CNAME

Fig. 6.3 shows a cold-cache resolution of `www.github.com`. The root refers DNSR to `com.`, a `.com` server refers it to `github.com.`, and GitHub's name server answers with a CNAME. DNSR restarts for `github.com` but begins at the cached `github.com.` zone, so the final answer needs only one more query.

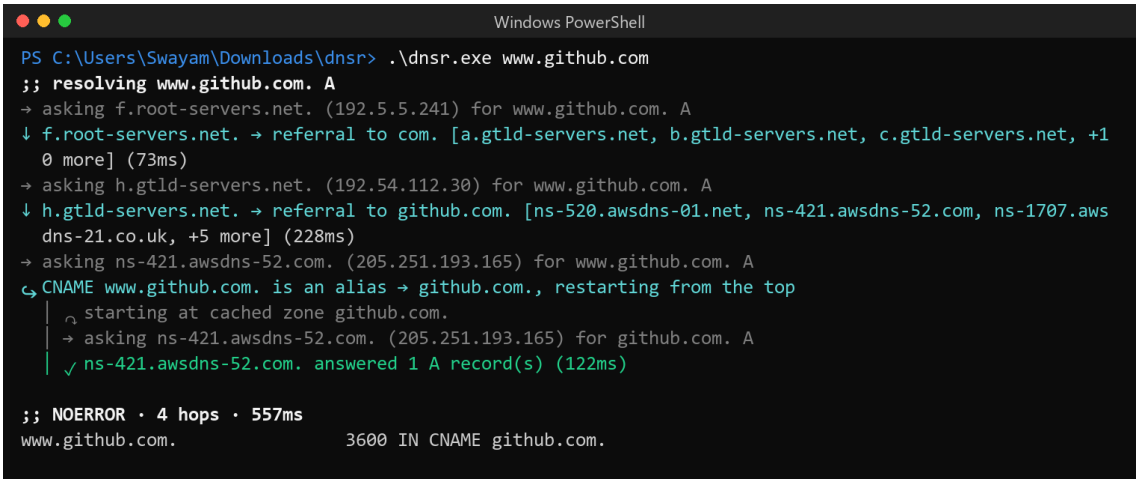


Fig. 6.3 Resolving www.github.com from the Root Servers

6.2.4 MX and AAAA lookups

Fig. 6.4 resolves the mail exchangers of `gmail.com`, showing the MX preference values. Fig. 6.5 resolves the IPv6 address of `www.wikipedia.org`, which is a CNAME to `dyna.wikimedia.org`; because the `org.` delegation is already cached, the nested resolution for the target starts at `org.` rather than at the root.

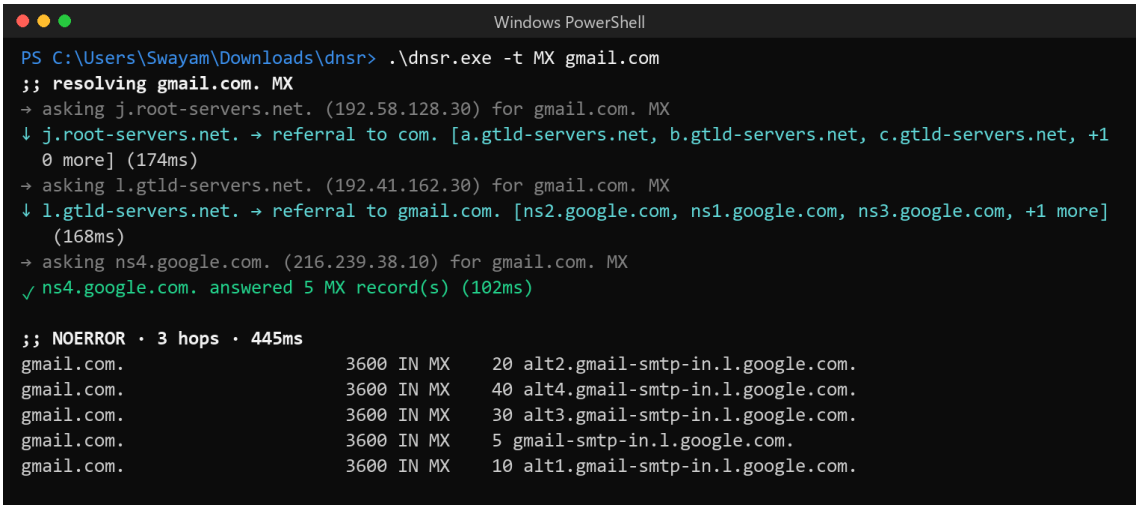


Fig. 6.4 MX Lookup for gmail.com

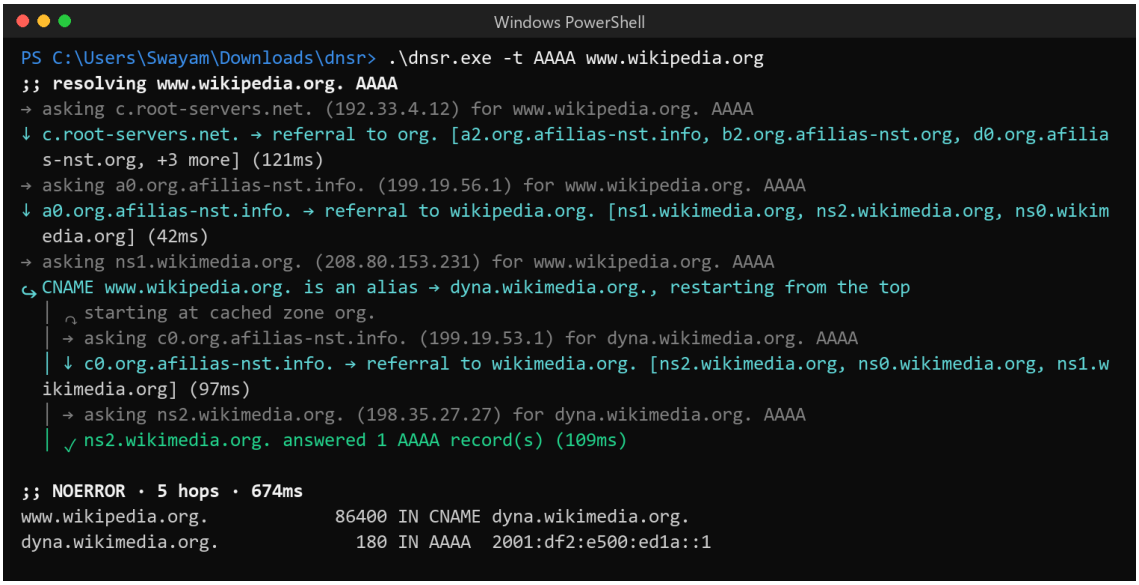


Fig. 6.5 AAAA Lookup for www.wikipedia.org Through a CNAME

6.2.5 Negative answers

Fig. 6.6 shows both kinds of negative answer. A name that does not exist under `.com` produces NXDOMAIN directly from the `.com` server, together with its SOA record. `github.com` exists but has no IPv6 address, so GitHub's authoritative server replies with AA = 1, no answer and an SOA record, which DNSR reports as NODATA with the NOERROR code.

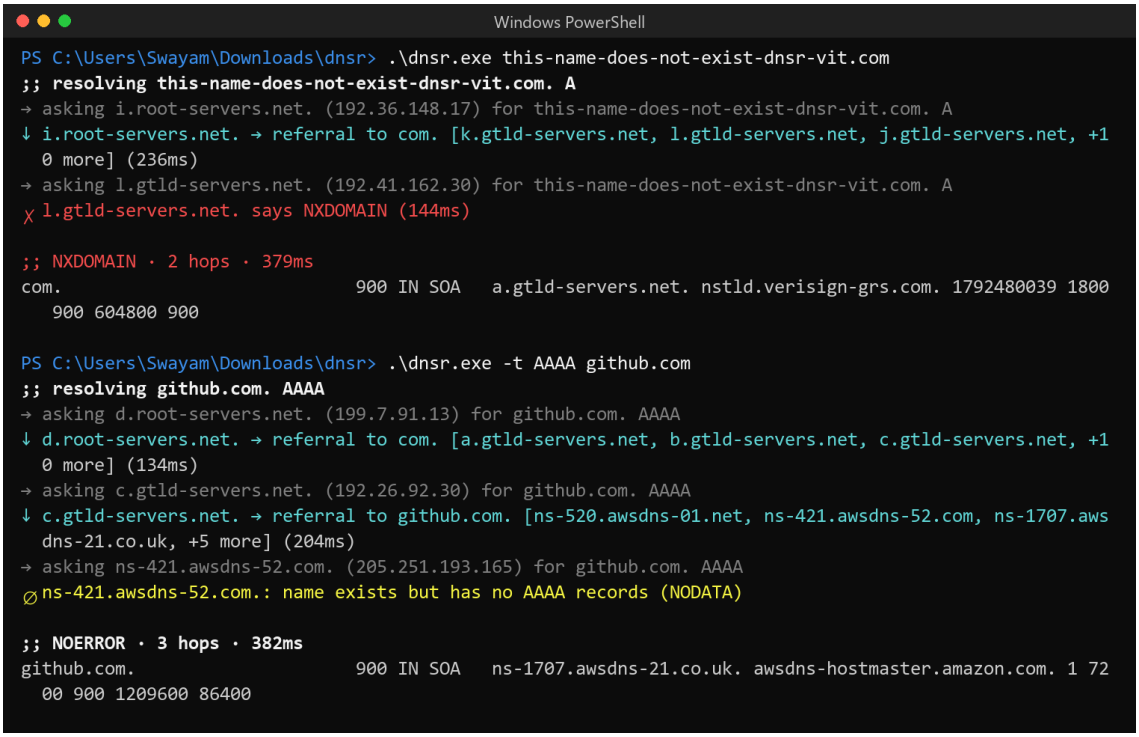


Fig. 6.6 NXDOMAIN and NODATA Responses

6.2.6 Glueless delegations

The name servers of `github.io` are in `.net` and `.co.uk`, so the `.io` server cannot provide glue for them. Fig. 6.7 shows DNSR pausing the main resolution, resolving the address of `dns3.p05.nsone.net` through the `.net` hierarchy, and then continuing. Fig. 6.8 shows the same behaviour for the university's own domain `vit.ac.in`, whose name servers are hosted on Amazon Route 53.

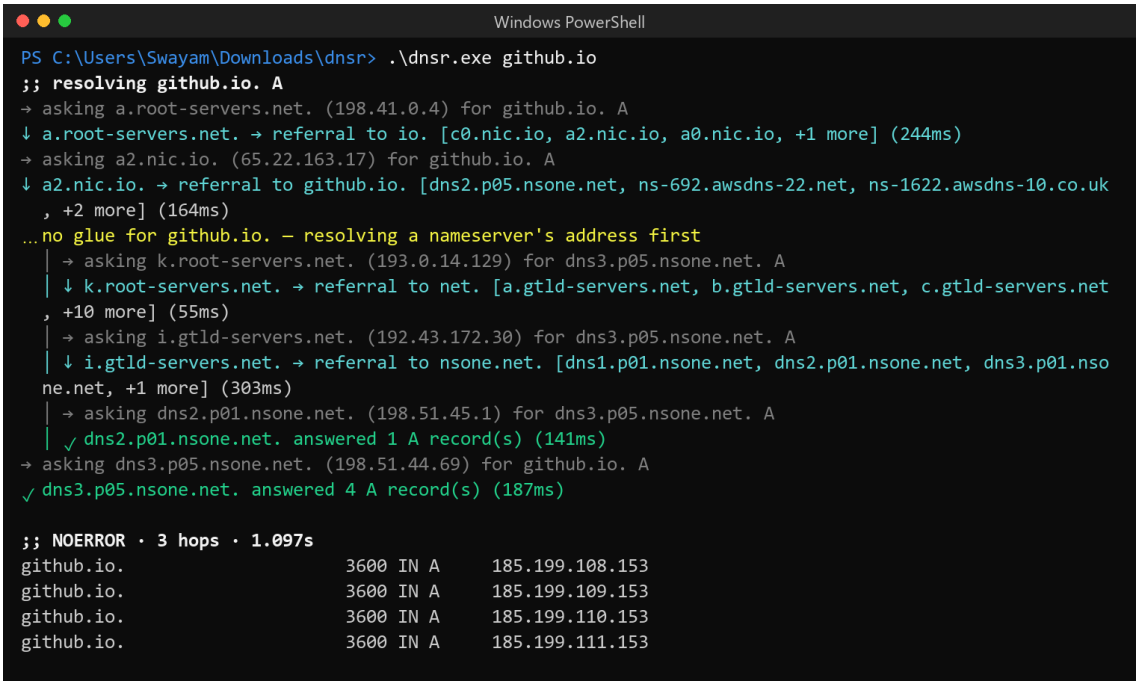


Fig. 6.7 Glueless Delegation for github.io

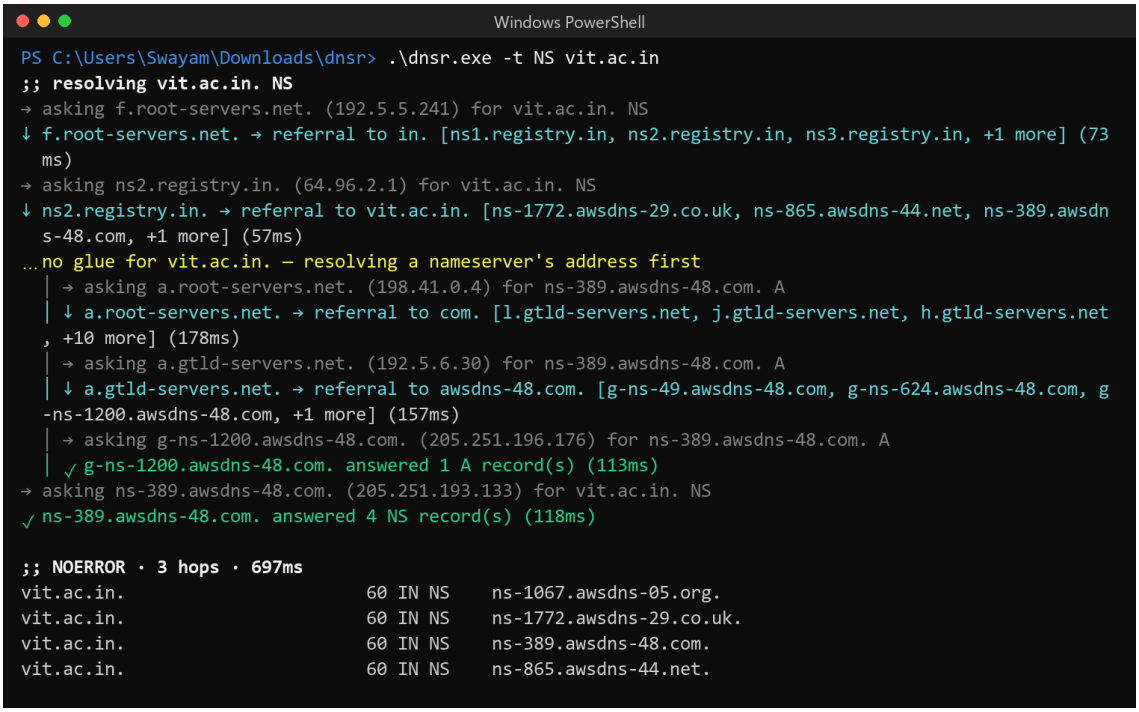


Fig. 6.8 Glueless Delegation and NS Lookup for vit.ac.in

6.2.7 Fallback from UDP to TCP

The TXT records of `apple.com` do not fit in a 1232-byte UDP reply. In Fig. 6.9 the authoritative server sets the TC bit and DNSR repeats the query over TCP, receiving all 23 TXT records (the screenshot shows the first lines of the answer).

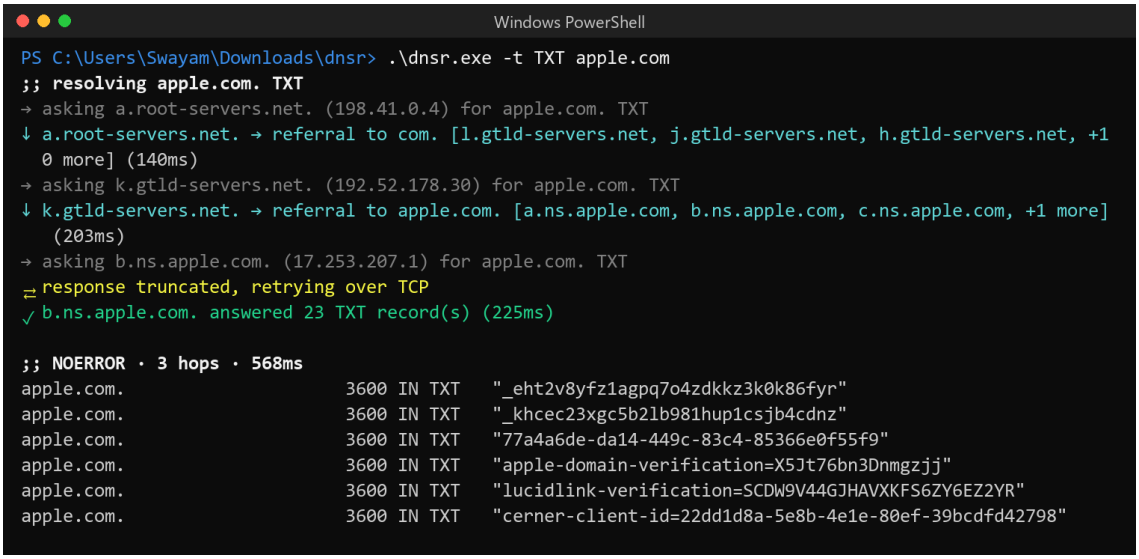


Fig. 6.9 Truncated UDP Reply Retried over TCP for apple.com TXT

6.2.8 Effect of the caches

In Fig. 6.10 three names are resolved in one run. The first lookup walks from the root (4 hops, 557 ms). The second, `api.github.com`, starts at the cached `github.com.` zone and needs a single query (125 ms). The third repeats the first name: the alias is asked once more, and the target `github.com` comes straight from the answer cache with its TTL reduced from 60 to 59 seconds.

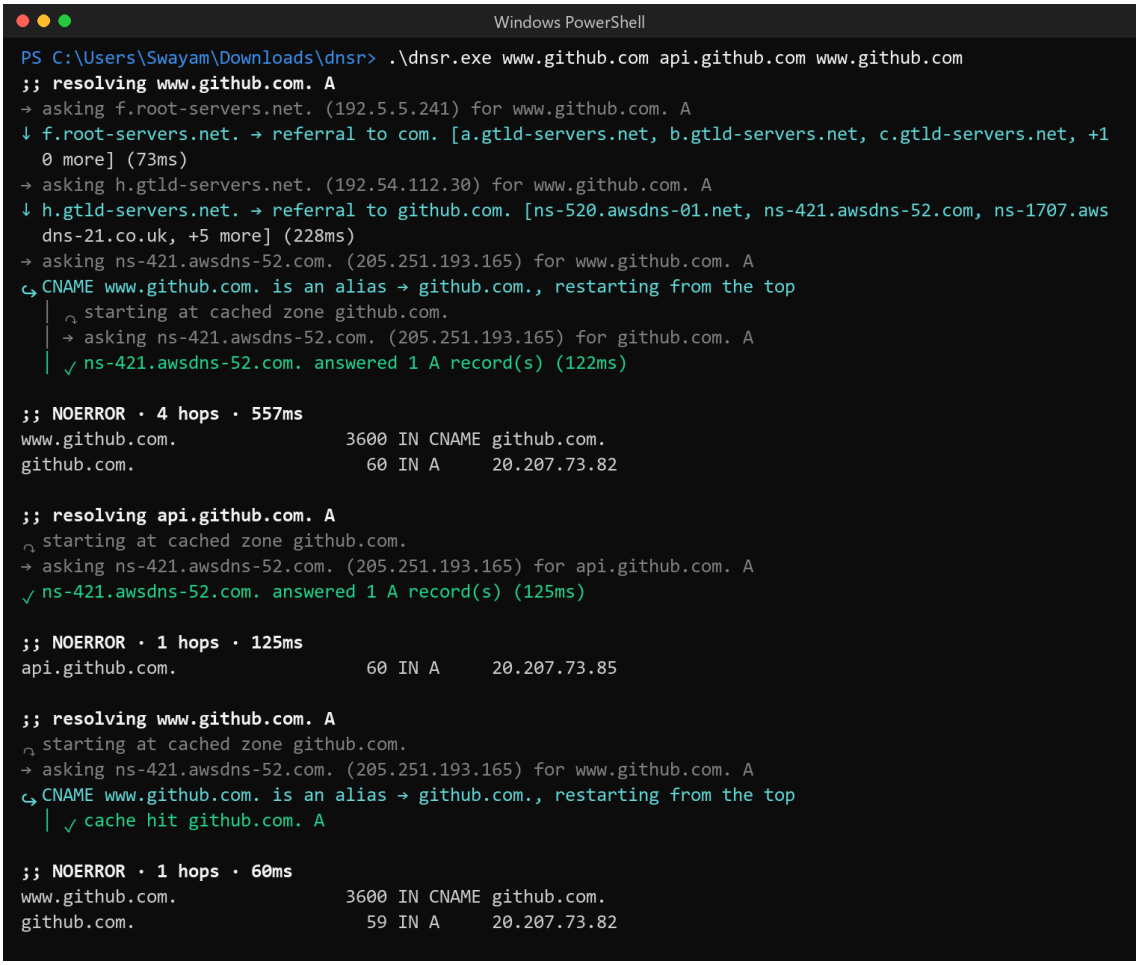


Fig. 6.10 Delegation Cache and Answer Cache in One Run

6.2.9 Quiet mode

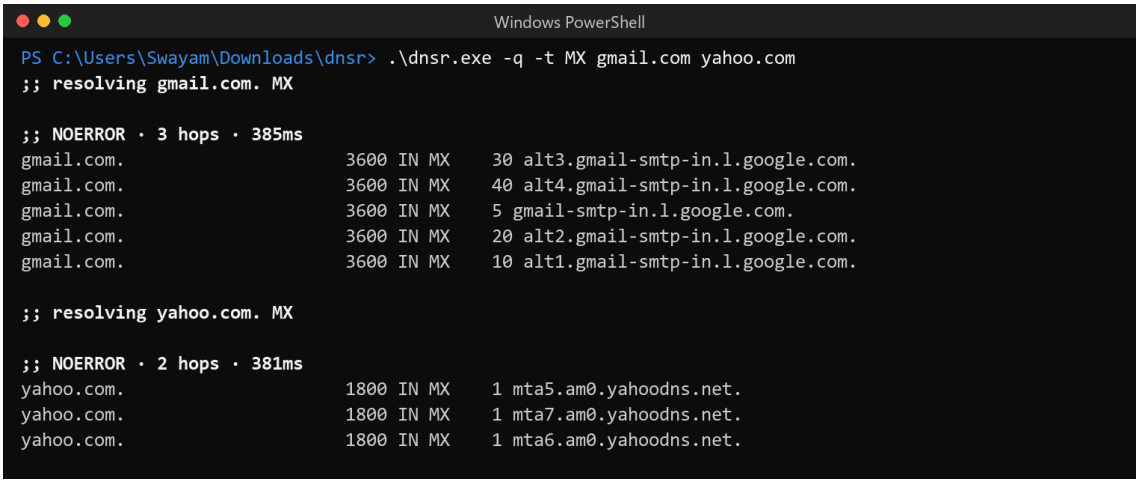


Fig. 6.11 Quiet Mode Printing Only the Answers

6.2.10 Recursive server mode

Fig. 6.12 shows DNSR running as a server on 127.0.0.1:8053 and logging each query, and Fig. 6.13 shows the client side. The client is an independent DNS implementation (dnspython [31]), so these screenshots also show that DNSR's responses are valid DNS messages. The second query for `www.github.com` takes 75 ms instead of 2,472 ms because the server's caches are warm. (The Windows `nslookup` tool ignores a custom port, which is why a Python client was used.)

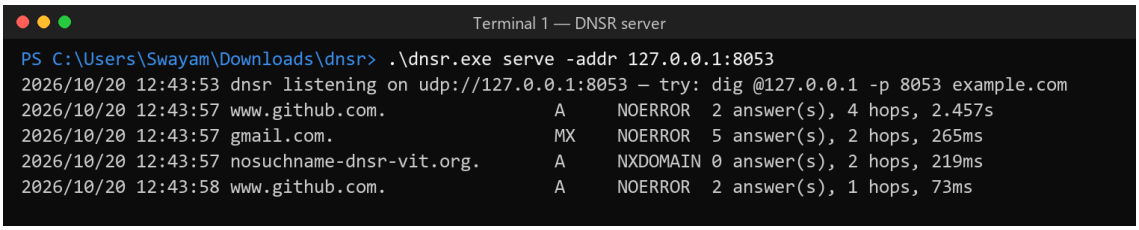


Fig. 6.12 DNSR Running as a Recursive Server

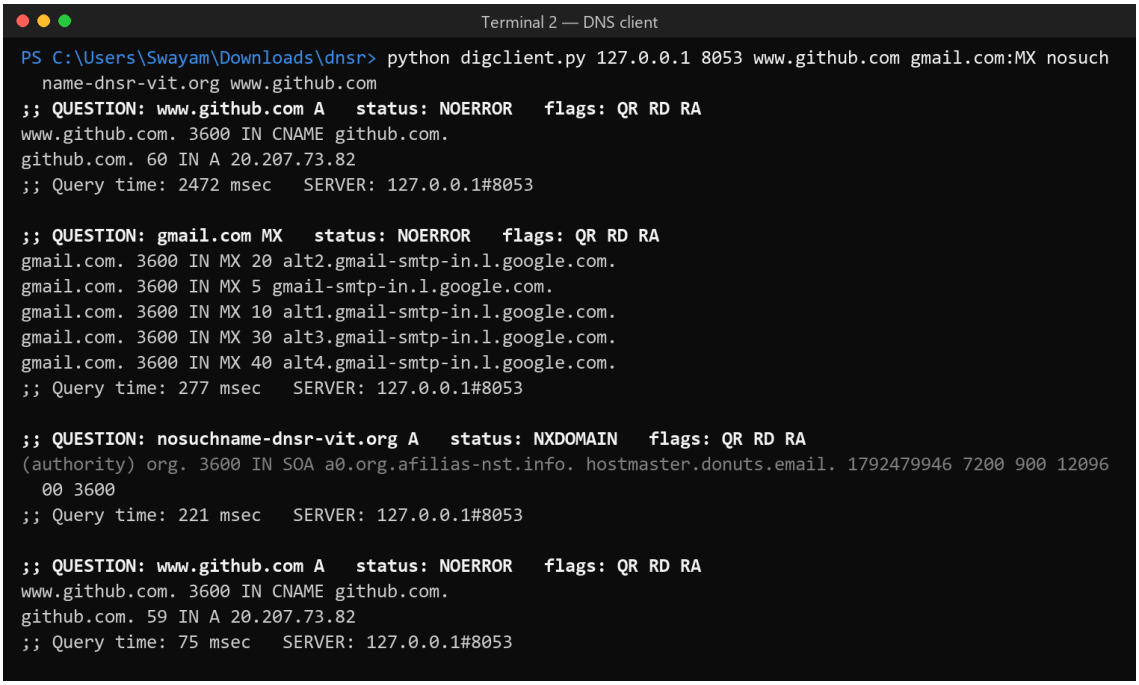


Fig. 6.13 A DNS Client Querying the DNSR Server

6.2.11 Fuzzing and benchmarks

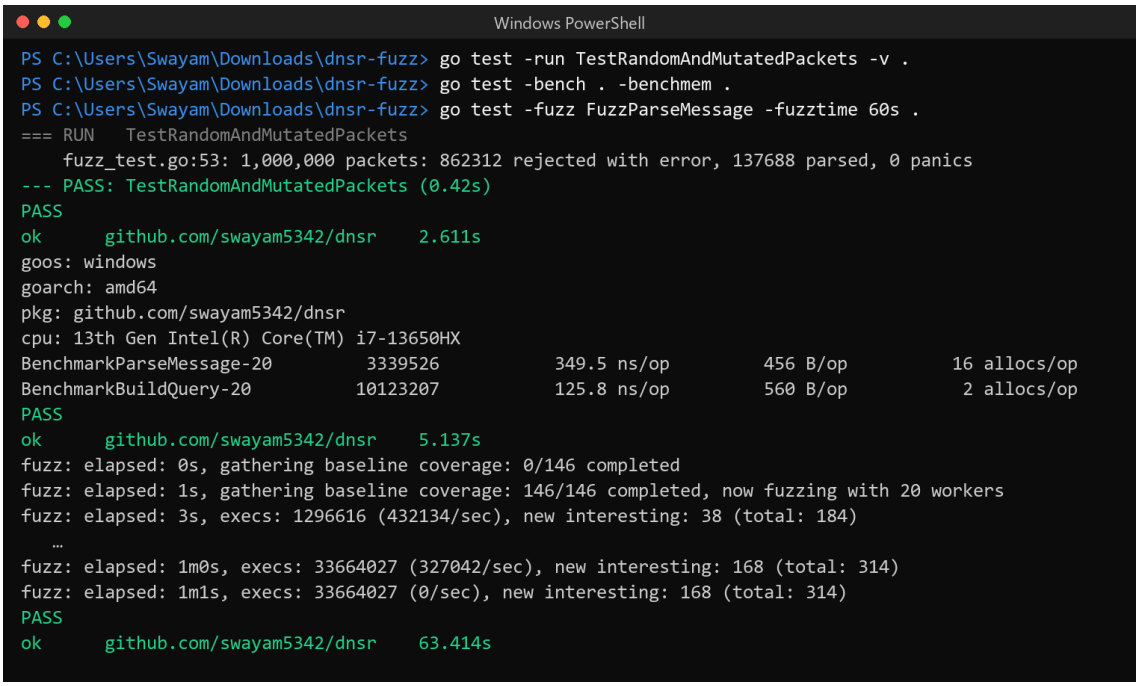


Fig. 6.14 Random-Packet Test, Benchmarks and Coverage-Guided Fuzzing

CHAPTER 7

RESULT AND DISCUSSION

This chapter evaluates DNSR against the goals of Section 2.4. It reports functional test results, correctness and latency of live resolution compared with a public resolver, the effect of the caches, the behaviour of server mode, the security and robustness evaluation, codec performance and the findings of a security review of the code. It ends with a short cost analysis and a discussion of limitations.

7.1 Experimental Setup

All experiments were run on the machine described in Table 3.3, connected to the Internet over a home broadband link in India, on 20 October 2026. For the live experiments a measurement script started a fresh DNSR process for each run (so every run began with empty caches), parsed the summary line of the trace, and compared the answers with those of Google Public DNS (8.8.8.8) queried through dnspython. Fifteen popular domains were chosen to cover different TLDs (`.com`, `.org`, `.in`, `.io`, `.dev`), different DNS hosting providers and both direct answers and CNAME chains. Each domain was resolved three times. The latency of one resolution is the wall-clock time reported by DNSR; the mean latency of a set of runs is given by Eqn. 7.1.

$$\bar{L} = \frac{1}{n}\sum_{i=1}^{n} L_i \tag{7.1}$$

7.2 Functional Verification

The offline suite of Section 4.1.11 was run with `go test -v`. All nine tests passed (Table 7.1); the whole suite, including starting the six fake servers, completes in under two seconds, so it can be run on every change.

Table 7.1 Offline Test Results

Test	Result	Time
TestReferralChain	PASS	0.01 s
TestDelegationCache	PASS	< 0.01 s
TestCNAME	PASS	< 0.01 s
TestNXDomainAndNoData	PASS	< 0.01 s
TestMX	PASS	< 0.01 s
TestGluelessDelegationIgnoresPoisonedGlue	PASS	< 0.01 s

Test	Result	Time
TestOutOfBailiwickReferralRejected	PASS	< 0.01 s
TestTruncationFallsBackToTCP	PASS	< 0.01 s
TestNameCompression	PASS	< 0.01 s
Package total (go test)	ok	1.74 s

7.3 Live Resolution: Correctness and Latency

Table 7.2 summarises the live runs. Of the 45 cold-cache runs, 43 completed successfully; the two that did not (one for `vit.ac.in` and one for `go.dev`) ended with an error from DNSR, most likely because the servers tried did not reply within the two-second timeout, and the repeat runs for the same domains succeeded. For all 15 domains DNSR returned the same response code as Google Public DNS (15 of 15), and for 12 domains the returned addresses overlapped with those of 8.8.8.8. The three exceptions, `google.com`, `microsoft.com` and `gmail.com`, are served by global content-delivery networks that return different addresses depending on where the question comes from; DNSR asks the authoritative servers from the user's own network, while 8.8.8.8 asks from Google's data centre, so different but equally valid addresses are expected.

Table 7.2 Live Cold-Cache Resolution of 15 Domains (Three Runs Each)

Domain	Runs OK	Hops	Mean (ms)	Min to max (ms)	A records	Same as 8.8.8.8
google.com	3/3	3	414	408 to 418	6	Code yes, IP differs (CDN)
www.github.com *	3/3	4	1205	541 to 2525	1	Yes
www.wikipedia.org *	3/3	5	1517	773 to 2599	1	Yes
amazon.com	3/3	3	1087	424 to 2381	3	Yes
vit.ac.in	2/3	3	1975	1167 to 2783	1	Yes
microsoft.com	3/3	3	997	276 to 2243	1	Code yes, IP differs (CDN)
cloudflare.com	3/3	3	633	526 to 690	2	Yes
stackoverflow.com	3/3	3	506	419 to 602	1	Yes
example.com	3/3	3	1234	425 to 2799	2	Yes
gmail.com	3/3	3	749	571 to 1041	1	Code yes, IP differs (CDN)
netflix.com	3/3	3	733	519 to 1011	3	Yes
iana.org	3/3	3	452	422 to 473	1	Yes
python.org	3/3	3	328	238 to 444	4	Yes
go.dev	2/3	3	827	749 to 905	4	Yes

Domain	Runs OK	Hops	Mean (ms)	Min to max (ms)	A records	Same as 8.8.8.8
github.io	3/3	3	2763	744 to 4699	4	Yes

* answer reached through a CNAME. Hops count the queries in the main resolution; glueless sub-resolutions (for example for `github.io` and `vit.ac.in`) are additional.

Across all 43 successful runs the mean cold-cache latency was 1011 ms and the median 571 ms, with a minimum of 238 ms and a maximum of 4699 ms. Fig. 7.1 shows the mean and range for each domain. The distribution is strongly skewed: most runs finish in 300 to 800 ms, while a few take two to five seconds. The slow runs coincide with the two-second timeout: when the randomly chosen server does not answer, DNSR waits the full timeout before trying the next one. This is the price of the sequential, polite server-selection strategy discussed in Section 4.3.3, and it is the main motivation for RTT-based server selection as future work.

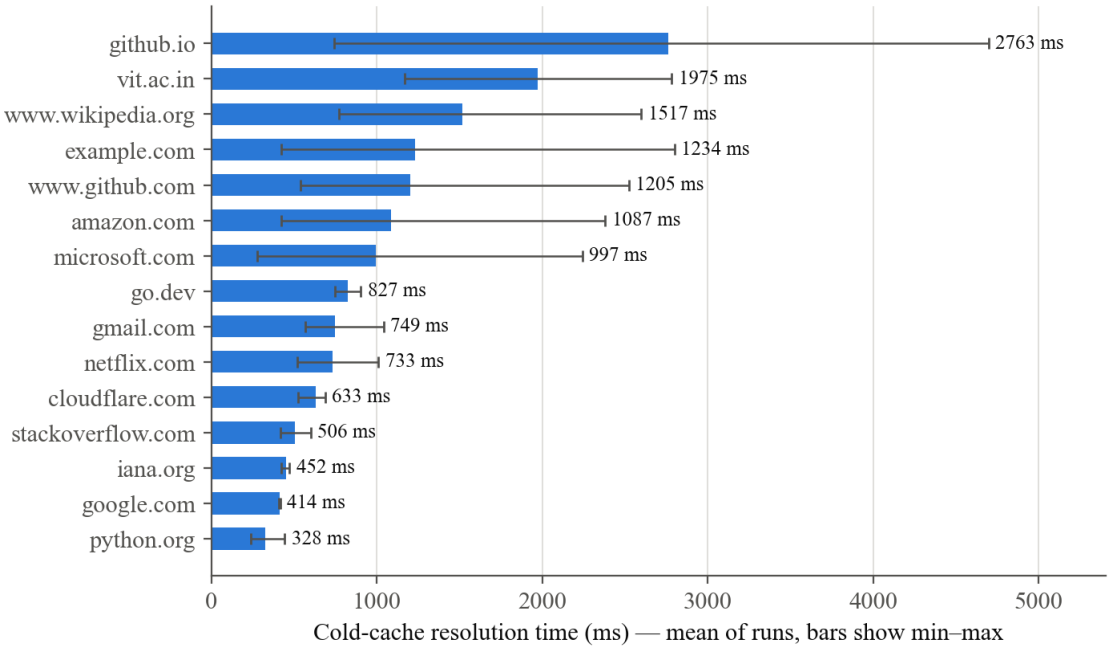


Fig. 7.1 Cold-Cache Resolution Time per Domain

Fig. 7.2 shows the number of queries per resolution. Domains answered directly need three queries (root, TLD, authoritative). Names that are aliases need more: `www.github.com` needs four and `www.wikipedia.org`, whose target lies in a different zone (`wikimedia.org`), needs five. The mean was 3.2 queries per name.

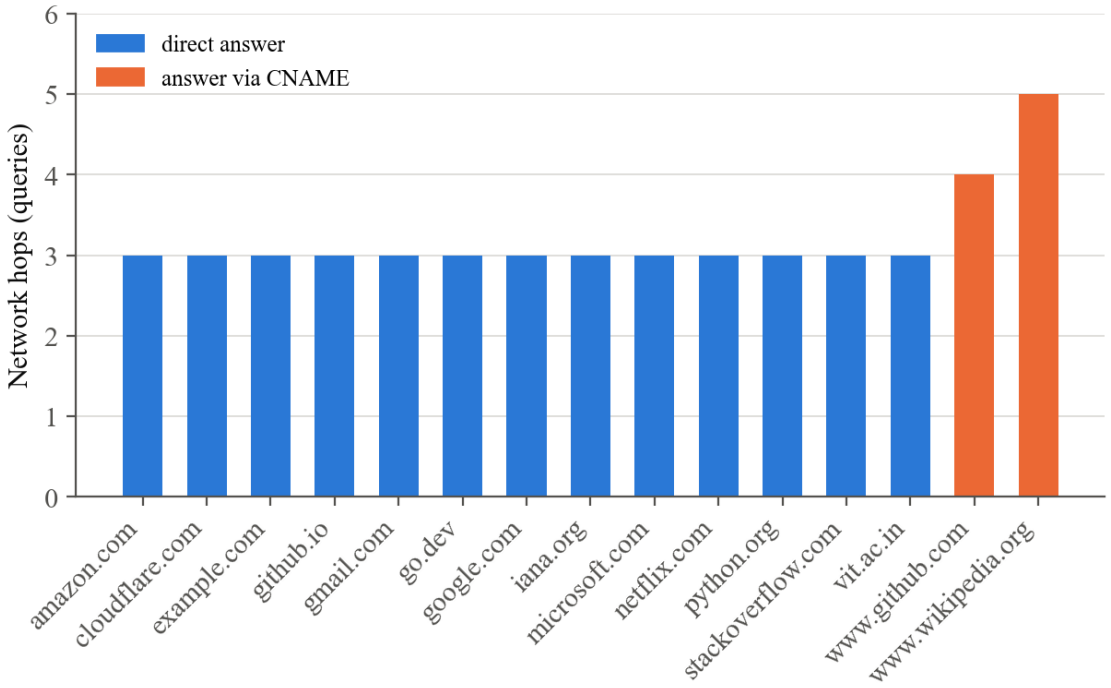


Fig. 7.2 Network Queries per Cold-Cache Resolution

7.4 Effect of Caching

To measure the two caches, five pairs of names were resolved within one process in the order: first name, a sibling name in the same domain, first name again. The first lookup starts with empty caches; the sibling lookup can use the delegation cache; the repeated lookup can use the answer cache. The relative reduction in latency is computed with Eqn. 7.2.

$$\rho = \frac{L_{cold} - L_{cached}}{L_{cold}} \times 100\% \tag{7.2}$$

Table 7.3 Latency with Empty, Delegation and Answer Caches (ms, hops in brackets)

First name / sibling	First (cold)	Sibling	ρ sibling	Repeat	ρ repeat
www.github.com / api.github.com	684 (4)	94 (1)	86%	32 (1)	95%
google.com / mail.google.com	583 (3)	89 (1)	85%	0 (0)	100%
www.wikipedia.org / en.wikipedia.org	1023 (5)	311 (1)	70%	91 (1)	91%
vit.ac.in / www.vit.ac.in	757 (3)	49 (1)	94%	0 (0)	100%
amazon.com / www.amazon.com	358 (3)	589 (4)	-65%	0 (0)	100%

The results, plotted in Fig. 7.3, show that the delegation cache removes the root and TLD steps and cuts the latency of a sibling name by 67 to 94 percent in four of the five cases. A repeated name that is not an alias is answered entirely from the answer

cache in under a millisecond (shown as 0 ms). Two observations reveal limitations of the current design. First, `www.amazon.com` took longer than the cold lookup of `amazon.com`, because it is a CNAME to a name in another provider's zone, which requires a new walk from the TLD. Second, a repeated lookup of an alias such as `www.github.com` still costs one query, because the CNAME record itself is not stored in the answer cache; only the final RRset of the target is cached. Caching CNAME records under the alias name would remove this last query.

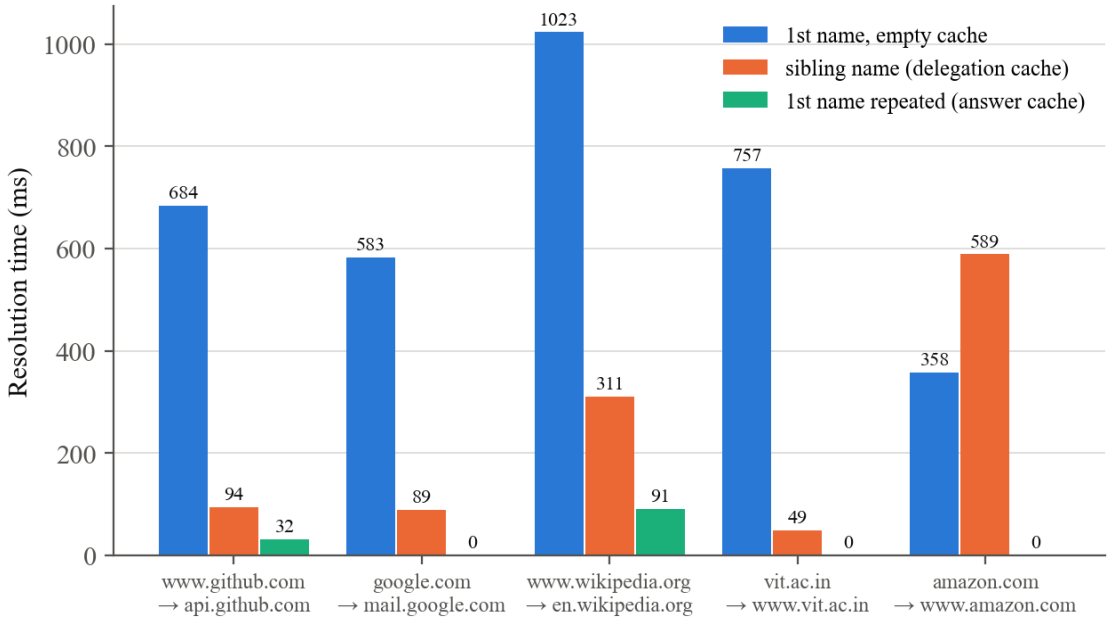


Fig. 7.3 Effect of the Delegation Cache and the Answer Cache

7.5 Record Types and Special Cases

Table 7.4 confirms that each supported record type is resolved and parsed from real servers. The PTR query for `8.8.8.8.in-addr.arpa` walks the reverse tree (`arpa.`, `in-addr.arpa.`, `8.in-addr.arpa.`) and returns `dns.google.`.

Table 7.4 Live Results for Each Supported Record Type

Type	Name queried	Records	Hops	Example record
A	example.com	2	3	104.20.23.154
AAAA	www.wikipedia.org	1	5	2001:df2:e500:ed1a::1
MX	gmail.com	5	3	10 alt1.gmail-smtp-in.l.google.com.
NS	vit.ac.in	4	3	ns-1067.awsdns-05.org.
TXT	google.com	17	3	"v=spf1 include:_spf.google.com ~all"
SOA	google.com	1	3	ns1.google.com. dns-admin.google.com. 100...
CNAME	www.github.com	1	3	github.com.

Type	Name queried	Records	Hops	Example record
PTR	8.8.8.8.in-addr.arpa	1	4	dns.google.

The special cases required by goal G3 were all observed live as well as offline: CNAME chains (`www.github.com`, `www.wikipedia.org`), NXDOMAIN (a non-existent `.com` name), NODATA (AAAA for `github.com`), glueless delegation (`github.io`, `vit.ac.in`) and TCP fallback (TXT for `apple.com`, `microsoft.com` and `amazon.com`, which returned 23, 61 and 66 TXT records respectively after the retry).

7.6 Server-Mode Evaluation

DNSR was started with `dnsr serve -addr 127.0.0.1:8053` and queried twice for each of six questions by dnspython. Table 7.5 and Fig. 7.4 show the client-observed latency. Every response was a valid DNS message with flags QR, RD and RA set, and dnspython decoded every record, which shows that the response encoder, including the re-encoding of names inside RDATA, produces correct messages.

Table 7.5 Server Mode: First and Repeated Query (ms)

Question	Status	First	Repeat	Answer records
example.com A	NOERROR	661.1	0.8	2
www.github.com A	NOERROR	335.0	58.7	2
gmail.com MX	NOERROR	315.0	1.4	5
vit.ac.in NS	NOERROR	2574.9	8.8	4
nosuchname-dnsr-vit.org A	NXDOMAIN	143.2	126.3	0
www.wikipedia.org AAAA	NOERROR	2916.6	337.4	2

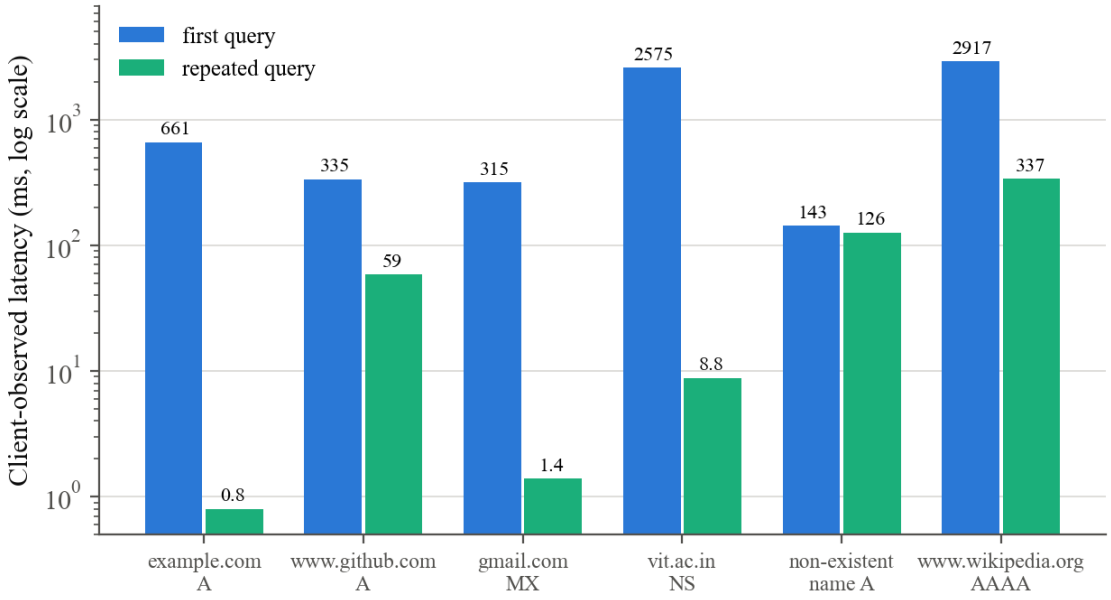


Fig. 7.4 Server Mode: Latency of First and Repeated Queries

Repeated queries for plain names (`example.com` A, `gmail.com` MX, `vit.ac.in` NS) were answered from the cache in 0.8 to 8.8 ms, a reduction of more than 99 percent. Repeated queries for aliases (`www.github.com`, `www.wikipedia.org`) still needed one upstream query, for the reason explained in Section 7.4. The repeated query for a non-existent name took almost as long as the first one (126 ms against 143 ms) because DNSR does not yet cache negative answers (RFC 2308); only the delegation for `org.` was reused. During this experiment it was also found that the default port 5353 cannot be used on Windows, where it is taken by the system's multicast DNS service (the bind fails with an access-permission error), so a different port had to be chosen.

7.7 Security and Robustness Evaluation

7.7.1 Poisoning and redirection

The two adversarial tests passed: DNSR ignored the poisoned glue `ns.dnshost.org A 6.6.6.6` offered by the `com.` server and resolved the real address of the name server through `.org`, and it rejected the referral from `com.` to `net.` with an out-of-bailiwick error. In the live runs, every referral received (from 13 root and TLD operators) passed the bailiwick check, showing that the check does not reject legitimate delegations.

7.7.2 Resistance to blind spoofing

Fig. 7.5 plots Eqn. 4.1 for the two guess spaces of Eqn. 4.2. With the transaction ID alone, an attacker needs about 45,000 forged packets that arrive in time for a 50 percent chance of success. With the random ephemeral port that DNSR obtains by opening a fresh, connected UDP socket for every query, the same probability needs about 7.4×10^8 packets, which is impractical within a two-second query window on ordinary links. The answer filtering and bailiwick checks further reduce what even a successful forgery could inject: only records for the exact name asked, never a delegation for a parent zone.

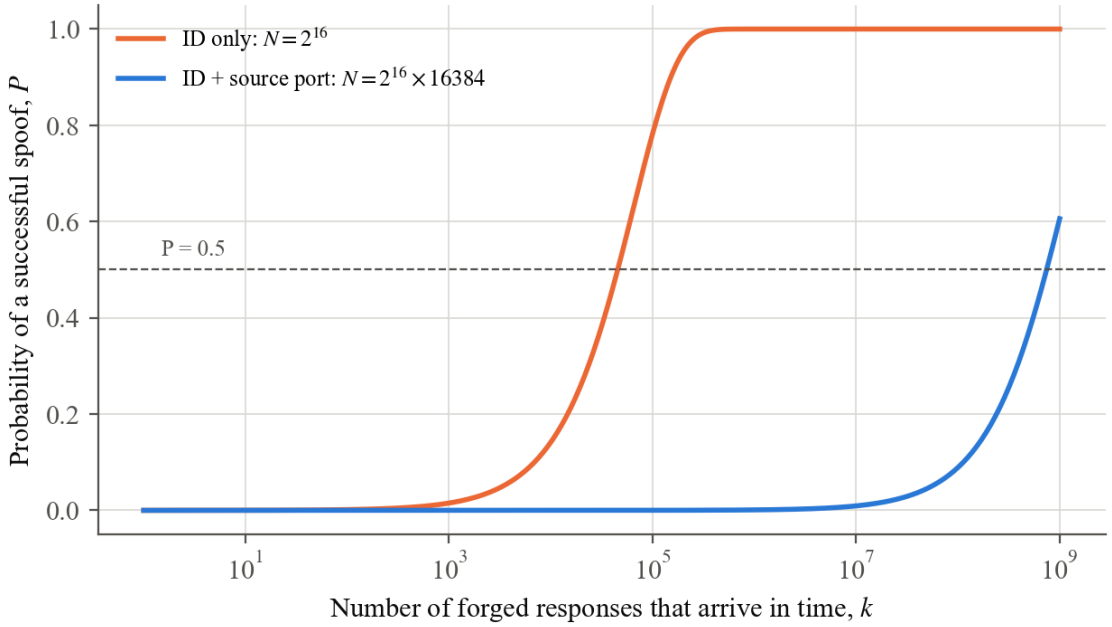


Fig. 7.5 Probability of a Successful Blind Spoof as a Function of Forged Packets

7.7.3 Parser robustness under fuzzing

Two fuzzing experiments were run against `ParseMessage` (Fig. 6.14). In the first, one million packets were generated with a fixed seed: half were random byte strings of up to 600 bytes, and half were copies of a valid query or reply with one to six bytes changed and, in a quarter of the cases, cut short. In the second, Go's native coverage-guided fuzzer [23] ran for 60 seconds on 20 workers. The results are summarised in Table 7.6 and Fig. 7.6.

Table 7.6 Fuzzing Results for ParseMessage

Experiment	Inputs	Outcome
Random and mutated packets (fixed seed)	1,000,000 in 0.42 s	862,312 rejected with an error, 137,688 parsed, 0 panics, 0 hangs
Go coverage-guided fuzzing, 60 s, 20 workers	33,664,027 executions	168 new interesting inputs (corpus 314), no failure found

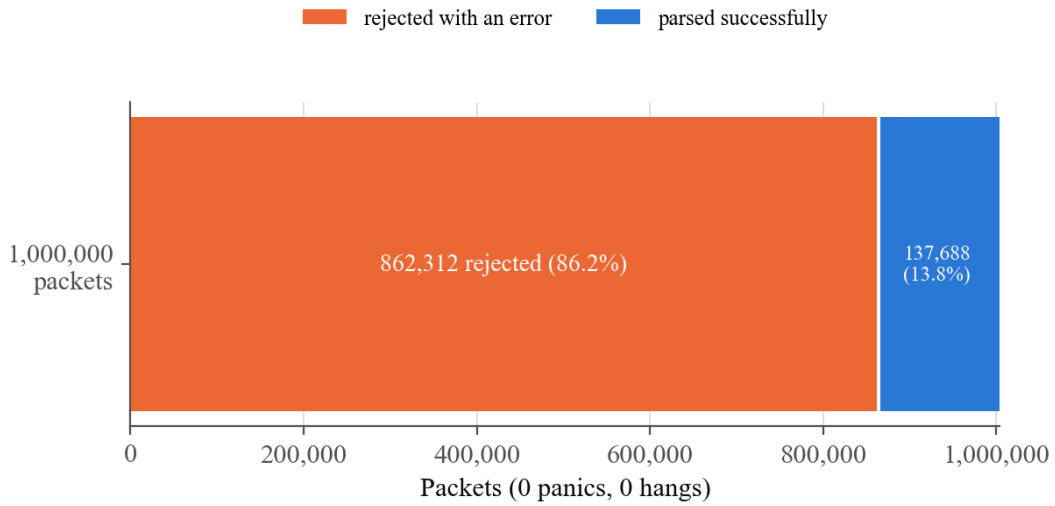


Fig. 7.6 Outcome of One Million Random and Mutated Packets

Not a single input caused a panic, an out-of-range access or a hang. The high proportion of rejected packets is the intended behaviour: inconsistent counts, truncated records and invalid label types are reported as errors rather than being partially accepted. Pointer loops, which were specifically tested by `TestNameCompression`, are cut off by the 64-jump limit.

7.8 Codec Performance

The cost of the codec was measured with Go benchmarks (Table 7.7). Parsing a typical one-answer reply takes about 0.35 microseconds and building a query about 0.13 microseconds on one core, so a single core could parse almost three million replies per second. The codec is therefore never the bottleneck: resolution time is dominated entirely by network round trips, which are six orders of magnitude slower.

Table 7.7 Benchmark Results (Intel Core i7-13650HX)

Benchmark	Iterations	Time per operation	Memory per operation	Allocations
BenchmarkParseMessage	3,339,526	349.5 ns	456 B	16

Benchmark	Iterations	Time per operation	Memory per operation	Allocations
BenchmarkBuildQuery	10,123,207	125.8 ns	560 B	2

7.9 Security Review Findings

In addition to testing, the source code was reviewed for weaknesses that the tests do not cover. Table 7.8 lists the findings in order of importance with a recommended fix for each. None of them breaks the defences described in Section 4.1.7, but several matter if DNSR is exposed as a long-running server.

Table 7.8 Findings of the Security Review

No.	Finding	Risk	Recommended fix
1	No DNSSEC validation	An on-path attacker or compromised server can still forge answers	Validate the DS/DNSKEY/RRSIG chain from the root trust anchor
2	No upper limit on TTL	A record accepted with TTL 2^{32} s (about 136 years) stays cached for the life of the process	Cap TTLs, for example at one day as Unbound does
3	Caches never shrink; delegations never expire	Queries for many random names grow memory without bound in server mode; stale delegations persist	Limit cache size; store the NS TTL with each delegation
4	Every NS name of a glueless referral may be resolved	A hostile zone listing many glueless servers can multiply queries (NXNSAttack [24])	Limit glueless lookups per referral, for example to three
5	Server starts one goroutine per packet without limit or rate control	Resource exhaustion; open resolver if bound to a public address	Worker pool, per-client rate limit and an access-control list
6	TCP reply checked only by ID	Weaker matching than on UDP (TCP is much harder to spoof)	Also compare the question name and type
7	No 0x20 case randomisation	Less entropy against spoofing than possible	Randomise letter case in queries and check it in replies [28]
8	Raw names printed in the trace	Terminal escape sequences in a hostile name could alter the user's terminal	Escape non-printable bytes when printing
9	Default server port 5353 clashes with mDNS on Windows	Server mode fails to start with default settings	Change the default port, for example to 8053

7.10 Achievement of Goals

Table 7.9 Goals and Results

ID	Goal	Result	Status
G1	Correct wire-format codec	All eight record types parsed from live replies; compression test passes	Achieved
G2	Complete iterative resolution	15 of 15 domains give the same response code as 8.8.8.8; 43 of 45 runs succeeded	Achieved
G3	Correct special cases	CNAME, NXDOMAIN, NODATA, glueless and TCP fallback shown offline and live	Achieved
G4	Spoofing and poisoning defences	Poisoned glue ignored; out-of-bailiwick referral rejected; spoof space about 10^9	Achieved
G5	Caching for performance	Sibling names 67 to 94% faster; repeated names under 1 ms (non-alias)	Achieved
G6	Two modes of operation	CLI trace and quiet mode; server answers an independent client	Achieved
G7	Robust parser	About 34.7 million fuzz inputs, no panic or hang	Achieved
G8	Small and dependency-free	969 lines of Go plus 292 lines of tests; standard library only	Achieved

7.11 Cost Analysis

The project has no direct financial cost. All software used is free and open source, and the hardware was the student's existing laptop. Table 7.10 summarises the resources.

Table 7.10 Cost of Resources

Resource	Item	Cost (INR)
Language and tools	Go tool chain, gofmt, go vet, go test	0
Libraries	Go standard library only	0
Validation tools	Python 3, dnspython, matplotlib	0
Infrastructure	Public root, TLD and authoritative DNS servers; Google Public DNS for comparison	0
Hardware	Existing laptop (Intel Core i7-13650HX)	0 (already owned)
Total		0

Running DNSR as a service would cost only the compute of a very small virtual machine, since the resolver uses a few megabytes of memory and negligible processor time; the network round trips, not computation, determine its performance.

7.12 Discussion

The results show that a resolver of about one thousand lines can resolve real-world names correctly, including the awkward cases (glueless delegations across TLDs, CNAMEs into other zones, truncated replies), while enforcing the checks that defeat classic cache-poisoning attacks. The trace output makes these checks visible, which was one of the main aims of the project.

The evaluation also exposed the limits of the simple design. Latency is dominated by the occasional unresponsive server, because servers are tried one at a time with a two-second timeout; production resolvers track the round-trip time of each server and prefer fast ones. The answer cache does not store CNAMEs or negative answers, so aliases and non-existent names are never fully served from cache. The server mode lacks the resource limits needed on an untrusted network. Finally, without DNSSEC the resolver can only defend against off-path attackers. These points are not defects in the checks that were implemented; they define the next steps, which are listed in Chapter 8.

CHAPTER 8

SUMMARY

This project designed, implemented and evaluated DNSR, a security-hardened iterative DNS resolver written from scratch in Go using only the standard library. DNSR encodes and decodes the RFC 1035 wire format by hand, resolves names starting from the IANA root hints, follows referrals with glue, resolves glueless delegations, chases CNAMEs, distinguishes NXDOMAIN from NODATA and retries truncated replies over TCP. It keeps an answer cache with counting-down TTLs and a delegation cache, and can be used either as a command-line tool that prints a step-by-step trace or as a recursive DNS server.

8.1 Findings and Results

- ✓ **Correctness:** DNSR resolved all 15 test domains from the root with the same response code as Google Public DNS; 43 of 45 cold runs succeeded and the addresses agreed except where content-delivery networks return location-dependent answers.
- ✓ **Performance:** a cold-cache resolution took a median of 571 ms and 3.2 queries on average. The delegation cache made sibling names 67 to 94 percent faster, and repeated names were served in under a millisecond.
- ✓ **Security:** the offline tests confirmed that poisoned glue and out-of-bailiwick referrals are rejected. Random IDs and fresh source ports raise the blind-spoofing effort to about 7.4×10^8 forged packets for an even chance.
- ✓ **Robustness:** about 34.7 million random, mutated and coverage-guided inputs caused no panic or hang in the parser.
- ✓ **Efficiency:** parsing a reply costs about 0.35 microseconds, so the resolver's speed is limited only by the network.

8.2 Recommendations

- ✓ Before deploying DNSR as a shared server, add a TTL cap, bounded caches with expiring delegations, a limit on glueless lookups per referral, a worker pool with rate limiting and an access-control list (findings 2 to 5 of Table 7.8).

- ✓ Change the default server port from 5353 to a port that is free on all platforms, such as 8053.
- ✓ Cache CNAME records and negative answers (RFC 2308) to remove the remaining upstream queries for aliases and non-existent names.
- ✓ Use the trace output and the offline hierarchy as teaching material: adding a new malicious server to the test hierarchy is a simple way to demonstrate a specific attack and its defence.

8.3 Future Work

- ✓ **DNSSEC validation:** request DNSSEC records (DO bit), and validate the chain of DS, DNSKEY and RRSIG records from the root trust anchor.
- ✓ **IPv6 transport:** use AAAA glue to reach name servers over IPv6.
- ✓ **QNAME minimisation (RFC 9156):** send each server only the part of the name it needs, improving privacy.
- ✓ **RTT-based server selection and parallel queries:** keep a smoothed round-trip time per server, as Unbound does, to avoid slow or dead servers.
- ✓ **Encrypted client transport:** accept queries over DNS over TLS or DNS over HTTPS in server mode.
- ✓ **TCP in server mode and response truncation:** honour the client's EDNS buffer size and set TC when a response does not fit.

In conclusion, DNSR meets all of its goals. It shows that the essential security logic of a recursive resolver, unpredictable queries, strict reply matching and bailiwick rules, can be expressed clearly in a small program, tested offline against hostile servers and verified against the live Internet. It provides a solid, transparent base for further study of DNS security.

CHAPTER 9

REFERENCES

[1] P. Mockapetris, "Domain Names – Concepts and Facilities," RFC 1034, IETF, Nov. 1987.

[2] P. Mockapetris, "Domain Names – Implementation and Specification," RFC 1035, IETF, Nov. 1987.

[3] P. Mockapetris and K. J. Dunlap, "Development of the Domain Name System," in Proc. ACM SIGCOMM '88, Stanford, CA, USA, 1988, pp. 123–133.

[4] R. Elz and R. Bush, "Clarifications to the DNS Specification," RFC 2181, IETF, Jul. 1997.

[5] M. Andrews, "Negative Caching of DNS Queries (DNS NCACHE)," RFC 2308, IETF, Mar. 1998.

[6] J. Damas, M. Graff, and P. Vixie, "Extension Mechanisms for DNS (EDNS(0))," RFC 6891, IETF, Apr. 2013.

[7] J. Dickinson, S. Dickinson, R. Bellis, A. Mankin, and D. Wessels, "DNS Transport over TCP – Implementation Requirements," RFC 7766, IETF, Mar. 2016.

[8] A. Hubert and R. van Mook, "Measures for Making DNS More Resilient against Forged Answers," RFC 5452, IETF, Jan. 2009.

[9] D. Kaminsky, "Black Ops 2008: It's the End of the Cache as We Know It," presented at Black Hat USA, Las Vegas, NV, USA, Aug. 2008.

[10] S. Son and V. Shmatikov, "The Hitchhiker's Guide to DNS Cache Poisoning," in Proc. SecureComm 2010, LNICST, vol. 50, Springer, 2010, pp. 466–483.

[11] K. Man, Z. Qian, Z. Wang, X. Zheng, Y. Huang, and H. Duan, "DNS Cache Poisoning Attack Reloaded: Revolutions with Side Channels," in Proc. ACM CCS '20, 2020, pp. 1337–1350.

[12] J. Jung, E. Sit, H. Balakrishnan, and R. Morris, "DNS Performance and the Effectiveness of Caching," IEEE/ACM Trans. Networking, vol. 10, no. 5, pp. 589–603, Oct. 2002.

[13] R. Arends, R. Austein, M. Larson, D. Massey, and S. Rose, "DNS Security Introduction and Requirements," RFC 4033, IETF, Mar. 2005.

[14] S. Bortzmeyer, R. Dolmans, and P. Hoffman, "DNS Query Name Minimisation to Improve Privacy," RFC 9156, IETF, Nov. 2021.

[15] Z. Hu, L. Zhu, J. Heidemann, A. Mankin, D. Wessels, and P. Hoffman, "Specification for DNS over Transport Layer Security (TLS)," RFC 7858, IETF, May 2016.

[16] P. Hoffman and P. McManus, "DNS Queries over HTTPS (DoH)," RFC 8484, IETF, Oct. 2018.

[17] NLnet Labs, "Unbound – A Validating, Recursive, Caching DNS Resolver." [Online]. Available: https://nlnetlabs.nl/projects/unbound/

[18] Internet Systems Consortium, "BIND 9." [Online]. Available: https://www.isc.org/bind/

[19] M. Gieben, "dns – A DNS Library in Go." [Online]. Available: https://github.com/miekg/dns

[20] DNS Flag Day 2020. [Online]. Available: https://dnsflagday.net/2020/

[21] IANA, "Root Files (named.root)." [Online]. Available: https://www.iana.org/domains/root/files

[22] A. A. A. Donovan and B. W. Kernighan, The Go Programming Language. Boston, MA, USA: Addison-Wesley, 2015.

[23] The Go Authors, "Go Fuzzing." [Online]. Available: https://go.dev/doc/security/fuzz/

[24] Y. Afek, A. Bremler-Barr, and L. Shafir, "NXNSAttack: Recursive DNS Inefficiencies and Vulnerabilities," in Proc. 29th USENIX Security Symposium, 2020, pp. 631–648.

[25] D. Atkins and R. Austein, "Threat Analysis of the Domain Name System (DNS)," RFC 3833, IETF, Aug. 2004.

[26] P. Hoffman, A. Sullivan, and K. Fujiwara, "DNS Terminology," RFC 8499, IETF, Jan. 2019.

[27] S. Bortzmeyer and S. Huque, "NXDOMAIN: There Really Is Nothing Underneath," RFC 8020, IETF, Nov. 2016.

[28] D. Dagon, M. Antonakakis, P. Vixie, T. Jinmei, and W. Lee, "Increased DNS Forgery Resistance Through 0x20-Bit Encoding," in Proc. ACM CCS '08, 2008, pp. 211–222.

[29] D. Eastlake 3rd and M. Andrews, "Domain Name System (DNS) Cookies," RFC 7873, IETF, May 2016.

[30] S. Thomson, C. Huitema, V. Ksinant, and M. Souissi, "DNS Extensions to Support IP Version 6," RFC 3596, IETF, Oct. 2003.

[31] B. Halley et al., "dnspython – A DNS Toolkit for Python." [Online]. Available: https://www.dnspython.org/

[32] Microsoft, "The default dynamic port range for TCP/IP has changed since Windows Vista and in Windows Server 2008," Microsoft Learn, KB929851. [Online]. Available: https://learn.microsoft.com/en-us/troubleshoot/windows-server/networking/default-dynamic-port-range-tcpip-chang

APPENDIX A

COMPLETE SOURCE CODE

This appendix lists the complete source code of DNSR as committed to the project repository (module github.com/swayam5342/dnsr), followed by the fuzzing and benchmark harness used in Chapter 7.

A.1 wire.go

```
package main

// wire.go — RFC 1035 message format, hand-rolled. No net/dns helpers.
//
//   +---------------------+
//   |        Header       |  12 bytes: ID, flags, 4 counts
//   +---------------------+
//   |       Question      |  QNAME QTYPE QCLASS
//   +---------------------+
//   |        Answer       |  RRs
//   +---------------------+
//   |      Authority      |  RRs (NS records on a referral)
//   +---------------------+
//   |      Additional     |  RRs (glue A/AAAA for those NS)
//   +---------------------+

import (
    "encoding/binary"
    "errors"
    "fmt"
    "net"
    "strings"
)

type RRType uint16

const (
    TypeA     RRType = 1
    TypeNS    RRType = 2
    TypeCNAME RRType = 5
    TypeSOA   RRType = 6
    TypePTR   RRType = 12
    TypeMX    RRType = 15
    TypeTXT   RRType = 16
    TypeAAAA  RRType = 28
    TypeOPT   RRType = 41
)

var typeNames = map[RRType]string{
    TypeA: "A", TypeNS: "NS", TypeCNAME: "CNAME", TypeSOA: "SOA", TypePTR: "PTR",
    TypeMX: "MX", TypeTXT: "TXT", TypeAAAA: "AAAA", TypeOPT: "OPT",
}

func (t RRType) String() string {
    if s, ok := typeNames[t]; ok {
        return s
    }
    return fmt.Sprintf("TYPE%d", uint16(t))
}

func ParseType(s string) (RRType, error) {
    s = strings.ToUpper(s)
    for t, n := range typeNames {
        if n == s {
            return t, nil
        }
```

```
    }
    return 0, fmt.Errorf("unknown record type %q", s)
}

const (
    RcodeNoError  = 0
    RcodeFormErr  = 1
    RcodeServFail = 2
    RcodeNXDomain = 3
    RcodeRefused  = 5
)

var rcodeNames = []string{"NOERROR", "FORMERR", "SERVFAIL", "NXDOMAIN", "NOTIMP", "REFUSED"}

func rcodeString(r int) string {
    if r < len(rcodeNames) {
        return rcodeNames[r]
    }
    return fmt.Sprintf("RCODE%d", r)
}

type Header struct {
    ID      uint16
    QR      bool
    Opcode  uint8
    AA      bool
    TC      bool
    RD      bool
    RA      bool
    Rcode   int
    QDCount uint16
    ANCount uint16
    NSCount uint16
    ARCount uint16
}

type Question struct {
    Name  string
    Type  RRType
    Class uint16
}

type RR struct {
    Name  string
    Type  RRType
    Class uint16
    TTL   uint32
    Data  string
    Raw   []byte
}

func (r RR) String() string {
    return fmt.Sprintf("%-28s %6d IN %-5s %s", r.Name, r.TTL, r.Type, r.Data)
}

type Message struct {
    Header     Header
    Questions  []Question
    Answers    []RR
    Authority  []RR
    Additional []RR
}

func encodeName(buf []byte, name string) ([]byte, error) {
    name = strings.TrimSuffix(name, ".")
    if name != "" {
        for _, label := range strings.Split(name, ".") {
            if len(label) == 0 || len(label) > 63 {
                return nil, fmt.Errorf("bad label %q in %q", label, name)
            }
            buf = append(buf, byte(len(label)))
            buf = append(buf, label...)
        }
    }
```

```
    return append(buf, 0), nil
}

// BuildQuery makes a non-recursive query (RD=0) with an EDNS0 OPT record
// advertising a 1232-byte UDP buffer (the DNS Flag Day 2020 value).
func BuildQuery(id uint16, name string, qtype RRType) ([]byte, error) {
    b := make([]byte, 12, 512)
    binary.BigEndian.PutUint16(b[0:], id)
    binary.BigEndian.PutUint16(b[2:], 0)
    binary.BigEndian.PutUint16(b[4:], 1)
    binary.BigEndian.PutUint16(b[10:], 1)

    var err error
    if b, err = encodeName(b, name); err != nil {
        return nil, err
    }
    b = binary.BigEndian.AppendUint16(b, uint16(qtype))
    b = binary.BigEndian.AppendUint16(b, 1)

    b = append(b, 0)
    b = binary.BigEndian.AppendUint16(b, uint16(TypeOPT))
    b = binary.BigEndian.AppendUint16(b, 1232)
    b = binary.BigEndian.AppendUint32(b, 0)
    b = binary.BigEndian.AppendUint16(b, 0)
    return b, nil
}

var errShort = errors.New("message truncated")

// readName decodes a possibly-compressed name starting at off.
// Returns the name and the offset just past it in the *original* stream.
func readName(msg []byte, off int) (string, int, error) {
    var labels []string
    end := -1
    for jumps := 0; ; {
        if off >= len(msg) {
            return "", 0, errShort
        }
        l := int(msg[off])
        switch {
        case l == 0:
            if end < 0 {
                end = off + 1
            }
            return strings.Join(labels, ".") + ".", end, nil
        case l&0xC0 == 0xC0:
            if off+1 >= len(msg) {
                return "", 0, errShort
            }
            if end < 0 {
                end = off + 2
            }
            jumps++
            if jumps > 64 {
                return "", 0, errors.New("compression pointer loop")
            }
            off = int(binary.BigEndian.Uint16(msg[off:]) & 0x3FFF)
        case l&0xC0 != 0:
            return "", 0, fmt.Errorf("unsupported label type 0x%x", l)
        default:
            off++
            if off+l > len(msg) {
                return "", 0, errShort
            }
            labels = append(labels, string(msg[off:off+l]))
            off += l
        }
    }
}

func readRR(msg []byte, off int) (RR, int, error) {
    var rr RR
    var err error
    if rr.Name, off, err = readName(msg, off); err != nil {
        return rr, 0, err
    }
```

```
    if off+10 > len(msg) {
        return rr, 0, errShort
    }
    rr.Type = RRType(binary.BigEndian.Uint16(msg[off:]))
    rr.Class = binary.BigEndian.Uint16(msg[off+2:])
    rr.TTL = binary.BigEndian.Uint32(msg[off+4:])
    rdlen := int(binary.BigEndian.Uint16(msg[off+8:]))
    off += 10
    if off+rdlen > len(msg) {
        return rr, 0, errShort
    }
    rd := msg[off : off+rdlen]
    rr.Raw = rd

    switch rr.Type {
    case TypeA:
        if rdlen == 4 {
            rr.Data = net.IP(rd).String()
        }
    case TypeAAAA:
        if rdlen == 16 {
            rr.Data = net.IP(rd).String()
        }
    case TypeNS, TypeCNAME, TypePTR:
        rr.Data, _, err = readName(msg, off)
    case TypeMX:
        if rdlen >= 3 {
            var host string
            host, _, err = readName(msg, off+2)
            rr.Data = fmt.Sprintf("%d %s", binary.BigEndian.Uint16(rd), host)
        }
    case TypeSOA:
        var mname, rname string
        var p int
        if mname, p, err = readName(msg, off); err == nil {
            if rname, p, err = readName(msg, p); err == nil && p+20 <= off+rdlen {
                rr.Data = fmt.Sprintf("%s %s %d %d %d %d %d", mname, rname,
                    binary.BigEndian.Uint32(msg[p:]), binary.BigEndian.Uint32(msg[p+4:]),
                    binary.BigEndian.Uint32(msg[p+8:]), binary.BigEndian.Uint32(msg[p+12:]),
                    binary.BigEndian.Uint32(msg[p+16:]))
            }
        }
    case TypeTXT:
        var parts []string
        for i := 0; i < len(rd); {
            n := int(rd[i])
            i++
            if i+n > len(rd) {
                break
            }
            parts = append(parts, fmt.Sprintf("%q", rd[i:i+n]))
            i += n
        }
        rr.Data = strings.Join(parts, " ")
    default:
        rr.Data = fmt.Sprintf("\\# %d %x", rdlen, rd)
    }
    if err != nil {
        return rr, 0, err
    }
    return rr, off + rdlen, nil
}

func ParseMessage(msg []byte) (*Message, error) {
    if len(msg) < 12 {
        return nil, errShort
    }
    m := &Message{}
    f := binary.BigEndian.Uint16(msg[2:])
    m.Header = Header{
        ID:      binary.BigEndian.Uint16(msg[0:]),
        QR:      f&0x8000 != 0,
        Opcode:  uint8(f>>11) & 0xF,
        AA:      f&0x0400 != 0,
        TC:      f&0x0200 != 0,
        RD:      f&0x0100 != 0,
        RA:      f&0x0080 != 0,
        Rcode:   int(f & 0xF),
```

```
        QDCount: binary.BigEndian.Uint16(msg[4:]),
        ANCount: binary.BigEndian.Uint16(msg[6:]),
        NSCount: binary.BigEndian.Uint16(msg[8:]),
        ARCount: binary.BigEndian.Uint16(msg[10:]),
    }
    off := 12
    for i := 0; i < int(m.Header.QDCount); i++ {
        name, p, err := readName(msg, off)
        if err != nil {
            return nil, err
        }
        if p+4 > len(msg) {
            return nil, errShort
        }
        m.Questions = append(m.Questions, Question{name,
            RRType(binary.BigEndian.Uint16(msg[p:])), binary.BigEndian.Uint16(msg[p+2:])})
        off = p + 4
    }
    sections := []struct {
        n   uint16
        dst *[]RR
    }{{m.Header.ANCount, &m.Answers}, {m.Header.NSCount, &m.Authority}, {m.Header.ARCount,
&m.Additional}}
    for _, s := range sections {
        for i := 0; i < int(s.n); i++ {
            rr, p, err := readRR(msg, off)
            if err != nil {
                return nil, err
            }
            if rr.Type != TypeOPT {
                *s.dst = append(*s.dst, rr)
            }
            off = p
        }
    }
    return m, nil
}
```

A.2 resolver.go

```
package main


// resolver.go — iterative resolution, the way a real recursor does it:
//
//   root (.)  ──referral──▶  TLD (com.)  ──referral──▶  example.com. NS  ──▶ answer
//
// We start from hard-coded root hints, send non-recursive queries, and follow
// referrals (NS in Authority + glue in Additional) until someone answers
// authoritatively. CNAMEs are chased, glueless delegations are resolved
// with a nested lookup, and truncated UDP responses fall back to TCP.


import (
    "encoding/binary"
    "errors"
    "fmt"
    "io"
    "math/rand"
    "net"
    "strings"
    "sync"
    "time"
)


var rootHints = []nameserver{
    {"a.root-servers.net.", "198.41.0.4"},
    {"b.root-servers.net.", "170.247.170.2"},
    {"c.root-servers.net.", "192.33.4.12"},
    {"d.root-servers.net.", "199.7.91.13"},
    {"e.root-servers.net.", "192.203.230.10"},
    {"f.root-servers.net.", "192.5.5.241"},
    {"g.root-servers.net.", "192.112.36.4"},
    {"h.root-servers.net.", "198.97.190.53"},
    {"i.root-servers.net.", "192.36.148.17"},
```

```
    {"j.root-servers.net.", "192.58.128.30"},
    {"k.root-servers.net.", "193.0.14.129"},
    {"l.root-servers.net.", "199.7.83.42"},
    {"m.root-servers.net.", "202.12.27.33"},
}

type nameserver struct {
    Name string
    IP   string
}

type Result struct {
    Rcode     int
    Answers   []RR
    Authority []RR
    Hops      int
    Elapsed   time.Duration
}

type cacheEntry struct {
    rrs     []RR
    expires time.Time
}

type Resolver struct {
    Timeout time.Duration
    MaxHops int
    Trace   io.Writer
    Port    string
    Roots   []nameserver
    color   bool
    mu      sync.Mutex
    cache   map[string]cacheEntry
    zoneNS  map[string][]nameserver
}

func NewResolver() *Resolver {
    return &Resolver{
        Timeout: 2 * time.Second,
        MaxHops: 40,
        Port:    "53",
        Roots:   rootHints,
        cache:   map[string]cacheEntry{},
        zoneNS:  map[string][]nameserver{},
    }
}

var (
    ErrLoop     = errors.New("resolution loop / too deep")
    ErrNoServer = errors.New("all nameservers failed")
)

func fqdn(s string) string {
    s = strings.ToLower(strings.TrimSpace(s))
    if !strings.HasSuffix(s, ".") {
        s += "."
    }
    return s
}

// isSubdomain reports whether child is at or below parent ("a.b.c." under "b.c.").
func isSubdomain(child, parent string) bool {
    if parent == "." {
        return true
    }
    return child == parent || strings.HasSuffix(child, "."+parent)
}

func (r *Resolver) Resolve(name string, qtype RRType) (*Result, error) {
    start := time.Now()
    res, err := r.resolve(fqdn(name), qtype, 0, 0)
    if res != nil {
        res.Elapsed = time.Since(start)
    }
```

```
    return res, err
}

const (
    cDim, cCyan, cGreen, cYellow, cRed, cBold, cReset = "\033[2m", "\033[36m", "\033[32m",
"\033[33m", "\033[31m", "\033[1m", "\033[0m"
)

func (r *Resolver) c(code, s string) string {
    if !r.color {
        return s
    }
    return code + s + cReset
}

func (r *Resolver) tracef(depth int, format string, a ...any) {
    if r.Trace == nil {
        return
    }
    fmt.Fprintf(r.Trace, "%s%s\n", strings.Repeat("  │ ", depth), fmt.Sprintf(format, a...))
}

func (r *Resolver) cacheGet(name string, t RRType) ([]RR, bool) {
    r.mu.Lock()
    defer r.mu.Unlock()
    e, ok := r.cache[name+"|"+t.String()]
    if !ok || time.Now().After(e.expires) {
        return nil, false
    }
    left := uint32(time.Until(e.expires).Seconds())
    out := make([]RR, len(e.rrs))
    for i, rr := range e.rrs {
        rr.TTL = left
        out[i] = rr
    }
    return out, true
}

func (r *Resolver) cachePut(name string, t RRType, rrs []RR) {
    if len(rrs) == 0 {
        return
    }
    min := rrs[0].TTL
    for _, rr := range rrs {
        if rr.TTL < min {
            min = rr.TTL
        }
    }
    r.mu.Lock()
    r.cache[name+"|"+t.String()] = cacheEntry{rrs, time.Now().Add(time.Duration(min) *
time.Second)}
    r.mu.Unlock()
}

func (r *Resolver) closestKnownZone(name string) (string, []nameserver) {
    r.mu.Lock()
    defer r.mu.Unlock()
    for z := name; ; {
        if ns, ok := r.zoneNS[z]; ok && len(ns) > 0 {
            return z, ns
        }
        if z == "." {
            return ".", r.Roots
        }
        if i := strings.IndexByte(z, '.'); i >= 0 && i+1 < len(z) {
            z = z[i+1:]
        } else {
            z = "."
        }
    }
}

func (r *Resolver) resolve(name string, qtype RRType, depth, hops int) (*Result, error) {
    if depth > 8 {
        return nil, ErrLoop
```

```
    }
    if rrs, ok := r.cacheGet(name, qtype); ok {
        r.tracef(depth, "%s %s %s", r.c(cGreen, "✓ cache hit"), name, qtype)
        return &Result{Rcode: RcodeNoError, Answers: rrs, Hops: hops}, nil
    }

    zone, servers := r.closestKnownZone(name)
    if zone != "." {
        r.tracef(depth, "%s starting at cached zone %s", r.c(cDim, "↷"), r.c(cBold, zone))
    }

    for ; hops < r.MaxHops; hops++ {
        msg, ns, rtt, err := r.queryAny(servers, name, qtype, depth)
        if err != nil {
            return nil, fmt.Errorf("querying %s servers for %s: %w", zone, name, err)
        }
        if msg.Header.Rcode == RcodeNXDomain {
            r.tracef(depth, "%s %s says %s (%s)", r.c(cRed, "✗"), ns.Name, r.c(cRed, "NXDOMAIN"),
rtt)
            return &Result{Rcode: RcodeNXDomain, Authority: msg.Authority, Hops: hops + 1}, nil
        }
        if msg.Header.Rcode != RcodeNoError {
            r.tracef(depth, "%s %s returned %s, trying next server", r.c(cYellow, "!"), ns.Name,
rcodeString(msg.Header.Rcode))
            servers = without(servers, ns)
            if len(servers) == 0 {
                return &Result{Rcode: msg.Header.Rcode, Hops: hops + 1}, nil
            }
            continue
        }

        if len(msg.Answers) > 0 {
            var direct, cnames []RR
            for _, rr := range msg.Answers {
                if !strings.EqualFold(rr.Name, name) {
                    continue
                }
                if rr.Type == qtype {
                    direct = append(direct, rr)
                } else if rr.Type == TypeCNAME {
                    cnames = append(cnames, rr)
                }
            }
            if len(direct) > 0 {
                r.tracef(depth, "%s %s answered %s (%s)", r.c(cGreen, "✓"), ns.Name,
                    r.c(cGreen, fmt.Sprintf("%d %s record(s)", len(direct), qtype)), rtt)
                r.cachePut(name, qtype, direct)
                return &Result{Rcode: RcodeNoError, Answers: direct, Hops: hops + 1}, nil
            }
            if len(cnames) > 0 {
                target := fqdn(cnames[0].Data)
                r.tracef(depth, "%s %s is an alias → %s, restarting from the top", r.c(cCyan, "↪
CNAME"), name, r.c(cBold, target))
                sub, err := r.resolve(target, qtype, depth+1, hops+1)
                if err != nil {
                    return nil, err
                }
                sub.Answers = append([]RR{cnames[0]}, sub.Answers...)
                return sub, nil
            }
        }
        if msg.Header.AA {
            r.tracef(depth, "%s %s: name exists but has no %s records (NODATA)", r.c(cYellow,
"∅"), ns.Name, qtype)
            return &Result{Rcode: RcodeNoError, Authority: msg.Authority, Hops: hops + 1}, nil
        }

        var nsRRs []RR
        var child string
        for _, rr := range msg.Authority {
            if rr.Type == TypeNS {
                nsRRs = append(nsRRs, rr)
                child = strings.ToLower(rr.Name)
            }
        }
        if len(nsRRs) == 0 {
```

```
            return nil, fmt.Errorf("lame response from %s: no answer, no referral", ns.Name)
        }

        if !isSubdomain(name, child) || !isSubdomain(child, zone) || child == zone {
            return nil, fmt.Errorf("out-of-bailiwick referral from %s: %s is not under %s",
ns.Name, child, zone)
        }

        next := r.glueFor(nsRRs, msg.Additional, zone)
        names := make([]string, 0, len(nsRRs))
        for _, rr := range nsRRs {
            names = append(names, strings.TrimSuffix(rr.Data, "."))
        }
        r.tracef(depth, "%s %s → referral to %s %s (%s)", r.c(cCyan, "↓"), ns.Name, r.c(cBold,
child),
            r.c(cDim, fmt.Sprintf("[%s]", summarize(names, 3))), rtt)

        if len(next) == 0 {
            r.tracef(depth, "%s no glue for %s — resolving a nameserver's address first",
r.c(cYellow, "…"), child)
            for _, nsrr := range shuffled(nsRRs) {
                sub, err := r.resolve(fqdn(nsrr.Data), TypeA, depth+1, 0)
                if err != nil || sub.Rcode != RcodeNoError {
                    continue
                }
                for _, a := range sub.Answers {
                    if a.Type == TypeA {
                        next = append(next, nameserver{fqdn(nsrr.Data), a.Data})
                    }
                }
                if len(next) > 0 {
                    break
                }
            }
            if len(next) == 0 {
                return nil, fmt.Errorf("could not find an address for any %s nameserver", child)
            }
        }

        r.mu.Lock()
        r.zoneNS[child] = next
        r.mu.Unlock()
        zone, servers = child, next
    }
    return nil, ErrLoop
}

// glueFor pairs NS names with A records from the Additional section.
// Glue is only trusted if it lies inside the zone of the server that sent it
// (askedZone) — a .com server may vouch for ns1.example.com, not for ns.evil.org.
func (r *Resolver) glueFor(nsRRs, additional []RR, askedZone string) []nameserver {
    want := map[string]bool{}
    for _, rr := range nsRRs {
        want[fqdn(rr.Data)] = true
    }
    var out []nameserver
    for _, rr := range additional {
        n := fqdn(rr.Name)
        if rr.Type == TypeA && want[n] && isSubdomain(n, askedZone) {
            out = append(out, nameserver{n, rr.Data})
        }
    }
    return out
}

// queryAny tries servers in random order until one gives a usable reply.
func (r *Resolver) queryAny(servers []nameserver, name string, qtype RRType, depth int)
(*Message, nameserver, time.Duration, error) {
    var lastErr error = ErrNoServer
    for i, ns := range shuffled(servers) {
        if i >= 4 {
            break
        }
        r.tracef(depth, "%s asking %s %s for %s %s", r.c(cDim, "→"), ns.Name, r.c(cDim,
"("+ns.IP+")"), name, qtype)
        t0 := time.Now()
```

```
        msg, err := r.exchange(ns.IP, name, qtype)
        rtt := time.Since(t0).Round(time.Millisecond)
        if err != nil {
            r.tracef(depth, "%s %s: %v", r.c(cYellow, "!"), ns.Name, err)
            lastErr = err
            continue
        }
        return msg, ns, rtt, nil
    }
    return nil, nameserver{}, 0, lastErr
}

// exchange sends one query over UDP, retrying over TCP if the answer is truncated.
func (r *Resolver) exchange(ip, name string, qtype RRType) (*Message, error) {
    id := uint16(rand.Uint32())
    q, err := BuildQuery(id, name, qtype)
    if err != nil {
        return nil, err
    }
    addr := net.JoinHostPort(ip, r.Port)

    conn, err := net.DialTimeout("udp", addr, r.Timeout)
    if err != nil {
        return nil, err
    }
    defer conn.Close()
    conn.SetDeadline(time.Now().Add(r.Timeout))
    if _, err := conn.Write(q); err != nil {
        return nil, err
    }
    buf := make([]byte, 4096)
    for {
        n, err := conn.Read(buf)
        if err != nil {
            return nil, err
        }
        m, err := ParseMessage(buf[:n])
        if err != nil {
            return nil, err
        }
        if m.Header.ID != id || !m.Header.QR || len(m.Questions) != 1 ||
            !strings.EqualFold(m.Questions[0].Name, name) || m.Questions[0].Type != qtype {
            continue
        }
        if m.Header.TC {
            r.tracef(0, "%s response truncated, retrying over TCP", r.c(cYellow, "⇄"))
            return exchangeTCP(addr, q, id, r.Timeout)
        }
        return m, nil
    }
}

// DNS over TCP: same message, prefixed with a 2-byte length (RFC 1035 §4.2.2).
func exchangeTCP(addr string, q []byte, id uint16, timeout time.Duration) (*Message, error) {
    conn, err := net.DialTimeout("tcp", addr, timeout)
    if err != nil {
        return nil, err
    }
    defer conn.Close()
    conn.SetDeadline(time.Now().Add(timeout))
    frame := binary.BigEndian.AppendUint16(nil, uint16(len(q)))
    if _, err := conn.Write(append(frame, q...)); err != nil {
        return nil, err
    }
    var lb [2]byte
    if _, err := io.ReadFull(conn, lb[:]); err != nil {
        return nil, err
    }
    buf := make([]byte, binary.BigEndian.Uint16(lb[:]))
    if _, err := io.ReadFull(conn, buf); err != nil {
        return nil, err
    }
    m, err := ParseMessage(buf)
    if err != nil {
        return nil, err
    }
    if m.Header.ID != id {
```

```
        return nil, errors.New("TCP response ID mismatch")
    }
    return m, nil
}

func shuffled[T any](in []T) []T {
    out := append([]T(nil), in...)
    rand.Shuffle(len(out), func(i, j int) { out[i], out[j] = out[j], out[i] })
    return out
}

func without(in []nameserver, x nameserver) []nameserver {
    var out []nameserver
    for _, s := range in {
        if s != x {
            out = append(out, s)
        }
    }
    return out
}

func summarize(s []string, n int) string {
    if len(s) <= n {
        return strings.Join(s, ", ")
    }
    return fmt.Sprintf("%s, +%d more", strings.Join(s[:n], ", "), len(s)-n)
}
```

A.3 server.go

```
package main

import (
    "encoding/binary"
    "fmt"
    "log"
    "net"
    "strings"
)

func encodeRData(rr RR) ([]byte, error) {
    switch rr.Type {
    case TypeNS, TypeCNAME, TypePTR:
        return encodeName(nil, rr.Data)
    case TypeMX:
        var pref uint16
        var host string
        if _, err := fmt.Sscanf(rr.Data, "%d %s", &pref, &host); err != nil {
            return nil, err
        }
        return encodeName(binary.BigEndian.AppendUint16(nil, pref), host)
    case TypeSOA:
        f := strings.Fields(rr.Data)
        if len(f) != 7 {
            return nil, fmt.Errorf("bad SOA %q", rr.Data)
        }
        b, err := encodeName(nil, f[0])
        if err != nil {
            return nil, err
        }
        if b, err = encodeName(b, f[1]); err != nil {
            return nil, err
        }
        for _, s := range f[2:] {
            var v uint32
            fmt.Sscan(s, &v)
            b = binary.BigEndian.AppendUint32(b, v)
        }
        return b, nil
    default:
        return rr.Raw, nil
    }
```

```
}

func appendRR(b []byte, rr RR) ([]byte, error) {
    b, err := encodeName(b, rr.Name)
    if err != nil {
        return nil, err
    }
    rd, err := encodeRData(rr)
    if err != nil {
        return nil, err
    }
    b = binary.BigEndian.AppendUint16(b, uint16(rr.Type))
    b = binary.BigEndian.AppendUint16(b, 1)
    b = binary.BigEndian.AppendUint32(b, rr.TTL)
    b = binary.BigEndian.AppendUint16(b, uint16(len(rd)))
    return append(b, rd...), nil
}

func buildResponse(id uint16, q Question, rd bool, rcode int, ans, auth, add []RR) []byte {
    flags := uint16(0x8000 | 0x0080)
    if rd {
        flags |= 0x0100
    }
    flags |= uint16(rcode & 0xF)

    b := make([]byte, 12)
    binary.BigEndian.PutUint16(b[0:], id)
    binary.BigEndian.PutUint16(b[2:], flags)
    binary.BigEndian.PutUint16(b[4:], 1)
    b, _ = encodeName(b, q.Name)
    b = binary.BigEndian.AppendUint16(b, uint16(q.Type))
    b = binary.BigEndian.AppendUint16(b, q.Class)

    var na, nn uint16
    for _, rr := range ans {
        if nb, err := appendRR(b, rr); err == nil {
            b, na = nb, na+1
        }
    }
    for _, rr := range auth {
        if nb, err := appendRR(b, rr); err == nil {
            b, nn = nb, nn+1
        }
    }
    var nr uint16
    for _, rr := range add {
        if nb, err := appendRR(b, rr); err == nil {
            b, nr = nb, nr+1
        }
    }
    binary.BigEndian.PutUint16(b[6:], na)
    binary.BigEndian.PutUint16(b[8:], nn)
    binary.BigEndian.PutUint16(b[10:], nr)
    return b
}

func serve(addr string, r *Resolver) error {
    pc, err := net.ListenPacket("udp", addr)
    if err != nil {
        return err
    }
    log.Printf("dnsr listening on udp://%s — try: dig @%s -p %s example.com",
        pc.LocalAddr(), strings.Split(pc.LocalAddr().String(), ":")[0],
strings.Split(pc.LocalAddr().String(), ":")[1])

    buf := make([]byte, 1500)
    for {
        n, client, err := pc.ReadFrom(buf)
        if err != nil {
            return err
        }
        pkt := append([]byte(nil), buf[:n]...)
        go func() {
            m, err := ParseMessage(pkt)
            if err != nil || len(m.Questions) != 1 || m.Header.QR {
                return
```

```
            }
            q := m.Questions[0]
            res, err := r.Resolve(q.Name, q.Type)
            var out []byte
            if err != nil {
                log.Printf("%-30s %-5s SERVFAIL (%v)", q.Name, q.Type, err)
                out = buildResponse(m.Header.ID, q, m.Header.RD, RcodeServFail, nil, nil, nil)
            } else {
                log.Printf("%-30s %-5s %-8s %d answer(s), %d hops, %s", q.Name, q.Type,
                    rcodeString(res.Rcode), len(res.Answers), res.Hops, res.Elapsed.Round(1e6))
                out = buildResponse(m.Header.ID, q, m.Header.RD, res.Rcode, res.Answers,
res.Authority, nil)
            }
            pc.WriteTo(out, client)
        }()
    }
}
```

A.4 main.go

```
package main

import (
    "flag"
    "fmt"
    "os"
)

const usage = `dnsr — an iterative DNS resolver built from scratch

usage:
  dnsr [-t TYPE] [-q] <name> [name...]    resolve from the root servers down
  dnsr serve [-addr 127.0.0.1:5353]       run as a recursive DNS server

flags:
  -t TYPE   record type: A, AAAA, NS, MX, TXT, CNAME, SOA, PTR (default A)
  -q        quiet: print only the final answer, no trace
  -nocolor  disable ANSI colours

examples:
  dnsr github.com
  dnsr -t MX gmail.com
  dnsr -t AAAA www.wikipedia.org swayam.li
`

func main() {
    if len(os.Args) > 1 && os.Args[1] == "serve" {
        fs := flag.NewFlagSet("serve", flag.ExitOnError)
        addr := fs.String("addr", "127.0.0.1:5353", "UDP listen address")
        trace := fs.Bool("trace", false, "print full resolution trace for each query")
        fs.Parse(os.Args[2:])
        r := NewResolver()
        if *trace {
            r.Trace = os.Stderr
        }
        if err := serve(*addr, r); err != nil {
            fmt.Fprintln(os.Stderr, "serve:", err)
            os.Exit(1)
        }
        return
    }

    qt := flag.String("t", "A", "record type")
    quiet := flag.Bool("q", false, "quiet")
    nocolor := flag.Bool("nocolor", false, "disable colour")
    flag.Usage = func() { fmt.Fprint(os.Stderr, usage) }
    flag.Parse()
    if flag.NArg() == 0 {
        flag.Usage()
        os.Exit(2)
    }
```

```
    qtype, err := ParseType(*qt)
    if err != nil {
        fmt.Fprintln(os.Stderr, err)
        os.Exit(2)
    }

    r := NewResolver()
    if !*quiet {
        r.Trace = os.Stdout
    }
    if fi, _ := os.Stdout.Stat(); !*nocolor && fi != nil && fi.Mode()&os.ModeCharDevice != 0 {
        r.color = true
    }

    exit := 0
    for i, name := range flag.Args() {
        if i > 0 {
            fmt.Println()
        }
        fmt.Println(r.c(cBold, fmt.Sprintf(";; resolving %s %s", fqdn(name), qtype)))
        res, err := r.Resolve(name, qtype)
        if err != nil {
            fmt.Println(r.c(cRed, ";; error: "+err.Error()))
            exit = 1
            continue
        }
        fmt.Printf("\n;; %s · %d hops · %s\n", rcodeString(res.Rcode), res.Hops,
res.Elapsed.Round(1e6))
        for _, rr := range res.Answers {
            fmt.Println(rr)
        }
        if len(res.Answers) == 0 {
            for _, rr := range res.Authority {
                fmt.Println(r.c(cDim, rr.String()))
            }
        }
    }
    os.Exit(exit)
}
```

A.5 resolver_test.go

```
package main

// resolver_test.go — a fake DNS hierarchy on loopback (127.0.0.10–15) so the
// full iterative walk can be tested offline and deterministically:
//
//   .            127.0.0.10  → delegates com., org.
//   com.         127.0.0.11  → delegates example.com. (with glue), glueless.com. (no glue)
//   example.com. 127.0.0.12  → authoritative: www, alias (CNAME), mail (MX), big (TC→TCP)
//   org.         127.0.0.13  → delegates dnshost.org.
//   dnshost.org. 127.0.0.14  → ns.dnshost.org A 127.0.0.15
//   glueless.com 127.0.0.15  → authoritative, its NS lives under .org

import (
    "bytes"
    "encoding/binary"
    "io"
    "net"
    "strings"
    "testing"
    "time"
)

const testPort = "15353"

type fakeZone struct {
    origin  string
    records []RR
    deleg   map[string][]RR
    glue    []RR
```

```
    tcOnUDP map[string]bool
}

func a(name, ip string) RR {
    return RR{Name: name, Type: TypeA, Class: 1, TTL: 300, Data: ip, Raw: net.ParseIP(ip).To4()}
}
func ns(zone, host string) RR { return RR{Name: zone, Type: TypeNS, Class: 1, TTL: 3600, Data:
host} }

func (z *fakeZone) answer(q Question, udp bool) []byte {
    resp := func(rcode int, aa, tc bool, an, au, ad []RR) []byte {
        b := buildResponse(0, q, false, rcode, an, au, ad)
        if aa {
            b[2] |= 0x04
        }
        if tc {
            b[2] |= 0x02
        }
        return b
    }
    soa := RR{Name: z.origin, Type: TypeSOA, Class: 1, TTL: 60,
        Data: "ns." + z.origin + " admin." + z.origin + " 1 7200 900 1209600 60"}

    for child, nsrrs := range z.deleg {
        if isSubdomain(q.Name, child) {
            return resp(0, false, false, nil, nsrrs, z.glue)
        }
    }
    var exists bool
    var an []RR
    for _, rr := range z.records {
        if rr.Name != q.Name {
            continue
        }
        exists = true
        if rr.Type == q.Type || rr.Type == TypeCNAME {
            an = append(an, rr)
        }
    }
    if !exists {
        return resp(RcodeNXDomain, true, false, nil, []RR{soa}, nil)
    }
    if udp && z.tcOnUDP[q.Name] {
        return resp(0, true, true, nil, nil, nil)
    }
    if len(an) == 0 {
        return resp(0, true, false, nil, []RR{soa}, nil)
    }
    return resp(0, true, false, an, nil, nil)
}

func startFake(t *testing.T, ip string, z *fakeZone) {
    t.Helper()
    addr := net.JoinHostPort(ip, testPort)
    pc, err := net.ListenPacket("udp", addr)
    if err != nil {
        t.Skipf("cannot bind %s: %v", addr, err)
    }
    ln, err := net.Listen("tcp", addr)
    if err != nil {
        t.Skipf("cannot bind tcp %s: %v", addr, err)
    }
    t.Cleanup(func() { pc.Close(); ln.Close() })

    reply := func(pkt []byte, udp bool) []byte {
        m, err := ParseMessage(pkt)
        if err != nil || len(m.Questions) != 1 {
            return nil
        }
        out := z.answer(m.Questions[0], udp)
        binary.BigEndian.PutUint16(out, m.Header.ID)
        return out
    }
    go func() {
        buf := make([]byte, 1500)
        for {
```

```
            n, from, err := pc.ReadFrom(buf)
            if err != nil {
                return
            }
            if out := reply(buf[:n], true); out != nil {
                pc.WriteTo(out, from)
            }
        }
    }()
    go func() {
        for {
            c, err := ln.Accept()
            if err != nil {
                return
            }
            go func(c net.Conn) {
                defer c.Close()
                var lb [2]byte
                if _, err := io.ReadFull(c, lb[:]); err != nil {
                    return
                }
                pkt := make([]byte, binary.BigEndian.Uint16(lb[:]))
                io.ReadFull(c, pkt)
                out := reply(pkt, false)
                c.Write(append(binary.BigEndian.AppendUint16(nil, uint16(len(out))), out...))
            }(c)
        }
    }()
}

func newTestResolver(t *testing.T) (*Resolver, *bytes.Buffer) {
    startFake(t, "127.0.0.10", &fakeZone{origin: ".",
        deleg: map[string][]RR{
            "com.": {ns("com.", "ns.gtld.com.")},
            "org.": {ns("org.", "ns.org.")},
        },
        glue: []RR{a("ns.gtld.com.", "127.0.0.11"), a("ns.org.", "127.0.0.13")},
    })
    startFake(t, "127.0.0.11", &fakeZone{origin: "com.",
        deleg: map[string][]RR{
            "example.com.":  {ns("example.com.", "ns1.example.com.")},
            "glueless.com.": {ns("glueless.com.", "ns.dnshost.org.")},
            "trap.com.":     {ns("net.", "ns.evil.net.")},
        },
        glue: []RR{
            a("ns1.example.com.", "127.0.0.12"),
            a("ns.dnshost.org.", "6.6.6.6"),
        },
    })
    startFake(t, "127.0.0.12", &fakeZone{origin: "example.com.",
        records: []RR{
            a("www.example.com.", "93.184.216.34"),
            {Name: "alias.example.com.", Type: TypeCNAME, Class: 1, TTL: 300, Data:
"www.example.com."},
            {Name: "example.com.", Type: TypeMX, Class: 1, TTL: 300, Data: "10
mail.example.com."},
            a("mail.example.com.", "93.184.216.35"),
            a("big.example.com.", "10.9.9.9"),
        },
        tcOnUDP: map[string]bool{"big.example.com.": true},
    })
    startFake(t, "127.0.0.13", &fakeZone{origin: "org.",
        deleg: map[string][]RR{"dnshost.org.": {ns("dnshost.org.", "ns.dnshost.org.")}},
        glue:  []RR{a("ns.dnshost.org.", "127.0.0.14")},
    })
    startFake(t, "127.0.0.14", &fakeZone{origin: "dnshost.org.",
        records: []RR{a("ns.dnshost.org.", "127.0.0.15")},
    })
    startFake(t, "127.0.0.15", &fakeZone{origin: "glueless.com.",
        records: []RR{a("glueless.com.", "10.0.0.1")},
    })


    r := NewResolver()
    r.Port = testPort
    r.Timeout = 500 * time.Millisecond
    r.Roots = []nameserver{{"fake-root.", "127.0.0.10"}}
    var trace bytes.Buffer
    r.Trace = &trace
```

```
    return r, &trace
}

func TestReferralChain(t *testing.T) {
    r, trace := newTestResolver(t)
    res, err := r.Resolve("www.example.com", TypeA)
    if err != nil {
        t.Fatal(err)
    }
    if len(res.Answers) != 1 || res.Answers[0].Data != "93.184.216.34" || res.Hops != 3 {
        t.Fatalf("got %+v\n%s", res, trace)
    }
    for _, want := range []string{"referral to com.", "referral to example.com.", "answered 1 A"}
{
        if !strings.Contains(trace.String(), want) {
            t.Errorf("trace missing %q:\n%s", want, trace)
        }
    }
}

func TestDelegationCache(t *testing.T) {
    r, trace := newTestResolver(t)
    r.Resolve("www.example.com", TypeA)
    trace.Reset()
    res, err := r.Resolve("mail.example.com", TypeA)
    if err != nil || res.Hops != 1 {
        t.Fatalf("expected 1 hop via cached example.com. delegation, got %+v err=%v\n%s", res,
err, trace)
    }
    if strings.Contains(trace.String(), "fake-root") {
        t.Errorf("should not have touched the root:\n%s", trace)
    }
}

func TestCNAME(t *testing.T) {
    r, trace := newTestResolver(t)
    res, err := r.Resolve("alias.example.com", TypeA)
    if err != nil {
        t.Fatal(err)
    }
    if len(res.Answers) != 2 || res.Answers[0].Type != TypeCNAME || res.Answers[1].Data !=
"93.184.216.34" {
        t.Fatalf("got %+v\n%s", res.Answers, trace)
    }
}

func TestNXDomainAndNoData(t *testing.T) {
    r, _ := newTestResolver(t)
    res, err := r.Resolve("nope.example.com", TypeA)
    if err != nil || res.Rcode != RcodeNXDomain || len(res.Authority) == 0 ||
res.Authority[0].Type != TypeSOA {
        t.Fatalf("NXDOMAIN: %+v %v", res, err)
    }
    res, err = r.Resolve("www.example.com", TypeAAAA)
    if err != nil || res.Rcode != RcodeNoError || len(res.Answers) != 0 {
        t.Fatalf("NODATA: %+v %v", res, err)
    }
}

func TestMX(t *testing.T) {
    r, _ := newTestResolver(t)
    res, err := r.Resolve("example.com", TypeMX)
    if err != nil || len(res.Answers) != 1 || res.Answers[0].Data != "10 mail.example.com." {
        t.Fatalf("%+v %v", res, err)
    }
}

func TestGluelessDelegationIgnoresPoisonedGlue(t *testing.T) {
    r, trace := newTestResolver(t)
    res, err := r.Resolve("glueless.com", TypeA)
    if err != nil || len(res.Answers) != 1 || res.Answers[0].Data != "10.0.0.1" {
        t.Fatalf("%+v %v\n%s", res, err, trace)
    }
    if strings.Contains(trace.String(), "6.6.6.6") {
        t.Errorf("used out-of-bailiwick glue:\n%s", trace)
    }
```

```
    if !strings.Contains(trace.String(), "no glue") {
        t.Errorf("expected glueless path:\n%s", trace)
    }
}

func TestOutOfBailiwickReferralRejected(t *testing.T) {
    r, _ := newTestResolver(t)
    _, err := r.Resolve("www.trap.com", TypeA)
    if err == nil || !strings.Contains(err.Error(), "bailiwick") {
        t.Fatalf("expected bailiwick error, got %v", err)
    }
}

func TestTruncationFallsBackToTCP(t *testing.T) {
    r, trace := newTestResolver(t)
    res, err := r.Resolve("big.example.com", TypeA)
    if err != nil || len(res.Answers) != 1 || res.Answers[0].Data != "10.9.9.9" {
        t.Fatalf("%+v %v\n%s", res, err, trace)
    }
    if !strings.Contains(trace.String(), "TCP") {
        t.Errorf("expected TCP retry in trace:\n%s", trace)
    }
}

func TestNameCompression(t *testing.T) {
    pkt := []byte{
        0x12, 0x34, 0x81, 0x80, 0, 1, 0, 1, 0, 0, 0, 0,
        3, 'w', 'w', 'w', 7, 'e', 'x', 'a', 'm', 'p', 'l', 'e', 3, 'c', 'o', 'm', 0,
        0, 1, 0, 1,
        0xC0, 0x0C, 0, 1, 0, 1, 0, 0, 0x0E, 0x10, 0, 4, 1, 2, 3, 4,
    }
    m, err := ParseMessage(pkt)
    if err != nil || m.Answers[0].Name != "www.example.com." || m.Answers[0].Data != "1.2.3.4" {
        t.Fatalf("%+v %v", m, err)
    }
    loop := append(append([]byte{}, pkt[:12]...), 0xC0, 0x0C)
    if _, err := ParseMessage(loop); err == nil {
        t.Fatal("expected loop error")
    }
}
```

A.6 go.mod

```
module github.com/swayam5342/dnsr

go 1.22
```

A.7 fuzz_test.go (evaluation harness, Chapter 7)

```
package main

import (
    "math/rand"
    "testing"
)

func seedPackets() [][]byte {
    q, _ := BuildQuery(0x1234, "www.example.com.", TypeA)
    resp := []byte{
        0x12, 0x34, 0x81, 0x80, 0, 1, 0, 1, 0, 0, 0, 0,
        3, 'w', 'w', 'w', 7, 'e', 'x', 'a', 'm', 'p', 'l', 'e', 3, 'c', 'o', 'm', 0,
        0, 1, 0, 1,
        0xC0, 0x0C, 0, 1, 0, 1, 0, 0, 0x0E, 0x10, 0, 4, 1, 2, 3, 4,
    }
    return [][]byte{q, resp}
}
```

```
func FuzzParseMessage(f *testing.F) {
    for _, s := range seedPackets() {
        f.Add(s)
    }
    f.Fuzz(func(t *testing.T, b []byte) {
        ParseMessage(b)
    })
}

func TestRandomAndMutatedPackets(t *testing.T) {
    rng := rand.New(rand.NewSource(42))
    seeds := seedPackets()
    errs, oks := 0, 0
    for i := 0; i < 1_000_000; i++ {
        var b []byte
        if i%2 == 0 {
            b = make([]byte, rng.Intn(600))
            rng.Read(b)
        } else {
            s := seeds[rng.Intn(len(seeds))]
            b = append([]byte(nil), s...)
            for k := 0; k < 1+rng.Intn(6); k++ {
                b[rng.Intn(len(b))] = byte(rng.Intn(256))
            }
            if rng.Intn(4) == 0 {
                b = b[:rng.Intn(len(b))]
            }
        }
        if _, err := ParseMessage(b); err != nil {
            errs++
        } else {
            oks++
        }
    }
    t.Logf("1,000,000 packets: %d rejected with error, %d parsed, 0 panics", errs, oks)
}

func BenchmarkParseMessage(b *testing.B) {
    p := seedPackets()[1]
    for i := 0; i < b.N; i++ {
        ParseMessage(p)
    }
}

func BenchmarkBuildQuery(b *testing.B) {
    for i := 0; i < b.N; i++ {
        BuildQuery(uint16(i), "www.example.com.", TypeA)
    }
}
```

APPENDIX B

USER MANUAL

B.1 Building

```
go build -o dnsr .        # Linux / macOS
go build -o dnsr.exe .    # Windows
go test -v ./...          # run the offline test suite
```

B.2 Resolving names

```
./dnsr github.com
./dnsr -t MX gmail.com
./dnsr -t AAAA www.wikipedia.org vit.ac.in   # several names share one cache
./dnsr -q -t TXT google.com                  # answer only, no trace
./dnsr -nocolor www.github.com               # plain output for logs
```

B.3 Running the recursive server

```
./dnsr serve -addr 127.0.0.1:8053            # Windows: 5353 is used by mDNS
./dnsr serve -addr 127.0.0.1:8053 -trace     # print a full trace per query
dig @127.0.0.1 -p 8053 example.com           # query it from another terminal
```

B.4 Requirements and troubleshooting

- ✓ Outbound UDP and TCP port 53 must be allowed. Some campus and corporate networks block it and force their own resolver; if every root server times out, try another network such as a phone hotspot.
- ✓ On Windows, choose a server port other than 5353, which is reserved by the multicast DNS service.
- ✓ The Windows `nslookup` tool does not honour a custom port; use `dig` or a script based on dnspython to query the server.
- ✓ Exit status 0 means every name resolved; 1 means at least one resolution failed; 2 means a usage error.